

File Reference: script.js

A file-specific explanation covering purpose, owner layer, metadata, source details, implementation signals, source excerpts, and safe-change rules.

Item	Explanation
File	script.js
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	158,556 bytes
Extension	.js
MIME type	application/javascript
Text lines	3,710

Why this file exists

- Public website JavaScript for navigation, portfolio rendering, search, AI UI, and interactive project windows.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/portfolio-ai	Line 1492. Handles AI assistant questions.
/api/portfolio-ai	Line 1493. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Browser DOM at line 1	Builder or website interface behavior in the browser/Electron renderer. Evidence: const grid = document.querySelector("#project-grid");
Browser DOM at line 2	Builder or website interface behavior in the browser/Electron renderer. Evidence: const searchInput = document.querySelector("#project-search");
Browser DOM at line 3	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectFilters = document.querySelector("#project-filters");
Browser DOM at line 4	Builder or website interface behavior in the browser/Electron renderer. Evidence: let filterButtons = [...document.querySelectorAll(".filter-button")];
Browser DOM at line 5	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCount = document.querySelector("#project-count");
Browser DOM at line 6	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectTrackCount = document.querySelector("#project-track-count");
Browser DOM at line 7	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectTrackLabels = document.querySelector("#project-track-labels");
Browser DOM at line 8	Builder or website interface behavior in the browser/Electron renderer. Evidence: const year = document.querySelector("#year");

Item	Explanation
Browser DOM at line 9	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const funFactsCallout = document.querySelector("#fun-facts-callout");</code>
Browser DOM at line 10	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const heroEyebrow = document.querySelector(".hero-content .eyebrow");</code>
Browser DOM at line 11	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const heroTitle = document.querySelector("#hero-title");</code>
Browser DOM at line 12	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const heroCopy = document.querySelector(".hero-copy");</code>
Browser DOM at line 13	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandName = document.querySelector(".brand strong");</code>
Browser DOM at line 14	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandSubtitle = document.querySelector(".brand small");</code>
Browser DOM at line 15	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandIcon = document.querySelector(".brand-icon");</code>
Browser DOM at line 16	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandText = document.querySelector(".omb-engraving");</code>
Browser DOM at line 17	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const headerAvatar = document.querySelector(".header-avatar");</code>
Browser DOM at line 18	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const headerAvatarImage = document.querySelector(".header-avatar img");</code>
Browser DOM at line 19	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const heroImage = document.querySelector(".hero-image");</code>
Browser DOM at line 20	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const resumeSection = document.querySelector("#resume");</code>
Browser DOM at line 21	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactBand = document.querySelector("#contact");</code>
Browser DOM at line 22	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profilePhoto = document.querySelector(".profile-photo");</code>
Browser DOM at line 23	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactTitle = document.querySelector("#contact-title");</code>
Browser DOM at line 24	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactIntro = document.querySelector(".contact-intro");</code>
Browser DOM at line 25	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactDetails = document.querySelector(".contact-details");</code>
Browser DOM at line 26	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactLinks = document.querySelector(".contact-links");</code>
Browser DOM at line 27	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const footerOwner = document.querySelector(".site-footer span");</code>
Browser DOM at line 29	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantForm = document.querySelector("#ai-assistant-form");</code>
Browser DOM at line 30	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantPanel = document.querySelector("#ask-ai");</code>
Browser DOM at line 31	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantInput = document.querySelector("#ai-assistant-input");</code>
Browser DOM at line 32	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantLog = document.querySelector("#ai-assistant-log");</code>
Browser DOM at line 33	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantStatus = document.querySelector("#ai-assistant-status");</code>
Cloudflare or AI at line 34	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>const aiClearChatButton = document.querySelector("#ai-clear-chat");</code>
Cloudflare or AI at line 57	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>let assistantChatHistory = [];</code>
Compiler or simulator at line 73	Part of the Compile Code workspace or language/tool detection. Evidence: <code>{ id: "verilog", label: "Verilog", aliases: ["verilog", "v"] },</code>

Item	Explanation
Compiler or simulator at line 74	Part of the Compile Code workspace or language/tool detection. Evidence: { id: "systemverilog", label: "SystemVerilog", aliases: ["systemverilog", "system verilog", "sv"] },
Compiler or simulator at line 76	Part of the Compile Code workspace or language/tool detection. Evidence: { id: "java", label: "Java", aliases: ["java"] },
Compiler or simulator at line 78	Part of the Compile Code workspace or language/tool detection. Evidence: { id: "python", label: "Python", aliases: ["python", "py"] },
Compiler or simulator at line 104	Part of the Compile Code workspace or language/tool detection. Evidence: "embedded-wireless-node": "appearance-deep-navy-cyan-click",
Browser DOM at line 175	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-draggable-dialog");
Browser DOM at line 184	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (dialog.querySelector("[data-section-dialog-resize-handle]")) return;
Browser DOM at line 186	Builder or website interface behavior in the browser/Electron renderer. Evidence: const handle = document.createElement("div");
Browser DOM at line 188	Builder or website interface behavior in the browser/Electron renderer. Evidence: handle.dataset.sectionDialogResizeHandle = direction;
Browser DOM at line 197	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-resized-dialog");
Browser DOM at line 255	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-dragging-dialog");
Browser DOM at line 273	Builder or website interface behavior in the browser/Electron renderer. Evidence: activeSectionDialogDrag.dialog.classList.remove("is-dragging-dialog");
Browser DOM at line 281	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (!handle !dialog?.open dialog.classList.contains("is-minimized-dialog")) return;
Browser DOM at line 285	Builder or website interface behavior in the browser/Electron renderer. Evidence: direction: handle.dataset.sectionDialogResizeHandle,
Browser DOM at line 293	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-resizing-dialog");
Browser DOM at line 309	Builder or website interface behavior in the browser/Electron renderer. Evidence: activeSectionDialogResize.dialog.classList.remove("is-resizing-dialog");
Browser DOM at line 314	Builder or website interface behavior in the browser/Electron renderer. Evidence: const minimized = dialog.classList.contains("is-minimized-dialog");
Browser DOM at line 315	Builder or website interface behavior in the browser/Electron renderer. Evidence: const button = dialog.querySelector(".section-view-minimize");
Browser DOM at line 324	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.toggle("is-minimized-dialog");
Browser DOM at line 331	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerdown", beginSectionDialogResize);
Browser DOM at line 332	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerdown", beginSectionDialogDrag);
Browser DOM at line 333	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointermove", moveSectionDialogResize);
Browser DOM at line 334	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointermove", moveSectionDialogDrag);
Browser DOM at line 335	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerup", endSectionDialogResize);
Browser DOM at line 336	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerup", endSectionDialogDrag);
Browser DOM at line 337	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointercancel", endSectionDialogResize);
Browser DOM at line 338	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointercancel", endSectionDialogDrag);
Browser DOM at line 339	Builder or website interface behavior in the browser/Electron renderer. Evidence: window.addEventListener("resize", () => {

Item	Explanation
Browser DOM at line 340	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>document.querySelectorAll(".section-view-dialog.is-draggable-dialog[open]").forEach((dialog) => {</code>
Rich text at line 370	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <code>function normalizeCodePasteMode(value = "") {</code>
Rich text at line 374	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <code>function normalizeCodeText(value = "", pasteMode = "source") {</code>
Rich text at line 376	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <code>if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/t/g, " ").replace(/s+\$/g, "");</code>
Compiler or simulator at line 400	Part of the Compile Code workspace or language/tool detection. Evidence: <code>if (/^(\b(always_ff always_comb always_latch logic interface covergroup assert s+property typedef s+enum class s+ w+) \b/.test(code)) return "systemverilog";</code>
Compiler or simulator at line 401	Part of the Compile Code workspace or language/tool detection. Evidence: <code>if (/^(\b(module endmodule always assign reg wire initial posedge negedge) \b/.test(code)) return "verilog";</code>
Cloudflare or AI at line 402	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>if (/^(\s*\.(?!(tran ac dc op model subckt ends param) \b/im.test(code) \bV w+ s+ w+ s+ w+ s+(?:DC SIN PULSE)?/i.test(code)) return "Itspice";</code>
Compiler or simulator at line 403	Part of the Compile Code workspace or language/tool detection. Evidence: <code>if (/^(\b(def elif import s+ w+ from s+ w+ s+import self None True False) \b/.test(code)) return "python";</code>
Compiler or simulator at line 404	Part of the Compile Code workspace or language/tool detection. Evidence: <code>if (/^(\b(public s+class private s+ protected s+ static s+void s+main System\.out) \b/.test(code)) return "java";</code>
Security or auth at line 412	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>function tokenizedCodeHtml(code = "", language = "javascript") {</code>
Security or auth at line 414	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>const tokens = [];</code>
Security or auth at line 417	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>const token = `@@CODE_TOKEN_\${tokens.length}@`;</code>
Security or auth at line 418	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>tokens.push(`<span class="\${className}">\${match}</span>`);</code>
Security or auth at line 419	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>return token;</code>
Security or auth at line 424	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>protect(/&lt;!--[\s\S]*?-->/g, "code-token-comment");</code>
Security or auth at line 425	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>protect(/&lt;\/?([a-zA-Z0-9-:~+])([\\s\S]*?)(\/?&gt;)/g, "code-token-tag");</code>
Compiler or simulator at line 427	Part of the Compile Code workspace or language/tool detection. Evidence: <code>const commentPattern = ["python", "Itspice"].includes(normalizeCodeLanguage(language))</code>
Security or auth at line 430	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <code>protect(commentPattern, "code-token-comment");</code>

Important constants and variables

Item	Explanation
grid	Line 1: <code>const grid = document.querySelector("#project-grid");</code>
searchInput	Line 2: <code>const searchInput = document.querySelector("#project-search");</code>
projectFilters	Line 3: <code>const projectFilters = document.querySelector("#project-filters");</code>
filterButtons	Line 4: <code>let filterButtons = [...document.querySelectorAll(".filter-button")];</code>
projectCount	Line 5: <code>const projectCount = document.querySelector("#project-count");</code>
projectTrackCount	Line 6: <code>const projectTrackCount = document.querySelector("#project-track-count");</code>
projectTrackLabels	Line 7: <code>const projectTrackLabels = document.querySelector("#project-track-labels");</code>
year	Line 8: <code>const year = document.querySelector("#year");</code>
funFactsCallout	Line 9: <code>const funFactsCallout = document.querySelector("#fun-facts-callout");</code>
heroEyebrow	Line 10: <code>const heroEyebrow = document.querySelector(".hero-content .eyebrow");</code>
heroTitle	Line 11: <code>const heroTitle = document.querySelector("#hero-title");</code>

Item	Explanation
heroCopy	Line 12: const heroCopy = document.querySelector(".hero-copy");
brandName	Line 13: const brandName = document.querySelector(".brand strong");
brandSubtitle	Line 14: const brandSubtitle = document.querySelector(".brand small");
brandIcon	Line 15: const brandIcon = document.querySelector(".brand-icon");
brandText	Line 16: const brandText = document.querySelector(".omb-engraving");
headerAvatar	Line 17: const headerAvatar = document.querySelector(".header-avatar");
headerAvatarImage	Line 18: const headerAvatarImage = document.querySelector(".header-avatar img");
heroImage	Line 19: const heroImage = document.querySelector(".hero-image");
resumeSection	Line 20: const resumeSection = document.querySelector("#resume");
contactBand	Line 21: const contactBand = document.querySelector("#contact");
profilePhoto	Line 22: const profilePhoto = document.querySelector(".profile-photo");
contactTitle	Line 23: const contactTitle = document.querySelector("#contact-title");
contactIntro	Line 24: const contactIntro = document.querySelector(".contact-intro");
contactDetails	Line 25: const contactDetails = document.querySelector(".contact-details");
contactLinks	Line 26: const contactLinks = document.querySelector(".contact-links");
footerOwner	Line 27: const footerOwner = document.querySelector(".site-footer span");
searchWrap	Line 28: const searchWrap = searchInput?.closest(".search-wrap");
aiAssistantForm	Line 29: const aiAssistantForm = document.querySelector("#ai-assistant-form");
aiAssistantPanel	Line 30: const aiAssistantPanel = document.querySelector("#ask-ai");
aiAssistantInput	Line 31: const aiAssistantInput = document.querySelector("#ai-assistant-input");
aiAssistantLog	Line 32: const aiAssistantLog = document.querySelector("#ai-assistant-log");
aiAssistantStatus	Line 33: const aiAssistantStatus = document.querySelector("#ai-assistant-status");
aiClearChatButton	Line 34: const aiClearChatButton = document.querySelector("#ai-clear-chat");
categories	Line 36: let categories = [];
projects	Line 37: let projects = [];
siteSections	Line 38: let siteSections = [];
funFacts	Line 39: let funFacts = [];
funFactsRich	Line 40: let funFactsRich = null;
fieldStyles	Line 41: let fieldStyles = {};
siteContent	Line 42: let siteContent = null;
siteContentRich	Line 43: let siteContentRich = {};
profile	Line 44: let profile = null;
profileRich	Line 45: let profileRich = {};
activeFilter	Line 46: let activeFilter = "all";
activeSectionDialogDrag	Line 47: let activeSectionDialogDrag = null;
activeSectionDialogResize	Line 48: let activeSectionDialogResize = null;
sectionDialogDragEnabled	Line 49: let sectionDialogDragEnabled = false;
isApplyingSectionRoute	Line 50: let isApplyingSectionRoute = false;
searchPanel	Line 51: let searchPanel = null;
searchStatus	Line 52: let searchStatus = null;
searchLimitSelect	Line 53: let searchLimitSelect = null;

Item	Explanation
currentSearchResults	Line 54: let currentSearchResults = [];
assistantSourceCounter	Line 55: let assistantSourceCounter = 0;
assistantSourceMap	Line 56: let assistantSourceMap = new Map();
assistantChatHistory	Line 57: let assistantChatHistory = [];
searchableEntries	Line 58: let searchableEntries = [];
searchResultLimit	Line 59: let searchResultLimit = 10;
searchHighlightTimer	Line 60: let searchHighlightTimer = 0;
searchHighlightName	Line 61: const searchHighlightName = "portfolio-search-highlight";
sectionRouteKey	Line 63: const sectionRouteKey = "portfolio-section";
sectionRouteParams	Line 64: const sectionRouteParams = {
supportedCodeLanguages	Line 70: const supportedCodeLanguages = [
legacyTemplateSkins	Line 82: const legacyTemplateSkins = {
defaultSiteContent	Line 108: const defaultSiteContent = {
defaultProfile	Line 114: const defaultProfile = {
items	Line 137: const items = Array.isArray(value) ? value : String(value "").split(/\r?\n/);
facts	Line 146: const facts = normalizeFunFacts(funFacts);
hasRichFacts	Line 147: const hasRichFacts = Boolean(funFactsRich?.blocks?.length);
rect	Line 162: const rect = dialog.getBoundingClientRect();
margin	Line 163: const margin = 12;
maxLeft	Line 164: const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);
maxTop	Line 165: const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);
rect	Line 173: const rect = dialog.getBoundingClientRect();
position	Line 174: const position = clampSectionDialogPosition(rect.left, rect.top, dialog);
handle	Line 186: const handle = document.createElement("div");
rect	Line 196: const rect = dialog.getBoundingClientRect();
margin	Line 204: const margin = 12;
minWidth	Line 205: const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
minHeight	Line 206: const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));

JSON/YAML-style keys detected

Item	Explanation
skin-light-blue	Line 83: "skin-light-blue": "appearance-light-blue-red-click",
skin-clean-white	Line 84: "skin-clean-white": "appearance-white-blue-click",
skin-deep-navy	Line 85: "skin-deep-navy": "appearance-deep-navy-cyan-click",
skin-red-warm	Line 86: "skin-red-warm": "appearance-warm-red-pale-click",
skin-emerald-instrument	Line 87: "skin-emerald-instrument": "appearance-emerald-mint-click",
skin-amber-power	Line 88: "skin-amber-power": "appearance-amber-cream-click",
skin-violet-mixed	Line 89: "skin-violet-mixed": "appearance-violet-lilac-click",
skin-graphite-asic	Line 90: "skin-graphite-asic": "appearance-graphite-white-click",
analog-opamp-topology	Line 91: "analog-opamp-topology": "appearance-light-blue-red-click",
analog-power-charger	Line 92: "analog-power-charger": "appearance-amber-cream-click",

Item	Explanation
analog-filter-front-end	Line 93: "analog-filter-front-end": "appearance-light-blue-red-click",
analog-sensor-interface	Line 94: "analog-sensor-interface": "appearance-emerald-mint-click",
analog-mixed-signal-timing	Line 95: "analog-mixed-signal-timing": "appearance-violet-lilac-click",
digital-fpga-pipeline	Line 96: "digital-fpga-pipeline": "appearance-deep-navy-cyan-click",
digital-asic-block	Line 97: "digital-asic-block": "appearance-graphite-white-click",
digital-verification-suite	Line 98: "digital-verification-suite": "appearance-graphite-white-click",
digital-interface-controller	Line 99: "digital-interface-controller": "appearance-deep-navy-cyan-click",
digital-signal-processing	Line 100: "digital-signal-processing": "appearance-violet-lilac-click",
embedded-stm32-sensor	Line 101: "embedded-stm32-sensor": "appearance-emerald-mint-click",
embedded-rtos-control	Line 102: "embedded-rtos-control": "appearance-graphite-white-click",
embedded-power-monitor	Line 103: "embedded-power-monitor": "appearance-amber-cream-click",
embedded-wireless-node	Line 104: "embedded-wireless-node": "appearance-deep-navy-cyan-click",
embedded-motor-control	Line 105: "embedded-motor-control": "appearance-violet-lilac-click"
Brief	Line 792: "Brief": "Overview",
Project Brief	Line 793: "Project Brief": "Overview",
Design Brief	Line 794: "Design Brief": "Design Overview",
Simulation Brief	Line 795: "Simulation Brief": "Simulation Overview"
--responsive-section-card-min	Line 1029: "--responsive-section-card-min": profile.sectionCardMin,
--responsive-content-max	Line 1030: "--responsive-content-max": profile.contentMax,
--responsive-media-max	Line 1031: "--responsive-media-max": profile.mediaMax,
--responsive-touch-target	Line 1032: "--responsive-touch-target": profile.touchTarget
--accent	Line 1047: "--accent": accent,
--template-bg	Line 1048: "--template-bg": visual.background,
--template-panel	Line 1049: "--template-panel": visual.panel,
--template-accent	Line 1050: "--template-accent": visual.accent,
--template-hover	Line 1051: "--template-hover": visual.hover,
--template-click	Line 1052: "--template-click": visual.click,
--template-click-text	Line 1053: "--template-click-text": visual.clickText,
--template-line	Line 1054: "--template-line": visual.line,
--template-text	Line 1055: "--template-text": visual.text,
--accent	Line 1080: "--accent": accent,
--template-bg	Line 1081: "--template-bg": visual.background,
--template-panel	Line 1082: "--template-panel": visual.panel,
--template-accent	Line 1083: "--template-accent": visual.accent,
--template-hover	Line 1084: "--template-hover": visual.hover,
--template-click	Line 1085: "--template-click": visual.click,
--template-click-text	Line 1086: "--template-click-text": visual.clickText,
--template-line	Line 1087: "--template-line": visual.line,
--template-text	Line 1088: "--template-text": visual.text,

Event handlers and user interactions

Item	Explanation
Line 331	<code>document.addEventListener("pointerdown", beginSectionDialogResize);</code>
Line 332	<code>document.addEventListener("pointerdown", beginSectionDialogDrag);</code>
Line 333	<code>document.addEventListener("pointermove", moveSectionDialogResize);</code>
Line 334	<code>document.addEventListener("pointermove", moveSectionDialogDrag);</code>
Line 335	<code>document.addEventListener("pointerup", endSectionDialogResize);</code>
Line 336	<code>document.addEventListener("pointerup", endSectionDialogDrag);</code>
Line 337	<code>document.addEventListener("pointercancel", endSectionDialogResize);</code>
Line 338	<code>document.addEventListener("pointercancel", endSectionDialogDrag);</code>
Line 339	<code>window.addEventListener("resize", () => {</code>
Line 1214	<code>button.addEventListener("click", () => {</code>
Line 3272	<code>searchLimitSelect.addEventListener("change", () => {</code>
Line 3276	<code>searchPanel.addEventListener("mousedown", (event) => event.preventDefault());</code>
Line 3277	<code>searchPanel.addEventListener("click", (event) => {</code>
Line 3434	<code>dialog.querySelector(".section-view-close").addEventListener("click", () => closeSectionDialog(dialog));</code>
Line 3435	<code>dialog.addEventListener("cancel", () => clearSectionRouteInHistory());</code>
Line 3436	<code>dialog.addEventListener("close", () => {</code>
Line 3445	<code>dialog.querySelector(".section-view-back").addEventListener("click", () => {</code>
Line 3457	<code>dialog.querySelector(".section-view-forward").addEventListener("click", () => {</code>
Line 3464	<code>dialog.querySelector("#section-view-content").addEventListener("click", (event) => {</code>
Line 3562	<code>window.addEventListener("popstate", (event) => {</code>
Line 3567	<code>searchInput.addEventListener("input", handleSearchInput);</code>
Line 3568	<code>searchInput.addEventListener("focus", showSearchPanelIfNeeded);</code>
Line 3569	<code>searchInput.addEventListener("keydown", (event) => {</code>
Line 3580	<code>document.addEventListener("click", (event) => {</code>
Line 3586	<code>document.addEventListener("click", (event) => {</code>
Line 3599	<code>aiAssistantForm?.addEventListener("submit", (event) => {</code>
Line 3606	<code>aiClearChatButton?.addEventListener("click", clearAssistantChat);</code>
Line 3607	<code>window.addEventListener("resize", updateAssistantPanelGrowth);</code>
Line 3609	<code>aiAssistantLog?.addEventListener("click", (event) => {</code>
Line 3616	<code>grid.addEventListener("click", (event) => {</code>

Function-by-function detail

normalize (script.js, lines 132-134)

normalize validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 132-134 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 133: `return String(value || "").toLowerCase();`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
132: function normalize(value) {
133:   return String(value || "").toLowerCase();
134: }
```

normalizeFunFacts (script.js, lines 136-142)

normalizeFunFacts validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 136-142 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, split, map, replace, trim, filter, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 138: return items.

Important local values include items at line 137. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
136: function normalizeFunFacts(value) {
137:   const items = Array.isArray(value) ? value : String(value || "").split(/\r?\n/);
138:   return items
139:     .map((item) => String(item || "").replace(/\s+/g, " ").trim())
140:     .filter(Boolean)
141:     .slice(0, 10);
142: }
```

renderFunFacts (script.js, lines 144-159)

renderFunFacts renders ui, markup, or display output. In this file, it appears around lines 144-159 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeFunFacts, renderRichContent, slice, join, map, and renderInlineMath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 145: if (!funFactsCallout) return;.

Important local values include facts at line 146; hasRichFacts at line 147. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
144: function renderFunFacts() {
145:   if (!funFactsCallout) return;
146:   const facts = normalizeFunFacts(funFacts);
147:   const hasRichFacts = Boolean(funFactsRich?.blocks?.length);
148:   funFactsCallout.hidden = !facts.length && !hasRichFacts;
149:   funFactsCallout.innerHTML = facts.length || hasRichFacts ? `
150:     <div class="fun-facts-window">
151:       <span class="fun-facts-label">Fun fact:</span>
152:       <div class="fun-facts-lines">
153:         ${hasRichFacts
154:           ? renderRichContent({ blocks: funFactsRich.blocks.slice(0, 3) }, facts.join("\n"))
155:           : facts.map((fact) => `<p class="fun-fact-line">${renderInlineMath(fact)}</p>`).join("")}
156:       </div>
157:     </div>
158:   ` : "";
159: }
```

clampSectionDialogPosition (script.js, lines 161-170)

clampSectionDialogPosition part of the public website runtime. In this file, it appears around lines 161-170 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, max, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 166: return {.

Important local values include rect at line 162; margin at line 163; maxLeft at line 164; maxTop at line 165. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
161: function clampSectionDialogPosition(left, top, dialog) {
162:   const rect = dialog.getBoundingClientRect();
163:   const margin = 12;
164:   const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);
165:   const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);
166:   return {
167:     left: Math.min(Math.max(left, margin), maxLeft),
168:     top: Math.min(Math.max(top, margin), maxTop)
169:   };
170: }
```

anchorSectionDialog (script.js, lines 172-181)

anchorSectionDialog part of the public website runtime. In this file, it appears around lines 172-181 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, clampSectionDialogPosition, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include rect at line 173; position at line 174. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
172: function anchorSectionDialog(dialog) {
173:   const rect = dialog.getBoundingClientRect();
174:   const position = clampSectionDialogPosition(rect.left, rect.top, dialog);
175:   dialog.classList.add("is-draggable-dialog");
176:   dialog.style.left = `${position.left}px`;
177:   dialog.style.top = `${position.top}px`;
178:   dialog.style.right = "auto";
179:   dialog.style.bottom = "auto";
180:   dialog.style.margin = "0";
181: }
```

ensureSectionResizeHandles (script.js, lines 183-192)

ensureSectionResizeHandles part of the public website runtime. In this file, it appears around lines 183-192 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, forEach, createElement, setAttribute, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 184: if (dialog.querySelector("[data-section-dialog-resize-handle]")) return;.

Important local values include handle at line 186. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
183: function ensureSectionResizeHandles(dialog) {
184:   if (dialog.querySelector("[data-section-dialog-resize-handle]")) return;
185:   ["n", "e", "s", "w", "ne", "nw", "se", "sw"].forEach((direction) => {
186:     const handle = document.createElement("div");
187:     handle.className = `dialog-resize-handle resize-${direction}`;
188:     handle.dataset.sectionDialogResizeHandle = direction;
189:     handle.setAttribute("aria-hidden", "true");
190:     dialog.append(handle);
191:   });
192: }
```

anchorSectionDialogForResize (script.js, lines 194-201)

anchorSectionDialogForResize part of the public website runtime. In this file, it appears around lines 194-201 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references anchorSectionDialog, getBoundingClientRect, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include rect at line 196. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
194: function anchorSectionDialogForResize(dialog) {
195:   anchorSectionDialog(dialog);
196:   const rect = dialog.getBoundingClientRect();
197:   dialog.classList.add("is-resized-dialog");
198:   dialog.style.width = `${rect.width}px`;
199:   dialog.style.height = `${rect.height}px`;
200:   dialog.style.maxHeight = "none";
201: }
```

sectionDialogResizeBounds (script.js, lines 203-230)

sectionDialogResizeBounds part of the public website runtime. In this file, it appears around lines 203-230 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references min, max, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 229: return { height, left, top, width };

Important local values include margin at line 204; minWidth at line 205; minHeight at line 206; direction at line 207; dx at line 209; dy at line 210; right at line 219; bottom at line 224. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
203: function sectionDialogResizeBounds(state, event) {
204:   const margin = 12;
205:   const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
206:   const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));
207:   const direction = state.direction;
208:   let { left, top, width, height } = state.rect;
209:   const dx = event.clientX - state.startX;
210:   const dy = event.clientY - state.startY;
211:
212:   if (direction.includes("e")) {
213:     width = Math.min(Math.max(state.rect.width + dx, minWidth), window.innerWidth - left - margin);
214:   }
215:   if (direction.includes("s")) {
216:     height = Math.min(Math.max(state.rect.height + dy, minHeight), window.innerHeight - top - margin);
217:   }
218:   if (direction.includes("w")) {
219:     const right = state.rect.left + state.rect.width;
220:     left = Math.min(Math.max(state.rect.left + dx, margin), right - minWidth);
221:     width = right - left;
222:   }
223:   if (direction.includes("n")) {
224:     const bottom = state.rect.top + state.rect.height;
225:     top = Math.min(Math.max(state.rect.top + dy, margin), bottom - minHeight);
226:     height = bottom - top;
227:   }
228:
229:   return { height, left, top, width };
230: }
```

sectionDialogFromDragEvent (script.js, lines 232-239)

sectionDialogFromDragEvent part of the public website runtime. In this file, it appears around lines 232-239 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 233: if (event.target.closest("[data-section-dialog-resize-handle]")) return null;; line 235: if (!handle) return null;; line 236: if (event.target.closest("button, a, input, textarea, select, label, summary")) return null;; line 238: return dialog?.open ? dialog : null;.

Important local values include handle at line 234; dialog at line 237. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
232: function sectionDialogFromDragEvent(event) {
233:   if (event.target.closest("[data-section-dialog-resize-handle]")) return null;
234:   const handle = event.target.closest("[data-section-dialog-drag='true']");
235:   if (!handle) return null;
236:   if (event.target.closest("button, a, input, textarea, select, label, summary")) return null;
237:   const dialog = handle.closest("dialog");
238:   return dialog?.open ? dialog : null;
239: }
```

beginSectionDialogDrag (script.js, lines 241-257)

beginSectionDialogDrag part of the public website runtime. In this file, it appears around lines 241-257 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionDialogFromDragEvent, anchorSectionDialog, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 242: if (event.button !== 0) return;; line 244: if (!dialog) return;.

Important local values include dialog at line 243; rect at line 246. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
241: function beginSectionDialogDrag(event) {
242:   if (event.button !== 0) return;
243:   const dialog = sectionDialogFromDragEvent(event);
244:   if (!dialog) return;
245:   anchorSectionDialog(dialog);
246:   const rect = dialog.getBoundingClientRect();
247:   activeSectionDialogDrag = {
248:     dialog,
249:     offsetX: event.clientX - rect.left,
250:     offsetY: event.clientY - rect.top,
251:     pointerId: event.pointerId,
252:     pointerTarget: event.target
253:   };
254:   event.target.setPointerCapture?.(event.pointerId);
255:   dialog.classList.add("is-dragging-dialog");
256:   event.preventDefault();
257: }
```

moveSectionDialogDrag (script.js, lines 259-268)

moveSectionDialogDrag part of the public website runtime. In this file, it appears around lines 259-268 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clampSectionDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 260: if (!activeSectionDialogDrag) return;.

Important local values include next at line 261. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
259: function moveSectionDialogDrag(event) {
260:   if (!activeSectionDialogDrag) return;
261:   const next = clampSectionDialogPosition(
262:     event.clientX - activeSectionDialogDrag.offsetX,
263:     event.clientY - activeSectionDialogDrag.offsetY,
264:     activeSectionDialogDrag.dialog
265:   );
266:   activeSectionDialogDrag.dialog.style.left = `${next.left}px`;
```

```
267:   activeSectionDialogDrag.dialog.style.top = `${next.top}px`;
268: }
```

endSectionDialogDrag (script.js, lines 270-275)

endSectionDialogDrag part of the public website runtime. In this file, it appears around lines 270-275 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 271: if (!activeSectionDialogDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
270: function endSectionDialogDrag() {
271:   if (!activeSectionDialogDrag) return;
272:   activeSectionDialogDrag.pointerTarget?.releasePointerCapture?.(activeSectionDialogDrag.pointerId);
273:   activeSectionDialogDrag.dialog.classList.remove("is-dragging-dialog");
274:   activeSectionDialogDrag = null;
275: }
```

beginSectionDialogResize (script.js, lines 277-295)

beginSectionDialogResize part of the public website runtime. In this file, it appears around lines 277-295 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, contains, anchorSectionDialogForResize, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 278: if (event.button !== 0) return;; line 281: if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog")) return;.

Important local values include handle at line 279; dialog at line 280. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
277: function beginSectionDialogResize(event) {
278:   if (event.button !== 0) return;
279:   const handle = event.target.closest("[data-section-dialog-resize-handle]");
280:   const dialog = handle?.closest("dialog");
281:   if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog")) return;
282:   anchorSectionDialogForResize(dialog);
283:   activeSectionDialogResize = {
284:     dialog,
285:     direction: handle.dataset.sectionDialogResizeHandle,
286:     pointerId: event.pointerId,
287:     pointerTarget: event.target,
288:     rect: dialog.getBoundingClientRect(),
289:     startX: event.clientX,
290:     startY: event.clientY
291:   };
292:   event.target.setPointerCapture?.(event.pointerId);
293:   dialog.classList.add("is-resizing-dialog");
294:   event.preventDefault();
295: }
```

moveSectionDialogResize (script.js, lines 297-304)

moveSectionDialogResize part of the public website runtime. In this file, it appears around lines 297-304 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionDialogResizeBounds. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 298: if (!activeSectionDialogResize) return;.

Important local values include next at line 299. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
297: function moveSectionDialogResize(event) {
298:   if (!activeSectionDialogResize) return;
299:   const next = sectionDialogResizeBounds(activeSectionDialogResize, event);
300:   activeSectionDialogResize.dialog.style.left = `${next.left}px`;
301:   activeSectionDialogResize.dialog.style.top = `${next.top}px`;
302:   activeSectionDialogResize.dialog.style.width = `${next.width}px`;
303:   activeSectionDialogResize.dialog.style.height = `${next.height}px`;
304: }
```

endSectionDialogResize (script.js, lines 306-311)

endSectionDialogResize part of the public website runtime. In this file, it appears around lines 306-311 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 307: if (!activeSectionDialogResize) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
306: function endSectionDialogResize() {
307:   if (!activeSectionDialogResize) return;
308:   activeSectionDialogResize.pointerTarget?.releasePointerCapture?.(activeSectionDialogResize.pointerId);
309:   activeSectionDialogResize.dialog.classList.remove("is-resizing-dialog");
310:   activeSectionDialogResize = null;
311: }
```

updateSectionDialogMinimize (script.js, lines 313-320)

updateSectionDialogMinimize updates ui, state, cache, or stored data. In this file, it appears around lines 313-320 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, querySelector, and setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 316: if (!button) return;.

Important local values include minimized at line 314; button at line 315. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
313: function updateSectionDialogMinimize(dialog) {
314:   const minimized = dialog.classList.contains("is-minimized-dialog");
315:   const button = dialog.querySelector(".section-view-minimize");
316:   if (!button) return;
317:   button.textContent = minimized ? "+" : "-";
318:   button.title = minimized ? "Restore window" : "Minimize window";
319:   button.setAttribute("aria-label", minimized ? "Restore window" : "Minimize window");
320: }
```

toggleSectionDialogMinimized (script.js, lines 322-326)

toggleSectionDialogMinimized part of the public website runtime. In this file, it appears around lines 322-326 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references anchorSectionDialog, toggle, and updateSectionDialogMinimize. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
322: function toggleSectionDialogMinimized(dialog) {
323:   anchorSectionDialog(dialog);
324:   dialog.classList.toggle("is-minimized-dialog");
325:   updateSectionDialogMinimize(dialog);
326: }
```

enableSectionDialogDrag (script.js, lines 328-347)

enableSectionDialogDrag part of the public website runtime. In this file, it appears around lines 328-347 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references addEventListener, querySelectorAll, forEach, getBoundingClientRect, and clampSectionDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 329: if (sectionDialogDragEnabled) return;.

Important local values include rect at line 341; next at line 342. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
328: function enableSectionDialogDrag() {
329:   if (sectionDialogDragEnabled) return;
330:   sectionDialogDragEnabled = true;
331:   document.addEventListener("pointerdown", beginSectionDialogResize);
332:   document.addEventListener("pointerdown", beginSectionDialogDrag);
333:   document.addEventListener("pointermove", moveSectionDialogResize);
334:   document.addEventListener("pointermove", moveSectionDialogDrag);
335:   document.addEventListener("pointerup", endSectionDialogResize);
336:   document.addEventListener("pointerup", endSectionDialogDrag);
337:   document.addEventListener("pointercancel", endSectionDialogResize);
338:   document.addEventListener("pointercancel", endSectionDialogDrag);
339:   window.addEventListener("resize", () => {
340:     document.querySelectorAll(".section-view-dialog.is-draggable-dialog[open]").forEach((dialog) => {
341:       const rect = dialog.getBoundingClientRect();
342:       const next = clampSectionDialogPosition(rect.left, rect.top, dialog);
343:       dialog.style.left = `${next.left}px`;
344:       dialog.style.top = `${next.top}px`;
345:     });
346:   });
347: }
```

escapeHtml (script.js, lines 349-356)

escapeHtml part of the public website runtime. In this file, it appears around lines 349-356 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 350: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
349: function escapeHtml(value) {
350:   return String(value || "")
351:     .replaceAll("&", "&amp;")
352:     .replaceAll("<", "&lt;")
353:     .replaceAll(">", "&gt;")
354:     .replaceAll('"', "&quot;")
355:     .replaceAll("'", "&#039;");
356: }
```

normalizeCodeLanguage (script.js, lines 358-364)

normalizeCodeLanguage validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 358-364 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, replace, find, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 363: return match?.id || "javascript";.

Important local values include clean at line 359; match at line 360. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
358: function normalizeCodeLanguage(value = "") {
359:   const clean = String(value || "").trim().toLowerCase().replace(/[_-]+/g, " ");
360:   const match = supportedCodeLanguages.find((language) =>
361:     language.id === clean || language.aliases.includes(clean)
362:   );
363:   return match?.id || "javascript";
364: }
```

codeLanguageLabel (script.js, lines 366-368)

codeLanguageLabel part of the public website runtime. In this file, it appears around lines 366-368 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 367: return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label || "Code";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
366: function codeLanguageLabel(value = "") {
367:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label
|| "Code";
368: }
```

normalizeCodePasteMode (script.js, lines 370-372)

normalizeCodePasteMode validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 370-372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 371: return value === "source" ? "source" : "basic";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
370: function normalizeCodePasteMode(value = "") {
371:   return value === "source" ? "source" : "basic";
372: }
```

normalizeCodeText (script.js, lines 374-383)

normalizeCodeText validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 374-383 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodePasteMode, split, map, trimEnd, join, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 376: if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, " ").replace(/\s+\$/g, ""); line 377: return normalized.

Important local values include normalized at line 375. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
374: function normalizeCodeText(value = "", pasteMode = "source") {
375:   const normalized = String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ");
376:   if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, "
").replace(/\s+$/g, "");
377:   return normalized
378:     .split("\n")
379:     .map((line) => line.trimEnd())
380:     .join("\n")
381:     .replace(/\n{4,}/g, "\n\n\n")
382:     .trim();
383: }
```

sourceLooksCpp (script.js, lines 385-389)

sourceLooksCpp part of the public website runtime. In this file, it appears around lines 385-389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 387: return
/\\#include\\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text) ||.

Important local values include text at line 386. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
385: function sourceLooksCpp(source = "") {
386:   const text = String(source || "");
387:   return
/\\#include\\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
||
388:     /\b(std::|cout|cin|cerr|namespace|template\\s*<|class\\s+\\w+|new\\s+\\w+|delete\\s+|constexpr|nullptr|using
\\s+namespace)\\b/.test(text);
389: }
```

sourceLooksC (script.js, lines 391-395)

sourceLooksC part of the public website runtime. In this file, it appears around lines 391-395 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 393: return
/\\#include\\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\\.h>/.test(text) ||.

Important local values include text at line 392. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
391: function sourceLooksC(source = "") {
392:   const text = String(source || "");
393:   return /\\#include\\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\\.h>/.test(text) ||
394:     /\b(printf|scanf|malloc|calloc|realloc|free|struct\\s+\\w+|typedef\\s+struct)\\b/.test(text);
395: }
```

detectCodeLanguage (script.js, lines 397-410)

detectCodeLanguage part of the public website runtime. In this file, it appears around lines 397-410 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

It calls or references b, sourceLooksCpp, and sourceLooksC. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Important local values include code at line 398. These variables show what the function is assembling, checking, or handing to another layer.

```

397: function detectCodeLanguage(value = "") {
398:   const code = String(value || "");
399:   if (/<\/?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html";
400:   if (/\\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|
class\s+\\w+\\b/.test(code)) return "systemverilog";
401:   if (/\\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\\b/.test(code)) return
"verilog";
402:   if (/^\\s*\\.?(tran|ac|dc|op|model|subckt|ends|param)\\b/im.test(code) ||
\\b\\w+\\s+\\w+\\s+\\w+\\s+(?:DC|SIN|PULSE)?/i.test(code)) return "ltspice";
403:   if (/\\b(def|elif|import\\s+\\w+|from\\s+\\w+\\s+import|self|None|True|False)\\b/.test(code)) return "python";
404:   if (/\\b(public\\s+class|private\\s+|protected\\s+|static\\s+void\\s+main|System\\.out)\\b/.test(code)) return
"java";
405:   if (sourceLooksCpp(code) || sourceLooksC(code) ||
/\\b(\\#include|main\\s*\\{\\}|puts\\s*\\(|fprintf|sizeof\\s*\\(|)\\b/.test(code)) {
406:     return sourceLooksCpp(code) ? "cpp" : "c";
407:   }
408:   if (/\\b(const|let|var|function|=>|console\\.log|document\\.|window\\.)\\b/.test(code)) return "javascript";
409:   return "javascript";
410: }

```

tokenizedCodeHtml part of the public website runtime. In this file, it appears around lines 412-454 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

It calls or references `escapeHtml`, `replace`, `push`, `protect`, `includes`, `normalizeCodeLanguage`, `b`, and `CODE_TOKEN_`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

Important local values include `html` at line 413; `tokens` at line 414; `protect` at line 415; `token` at line 417; `commentPattern` at line 427; `keywordMap` at line 439; `keywords` at line 450. These variables show what the function is assembling, checking, or handing to another layer.

```

412: function tokenizedCodeHtml(code = "", language = "javascript") {
413:   let html = escapeHtml(code);
414:   const tokens = [];
415:   const protect = (pattern, className) => {
416:     html = html.replace(pattern, (match) => {
417:       const token = `@@CODE_TOKEN_${tokens.length}@@`;
418:       tokens.push(`
```

```

429:      : /\^*[\s\S]*?*\|\/\^*\/g;
430:      protect(commentPattern, "code-token-comment");
431:      protect("(?:\\.|[^\n\\])*'|(?:\\.|[^\n\\])*`|(?:\\.|[^\n\\])*`/g, "code-token-string");
432:      if (["c", "cpp"].includes(normalizeCodeLanguage(language))) {
433:          protect(/^#\s*`w+[\n]*/gm, "code-token-preprocessor");
434:      }
435:  }
436:
437:  protect(/\b(?:0x[\da-f]+|\d+(?:\.\d+)?(?:e[+-]?[d+])?)\b/gi, "code-token-number");
438:
439:  const keywordMap = {
440:      c: "_Alignas|_Alignof|_Atomic|_Bool|_Complex|_Generic|_Imaginary|_Noreturn|_Static_assert|_Thread_loc
al|auto|break|case|char|const|continue|default|do|double|else|enum|extern|float|for|goto|if|inline|int|long|regi
ster|restrict|return|short|signed|sizeof|static|struct|switch|typedef|union|unsigned|void|volatile|while",
441:      cpp: "alignas|alignof|and|and_eq|asm|auto|bitand|bitor|bool|break|case|catch|char|char8_t|char16_t|ch
ar32_t|class|compl|concept|const|consteval|constexpr|constinit|const_cast|continue|co_await|co_return|co_yield|d
ecltype|default|delete|do|double|dynamic_cast|else|enum|explicit|export|extern|false|float|for|friend|if|inline|
int|long|mutable|namespace|new|noexcept|not|not_eq|nullptr|operator|or|or_eq|private|protected|public|register|r
einterpret_cast|requires|return|short|signed|sizeof|static|static_assert|static_cast|struct|switch|template|this
|thread_local|throw|true|try|typedef|typeid|typename|union|unsigned|using|virtual|void|volatile|wchar_t|while|xor
r|xor_eq",
442:      verilog: "always|and|assign|begin|buf|case|casex|casez|deassign|default|defparam|disable|edge|else|en
d|endcase|endfunction|endmodule|endprimitive|endspecify|endtable|endtask|event|for|force|forever|fork|function|g
enerate|genvar|if|initial|inout|input|integer|join|module|nand|negedge|nor|not|or|output|parameter|posedge|primiti
ve|reg|release|repeat|signed|specify|supply0|supply1|table|task|tri|wand|while|wire|wor|xnor|xor",
443:      systemverilog: "always|always_comb|always_ff|always_latch|assign|automatic|begin|bit|case|class|clock
ing|covergroup|default|disable|do|else|end|endcase|endclass|endclocking|endfunction|endmodule|endpackage|endtask
|enum|for|forever|function|generate|genvar|if|initial|input|int|interface|logic|module|negedge|output|package|pa
rameter|posedge|reg|return|signed|task|typedef|wire",
444:      ltspice: "ac|dc|end|ends|four|func|global|ic|include|lib|meas|model|nodeset|op|options|param|plot|pro
be|save|step|subckt|temp|tran",
445:      java: "abstract|assert|boolean|break|byte|case|catch|char|class|const|continue|default|do|double|else
|enum|extends|final|finally|float|for|if|implements|import|instanceof|int|interface|long|native|new|null|package
|private|protected|public|return|short|static|strictfp|super|switch|synchronized|this|throw|throws|transient|try
|void|volatile|while",
446:      javascript: "async|await|break|case|catch|class|const|continue|debugger|default|delete|do|else|export
|extends|false|finally|for|from|function|if|import|in|instanceof|let|new|null|of|return|static|super|switch|this
|throw|true|try|typeof|undefined|var|void|while|yield",
447:      python: "and|as|assert|async|await|break|class|continue|def|del|elif|else|except|False|finally|for|fr
om|global|if|import|in|is|lambda|None|nonlocal|not|or|pass|raise|return|True|try|while|with|yield",
448:      html: "html|head|body|main|section|article|div|span|script|style|link|meta|title|header|footer|nav|bu
tton|input|form|label|img|a|p|pre|code"
449:  };
450:  const keywords = keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript;
451:  html = html.replace(new RegExp(`\\b(${keywords})\\b`, "g"), `

```

protect (script.js, lines 415-421)

protect part of the public website runtime. In this file, it appears around lines 415-421 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 419: return token;.

Important local values include protect at line 415; token at line 417. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

415:  const protect = (pattern, className) => {
416:      html = html.replace(pattern, (match) => {
417:          const token = `@@CODE_TOKEN_${tokens.length}@@`;
418:          tokens.push(`

```

renderRichCodeBlock (script.js, lines 456-465)

renderRichCodeBlock renders ui, markup, or display output. In this file, it appears around lines 456-465 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeCodeLanguage`, `detectCodeLanguage`, `normalizeCodeText`, `escapeHtml`, `codeLanguageLabel`, and `tokenizedCodeHtml`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 459: `return ``.

Important local values include `language` at line 457; `code` at line 458. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
456: function renderRichCodeBlock(block = {}) {
457:   const language = normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""));
458:   const code = normalizeCodeText(block.code || "", block.pasteMode || "source");
459:   return `
460:     <figure class="rich-code-block rich-code-language-${language}">
461:       <figcaption><span>&lt;/span> </span> ${escapeHtml(codeLanguageLabel(language))}</figcaption>
462:       <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
463:     </figure>
464:   `;
465: }
```

renderPlainMultiline (script.js, lines 467-469)

`renderPlainMultiline` renders ui, markup, or display output. In this file, it appears around lines 467-469 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `escapeHtml`, and `replace`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 468: `return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
467: function renderPlainMultiline(value = "") {
468:   return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");
469: }
```

normalizeSiteContent (script.js, lines 471-477)

`normalizeSiteContent` validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 471-477 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `trim`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 472: `return {`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
471: function normalizeSiteContent(content = {}) {
472:   return {
473:     heroCopy: String(content.heroCopy || "").trim() || defaultSiteContent.heroCopy,
474:     heroEyebrow: String(content.heroEyebrow || "").trim() || defaultSiteContent.heroEyebrow,
475:     heroTitle: String(content.heroTitle || "").trim() || defaultSiteContent.heroTitle
476:   };
477: }
```

normalizeProfile (script.js, lines 479-495)

`normalizeProfile` validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 479-495 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 480: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
479: function normalizeProfile(profileValue = {}) {
480:   return {
481:     brandImage: String(profileValue.brandImage || "").trim(),
482:     brandText: String(profileValue.brandText || "").trim() || String(profileValue.displayName ||
483:     "").trim() || defaultProfile.brandText,
484:     contactIntro: String(profileValue.contactIntro || "").trim(),
485:     displayName: String(profileValue.displayName || "").trim(),
486:     email: String(profileValue.email || "").trim(),
487:     githubUrl: String(profileValue.githubUrl || "").trim(),
488:     heroImage: String(profileValue.heroImage || "").trim(),
489:     linkedinUrl: String(profileValue.linkedinUrl || "").trim(),
490:     phone: String(profileValue.phone || "").trim(),
491:     portfolioLabel: String(profileValue.portfolioLabel || "").trim() || defaultProfile.portfolioLabel,
492:     profileImage: String(profileValue.profileImage || "").trim(),
493:     resumeUrl: String(profileValue.resumeUrl || "").trim(),
494:     websiteUrl: String(profileValue.websiteUrl || "").trim()
495:   };
```

mailComposeLink (script.js, lines 497-500)

mailComposeLink part of the public website runtime. In this file, it appears around lines 497-500 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 499: return clean ?

`https://mail.google.com/mail/?view=cm&fs=1&to=\${encodeURIComponent(clean)}` : "";

Important local values include clean at line 498. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
497: function mailComposeLink(email = "") {
498:   const clean = String(email || "").trim();
499:   return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(clean)}` : "";
500: }
```

phoneLink (script.js, lines 502-505)

phoneLink part of the public website runtime. In this file, it appears around lines 502-505 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 504: return clean ? `tel:\${clean.replace(/[^\d+]/g, "")}` : "";

Important local values include clean at line 503. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
502: function phoneLink(phone = "") {
503:   const clean = String(phone || "").trim();
504:   return clean ? `tel:${clean.replace(/[^\d+]/g, "")}` : "";
505: }
```

renderSiteContent (script.js, lines 507-515)

renderSiteContent renders ui, markup, or display output. In this file, it appears around lines 507-515 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeSiteContent, renderRichFieldContent, and applyPlainFieldStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include content at line 508. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
507: function renderSiteContent() {
508:   const content = normalizeSiteContent(siteContent || {});
509:   if (heroEyebrow) heroEyebrow.innerHTML = renderRichFieldContent(siteContentRich.heroEyebrow,
content.heroEyebrow);
510:   if (heroTitle) heroTitle.innerHTML = renderRichFieldContent(siteContentRich.heroTitle,
content.heroTitle);
511:   if (heroCopy) heroCopy.innerHTML = renderRichFieldContent(siteContentRich.heroCopy, content.heroCopy);
512:   applyPlainFieldStyle(heroEyebrow, "site-hero-eyebrow");
513:   applyPlainFieldStyle(heroTitle, "site-hero-title");
514:   applyPlainFieldStyle(heroCopy, "site-hero-copy");
515: }
```

renderProfile (script.js, lines 517-627)

renderProfile renders ui, markup, or display output. In this file, it appears around lines 517-627 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeProfile, mailComposeLink, phoneLink, normalizeLinkTarget, filter, renderRichFieldContent, applyPlainFieldStyle, slice, toUpperCase, setAttribute, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 622: return `

Important local values include current at line 518; displayName at line 519; label at line 520; emailLink at line 521; callLink at line 522; links at line 523; heroGithubLink at line 569; resumeObject at line 581; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
517: function renderProfile() {
518:   const current = normalizeProfile(profile || {});
519:   const displayName = current.displayName || "Portfolio";
520:   const label = current.portfolioLabel || "Portfolio";
521:   const emailLink = mailComposeLink(current.email);
522:   const callLink = phoneLink(current.phone);
523:   const links = [
524:     current.email ? { label: "Email", url: emailLink, external: true } : null,
525:     current.phone ? { label: "Call", url: callLink } : null,
526:     current.githubUrl ? { label: "GitHub", url: normalizeLinkTarget(current.githubUrl, { assumeWeb: true
}), external: true } : null,
527:     current.linkedinUrl ? { label: "LinkedIn", url: normalizeLinkTarget(current.linkedinUrl, { assumeWeb:
true }}, external: true } : null,
528:     current.websiteUrl ? { label: "Website", url: normalizeLinkTarget(current.websiteUrl, { assumeWeb:
true }}, external: true } : null,
529:     current.resumeUrl ? { label: "Resume", url: current.resumeUrl, external: false } : null
530:   ].filter(Boolean);
531:
532:   document.title = `${displayName} | ${label}`;
533:   if (brandName) brandName.innerHTML = renderRichFieldContent(profileRich.displayName, displayName);
534:   if (brandSubtitle) brandSubtitle.innerHTML = renderRichFieldContent(profileRich.portfolioLabel, label);
535:   applyPlainFieldStyle(brandName, "profile-display-name");
536:   applyPlainFieldStyle(brandSubtitle, "profile-portfolio-label");
537:   if (brandText) brandText.textContent = current.brandText || displayName.slice(0, 3).toUpperCase() ||
"PORT";
538:   if (brandIcon) {
539:     if (current.brandImage) {
540:       brandIcon.src = current.brandImage;
541:       brandIcon.hidden = false;
542:     } else {
543:       brandIcon.hidden = true;
544:     }
545:   }
546:
547:   if (headerAvatar && headerAvatarImage) {
548:     headerAvatar.hidden = !current.profileImage;
549:     if (current.profileImage) {
550:       headerAvatarImage.src = current.profileImage;
```

```

551:     headerAvatarImage.alt = displayName;
552:     headerAvatar.setAttribute("aria-label", `Go to ${displayName} contact information`);
553:   }
554: }
555:
556: if (heroImage) {
557:   if (current.heroImage) {
558:     heroImage.hidden = false;
559:     heroImage.src = current.heroImage;
560:     heroImage.alt = `${displayName} portfolio background`;
561:   } else {
562:     heroImage.hidden = true;
563:   }
564: }
565:
566: document.querySelectorAll('a[href="#resume"]').forEach((link) => {
567:   link.hidden = !current.resumeUrl;
568: });
569: const heroGithubLink = [...document.querySelectorAll(".hero-actions a")]
570:   .find((link) => /github/i.test(link.textContent || ""));
571: if (heroGithubLink) {
572:   heroGithubLink.hidden = !current.githubUrl;
573:   if (current.githubUrl) heroGithubLink.href = normalizeLinkTarget(current.githubUrl, { assumeWeb: true
});
574: }
575:
576: if (resumeSection) {
577:   resumeSection.hidden = !current.resumeUrl;
578:   resumeSection.querySelectorAll("a").forEach((link) => {
579:     link.href = current.resumeUrl || "#";
580:   });
581:   const resumeObject = resumeSection.querySelector("object");
582:   if (resumeObject) resumeObject.data = current.resumeUrl || "";
583:   const fallbackLink = resumeSection.querySelector("object a");
584:   if (fallbackLink) fallbackLink.href = current.resumeUrl || "#";
585: }
586:
587: if (profilePhoto) {
588:   profilePhoto.hidden = !current.profileImage;
589:   if (current.profileImage) {
590:     profilePhoto.src = current.profileImage;
591:     profilePhoto.alt = `Portrait of ${displayName}`;
592:   }
593: }
594: if (contactTitle) contactTitle.innerHTML = renderRichFieldContent(profileRich.displayName,
displayName);
595: applyPlainFieldStyle(contactTitle, "profile-display-name");
596: if (contactIntro) {
597:   contactIntro.innerHTML = renderRichFieldContent(
598:     profileRich.contactIntro,
599:     current.contactIntro || (current.displayName ? `${displayName}'s contact information and public
links` : "Add contact details in the builder.")
600:   );
601: }
602: applyPlainFieldStyle(contactIntro, "profile-contact-intro");
603: if (contactDetails) {
604:   contactDetails.innerHTML = [
605:     current.email ? `

```

textBlocksFromPlainText (script.js, lines 629-637)

textBlocksFromPlainText part of the public website runtime. In this file, it appears around lines 629-637 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 630: return [{.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
629: function textBlocksFromPlainText(text) {
630:   return [{
631:     align: "left",
632:     fontFamily: "Arial",
633:     fontSize: "normal",
634:     text: String(text || ""),
635:     type: "paragraph"
636:   }];
637: }
```

unwrapFormula (script.js, lines 639-652)

unwrapFormula part of the public website runtime. In this file, it appears around lines 639-652 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 649: if (match) return match[group].trim();; line 651: return text;.

Important local values include text at line 640; wrappers at line 641; match at line 648. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
639: function unwrapFormula(value) {
640:   const text = String(value || "").trim();
641:   const wrappers = [
642:     [/^$\$([^\$]+)\$$/, 1],
643:     [/^\\$([\\\$]+)\\$/, 1],
644:     [/^\\$([\\\$]+)\\$/, 1],
645:     [/^$\$([^\$]+)\$$/, 1]
646:   ];
647:   for (const [pattern, group] of wrappers) {
648:     const match = text.match(pattern);
649:     if (match) return match[group].trim();
650:   }
651:   return text;
652: }
```

renderInlineMath (script.js, lines 654-663)

renderInlineMath renders ui, markup, or display output. In this file, it appears around lines 654-663 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, replace, b, match, slice, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 657: return withMath.replace(/b(https?:VV[^\s<]+|www\.[^\s<]+|(?github|linkedin|gitlab|bitbucket|drive|docs)\.comV[^\s<]+)/gi, (match) => {; line 661: return `<a href="\${href}" target="\_blank" rel="noreferrer">\${clean}</a>\${trailing}`;.

Important local values include escaped at line 655; withMath at line 656; trailing at line 658; clean at line 659; href at line 660. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
654: function renderInlineMath(text) {
655:   const escaped = escapeHtml(text);
```


renderMultilineInlineText (script.js, lines 665-667)

looksLikeBareWebAddress (*script.js*, lines 669-676)

***looksLikeWebOrContactText* (script.js, lines 678-686)**

It has 6 return statement(s). The most important visible returns are: line 680: if (!clean) return false;; line 681: if (/\b(?:https?:\/\/www\.)[\^\$<>"]+/i.test(clean)) return true;; line 682: if (/\b[a-z0-9._%+~]@[a-z0-9.-]+\.[a-z]{2,}/i.test(clean)) return true;; line 683: if (/\b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[\^\$<>"]+/i.test(clean)) return true;. There are 2 additional return statements in the function body.

Important local values include `clean` at line 679. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
678: function looksLikeWebOrContactText(value = "") {
679:   const clean = String(value || "").trim();
680:   if (!clean) return false;
681:   if (/b(?:https?:\/\/|www\.)[\s<>'"]+/i.test(clean)) return true;
682:   if (/b[a-z0-9._%+~@]{1,64}[a-z0-9]{1,4}\b/i.test(clean)) return true;
683:   if (/b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[\s<>'"]+/i.test(clean)) return true;
684:   if (looksLikeBareWebAddress(clean)) return true;
685:   return clean.split(/\s+/).some((word) => looksLikeBareWebAddress(word.replace(/^[('"]+|['"]|'.';:;!]?+$/g,
  "")));
686: }
```

normalizeLinkTarget (script.js, lines 688-695)

`normalizeLinkTarget` validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 688-695 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `trim`, and `looksLikeBareWebAddress`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 690: `if (!clean) return ""`; line 691: `if (/^V/.test(clean)) return `https:${clean}``; line 692: `if (/^www\./i.test(clean)) return `https://${clean}``; line 693: `if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}``. There are 1 additional return statements in the function body.

Important local values include `clean` at line 689. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
688: function normalizeLinkTarget(target = "", options = {}) {
689:   const clean = String(target || "").trim();
690:   if (!clean) return "";
691:   if (/^\/\//.test(clean)) return `https:${clean}`;
692:   if (/^www\./i.test(clean)) return `https://${clean}`;
693:   if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}`;
694:   return clean;
695: }
```

isWebsiteLinkItem (script.js, lines 697-705)

`isWebsiteLinkItem` part of the public website runtime. In this file, it appears around lines 697-705 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `trim`, `filter`, `join`, `toLowerCase`, and `b`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 699: `if (/^(https?:)?V/i.test(clean) || /^www\./i.test(clean)) return true`; line 704: `return /b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText)`;

Important local values include `clean` at line 698; `typeText` at line 700. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
697: function isWebsiteLinkItem(item = {}, target = "") {
698:   const clean = String(target || "").trim();
699:   if (/^(https?:)?V/i.test(clean) || /^www\./i.test(clean)) return true;
700:   const typeText = [item.kind, item.type, item.status, item.meta, item.label, item.title]
701:     .filter(Boolean)
702:     .join(" ")
703:     .toLowerCase();
704:   return /b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);
705: }
```

linkifyRichTextNodes (script.js, lines 707-742)

linkifyRichTextNodes part of the public website runtime. In this file, it appears around lines 707-742 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createTreeWalker, acceptNode, closest, nextNode, push, forEach, createDocumentFragment, replace, b, append, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 710: if (!node.textContent || !(https?:\\|www\\.|(?:(?:github|linkedin|gitlab|bitbucket|drive|docs)\\.com\\)/i.test(node.textContent))) return NodeFilter.FILTER_REJECT;; line 711: return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;; line 737: return match;.

Important local values include walker at line 708; nodes at line 714; node at line 715; text at line 722; fragment at line 723; cursor at line 724; trailing at line 727; clean at line 728; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
707: function linkifyRichTextNodes(root) {
708:   const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
709:     acceptNode(node) {
710:       if (!node.textContent ||
!(https?:\\|www\\.|(?:(?:github|linkedin|gitlab|bitbucket|drive|docs)\\.com\\)/i.test(node.textContent))) return NodeFilter.FILTER_REJECT;
711:       return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;
712:     }
713:   });
714:   const nodes = [];
715:   let node = walker.nextNode();
716:   while (node) {
717:     nodes.push(node);
718:     node = walker.nextNode();
719:   }
720:
721:   nodes.forEach((textNode) => {
722:     const text = textNode.textContent || "";
723:     const fragment = document.createDocumentFragment();
724:     let cursor = 0;
725:     text.replace(/\\b(https?:\\|\\/[\\^\\s<]+|www\\.\\[\\^\\s<]+|(?:(?:github|linkedin|gitlab|bitbucket|drive|docs)\\.com
\\/[\\^\\s<]+))/gi, (match, _url, offset) => {
726:       if (offset > cursor) fragment.appendChild(document.createTextNode(text.slice(cursor, offset)));
727:       const trailing = match.match(/[.,:;!]+$/)?.[0] || "";
728:       const clean = trailing ? match.slice(0, -trailing.length) : match;
729:       const link = document.createElement("a");
730:       link.href = normalizeLinkTarget(clean, { assumeWeb: true });
731:       link.target = "_blank";
732:       link.rel = "noreferrer";
733:       link.textContent = clean;
734:       fragment.appendChild(link);
735:       if (trailing) fragment.appendChild(document.createTextNode(trailing));
736:       cursor = offset + match.length;
737:       return match;
738:     });
739:     if (cursor < text.length) fragment.appendChild(document.createTextNode(text.slice(cursor)));
740:     textNode.replaceWith(fragment);
741:   });
742: }
```

sanitizeRichInlineHtml (script.js, lines 744-787)

sanitizeRichInlineHtml validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 744-787 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, Set, querySelectorAll, forEach, has, replaceWith, getAttribute, trim, normalizeLinkTarget, cleanFontFamily, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 752: return;; line 786: return template.innerHTML;.

Important local values include template at line 745; allowedTags at line 747; href at line 755; normalizedHref at line 756; safeHref at line 757; fontFamily at line 758; fontPx at line 759; color at line 760; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

744: function sanitizeRichInlineHtml(value = "") {
745:   const template = document.createElement("template");
746:   template.innerHTML = String(value || "");
747:   const allowedTags = new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"]);
748:
749:   [...template.content.querySelectorAll("*")].forEach((node) => {
750:     if (!allowedTags.has(node.tagName)) {
751:       node.replaceWith(...node.childNodes);
752:       return;
753:     }
754:
755:     const href = node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : "";
756:     const normalizedHref = normalizeLinkTarget(href, { assumeWeb: true });
757:     const safeHref = /^((https?:|mailto:|tel:|#|\/)/i.test(normalizedHref) ? normalizedHref : "");
758:     const fontFamily = cleanFontFamily(node.style.fontFamily || "");
759:     const fontPx = normalizeFontPx(parseFloat(node.style.fontSize || ""));
760:     const color = normalizeTextColor(node.style.color || "");
761:     const rawFontWeight = node.style.fontWeight || "";
762:     const fontWeight = /^(bold|[6-9]00)$/i.test(rawFontWeight)
763:       ? "700"
764:       : /^(normal|[1-5]00)$/i.test(rawFontWeight)
765:       ? "400"
766:       : "";
767:     const fontStyle = node.style.fontStyle === "italic" ? "italic" : "";
768:     const textDecoration = node.style.textDecoration.includes("underline") ? "underline" : "";
769:
770:     [...node.attributes].forEach((attribute) => node.removeAttribute(attribute.name));
771:
772:     if (node.tagName === "A" && safeHref) {
773:       node.setAttribute("href", safeHref);
774:       node.setAttribute("target", "_blank");
775:       node.setAttribute("rel", "noreferrer");
776:     }
777:     if (fontFamily) node.style.fontFamily = fontFamily;
778:     if (fontPx) node.style.fontSize = `${fontPx}px`;
779:     if (color) node.style.color = color;
780:     if (fontWeight) node.style.fontWeight = fontWeight;
781:     if (fontStyle) node.style.fontStyle = fontStyle;
782:     if (textDecoration) node.style.textDecoration = textDecoration;
783:   });
784:
785:   linkifyRichTextNodes(template.content);
786:   return template.innerHTML;
787: }

```

displayTitle (script.js, lines 789-798)

displayTitle part of the public website runtime. In this file, it appears around lines 789-798 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 797: return replacements[title] || title;

Important local values include title at line 790; replacements at line 791. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

789: function displayTitle(value, fallback = "Untitled item") {
790:   const title = String(value || fallback).trim();
791:   const replacements = {
792:     "Brief": "Overview",
793:     "Project Brief": "Overview",
794:     "Design Brief": "Design Overview",
795:     "Simulation Brief": "Simulation Overview"
796:   };
797:   return replacements[title] || title;
798: }

```

cleanFontFamily (script.js, lines 800-807)

cleanFontFamily part of the public website runtime. In this file, it appears around lines 800-807 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, replace, trim, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 801: return String(value).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
800: function cleanFontFamily(value = "") {
801:   return String(value)
802:     .split(",")
803:     .map((item) => item.replace(/^[^w\s-]/g, "").trim())
804:     .filter(Boolean)
805:     .slice(0, 3)
806:     .join(", ");
807: }
```

normalizeFontPx (script.js, lines 809-812)

normalizeFontPx validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 809-812 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 811: return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";

Important local values include size at line 810. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
809: function normalizeFontPx(value) {
810:   const size = Number(value);
811:   return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";
812: }
```

normalizeTextColor (script.js, lines 813-820)

normalizeTextColor validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 813-820 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 815: return /^#([0-9a-f]{3}|[0-9a-f]{6})\$/i.test(color) ||

Important local values include color at line 814. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
813: function normalizeTextColor(value = "") {
814:   const color = String(value || "").trim();
815:   return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||
816:     /^rgba?\(/i.test(color) ||
817:     /^hsla?\(/i.test(color)
818:     ? color
819:     : "";
820: }
```

rgbColorToHex (script.js, lines 822-830)

rgbColorToHex part of the public website runtime. In this file, it appears around lines 822-830 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, match, map, max, min, toString, padStart, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 824: if (/^#[0-9a-f]{6}\$/i.test(color)) return color.toLowerCase();; line 826: if (shortHex) return `#\${shortHex[1]}\${shortHex[1]}\${shortHex[2]}\${shortHex[2]}\${shortHex[3]}\${shortHex[3]}`.toLowerCase();; line 828: if (!rgb) return color;; line 829: return `#\${[rgb[1], rgb[2], rgb[3]].map((part) => Math.max(0, Math.min(255, Number(part))).toString(16).padStart(2, "0")).join("")}`;.

Important local values include color at line 823; shortHex at line 825; rgb at line 827. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
822: function rgbColorToHex(value = "") {
823:   const color = String(value || "").trim();
824:   if (/^#[0-9a-f]{6}$/i.test(color)) return color.toLowerCase();
825:   const shortHex = color.match(/^#([0-9a-f])([0-9a-f])([0-9a-f])$/i);
826:   if (shortHex) return
`#${shortHex[1]}${shortHex[1]}${shortHex[2]}${shortHex[2]}${shortHex[3]}${shortHex[3]}`.toLowerCase();
827:   const rgb = color.match(/^rgba?\s*((\d+)\s*(\d+)\s*(\d+)\s*(\d+)/i);
828:   if (!rgb) return color;
829:   return `#${[rgb[1], rgb[2], rgb[3]].map((part) => Math.max(0, Math.min(255,
Number(part))).toString(16).padStart(2, "0")).join("")}`;
830: }
```

normalizePlainFieldStyle (script.js, lines 832-844)

normalizePlainFieldStyle validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 832-844 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, and rgbColorToHex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 843: return normalized;.

Important local values include normalized at line 833; fontFamily at line 834; fontPx at line 835; color at line 836. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
832: function normalizePlainFieldStyle(style = {}) {
833:   const normalized = {};
834:   const fontFamily = cleanFontFamily(style.fontFamily || "");
835:   const fontPx = normalizeFontPx(style.fontPx || style.fontSize || "");
836:   const color = normalizeTextColor(style.color || "");
837:   if (fontFamily) normalized.fontFamily = fontFamily;
838:   if (fontPx) normalized.fontPx = fontPx;
839:   if (color) normalized.color = rgbColorToHex(color);
840:   if (style.bold === true || style.bold === "true") normalized.bold = true;
841:   if (style.italic === true || style.italic === "true") normalized.italic = true;
842:   if (style.underline === true || style.underline === "true") normalized.underline = true;
843:   return normalized;
844: }
```

normalizeFieldStyles (script.js, lines 846-852)

normalizeFieldStyles validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 846-852 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fromEntries, entries, map, normalizePlainFieldStyle, filter, and keys. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 847: return Object.fromEntries(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
846: function normalizeFieldStyles(styles = {}) {
847:   return Object.fromEntries(
848:     Object.entries(styles || {}).
849:       .map(([fieldId, style]) => [fieldId, normalizePlainFieldStyle(style)])
850:       .filter(([_, style]) => Object.keys(style).length)
851:   );
852: }
```

plainFieldStyleToCss (script.js, lines 854-864)

plainFieldStyleToCss part of the public website runtime. In this file, it appears around lines 854-864 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 863: return rules.join("; ");.

Important local values include normalized at line 855; rules at line 856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
854: function plainFieldStyleToCss(style = {}) {
855:   const normalized = normalizePlainFieldStyle(style);
856:   const rules = [];
857:   if (normalized.fontFamily) rules.push(`font-family: ${normalized.fontFamily}`);
858:   if (normalized.fontPx) rules.push(`font-size: ${normalized.fontPx}px`);
859:   if (normalized.color) rules.push(`color: ${normalized.color}`);
860:   if (normalized.bold) rules.push("font-weight: 700");
861:   if (normalized.italic) rules.push("font-style: italic");
862:   if (normalized.underline) rules.push("text-decoration: underline");
863:   return rules.join("; ");
864: }
```

applyPlainFieldStyle (script.js, lines 866-875)

applyPlainFieldStyle part of the public website runtime. In this file, it appears around lines 866-875 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 867: if (!element) return;.

Important local values include style at line 868. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
866: function applyPlainFieldStyle(element, fieldId = "") {
867:   if (!element) return;
868:   const style = normalizePlainFieldStyle(fieldStyles[fieldId] || {});
869:   element.style.fontFamily = style.fontFamily || "";
870:   element.style.fontSize = style.fontPx ? `${style.fontPx}px` : "";
871:   element.style.color = style.color || "";
872:   element.style.fontWeight = style.bold ? "700" : "";
873:   element.style.fontStyle = style.italic ? "italic" : "";
874:   element.style.textDecoration = style.underline ? "underline" : "";
875: }
```

plainFieldStyleAttribute (script.js, lines 877-880)

plainFieldStyleAttribute part of the public website runtime. In this file, it appears around lines 877-880 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references plainFieldStyleToCss, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 879: return css ? `style="\${escapeHtml(css)}"` : "";

Important local values include css at line 878. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
877: function plainFieldStyleAttribute(fieldId = "") {
878:   const css = plainFieldStyleToCss(fieldStyles[fieldId] || {});
879:   return css ? `style="${escapeHtml(css)}"` : "";
880: }
```

richTextStyle (script.js, lines 882-896)

richTextStyle part of the public website runtime. In this file, it appears around lines 882-896 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 895: return styles.length ? `style="\${escapeHtml(styles.join(";"))}"` : "";

Important local values include styles at line 883; fontFamily at line 884; fontPx at line 885; color at line 886. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
882: function richTextStyle(block = {}) {
883:   const styles = [];
884:   const fontFamily = cleanFontFamily(block.fontFamily || "Arial") || "Arial";
885:   const fontPx = normalizeFontPx(block.fontPx);
886:   const color = normalizeTextColor(block.color || "");
887:
888:   if (fontFamily) styles.push(`font-family: ${fontFamily}`);
889:   if (fontPx) styles.push(`font-size: ${fontPx}px`);
890:   if (color) styles.push(`color: ${color}`);
891:   if (block.bold) styles.push("font-weight: 700");
892:   if (block.italic) styles.push("font-style: italic");
893:   if (block.underline) styles.push("text-decoration: underline");
894:
895:   return styles.length ? `style="${escapeHtml(styles.join(";"))}"` : "";
896: }
```

normalizeCropAspect (script.js, lines 898-900)

normalizeCropAspect validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 898-900 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 899: return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
898: function normalizeCropAspect(value = "") {
899:   return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";
900: }
```

normalizeCropZoom (script.js, lines 902-905)

normalizeCropZoom validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 902-905 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 904: return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;

Important local values include zoom at line 903. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:


```

902: function normalizeCropZoom(value = 1) {
903:   const zoom = Number(value);
904:   return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;
905: }

```

normalizeCropPosition (script.js, lines 907-910)

normalizeCropPosition validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 907-910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 909: return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;.

Important local values include position at line 908. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

907: function normalizeCropPosition(value = 50) {
908:   const position = Number(value);
909:   return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;
910: }

```

richImageCropStyle (script.js, lines 912-921)

richImageCropStyle part of the public website runtime. In this file, it appears around lines 912-921 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 920: return ` style="\${escapeHtml(styles.join("; ")))}`;.

Important local values include aspect at line 913; styles at line 914. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

912: function richImageCropStyle(block = {}) {
913:   const aspect = normalizeCropAspect(block.cropAspect);
914:   const styles = [
915:     `--crop-zoom: ${normalizeCropZoom(block.cropZoom)}`,
916:     `--crop-x: ${normalizeCropPosition(block.cropX)}%`,
917:     `--crop-y: ${normalizeCropPosition(block.cropY)}%`
918:   ];
919:   if (aspect !== "original") styles.push(`--crop-aspect: ${aspect}`);
920:   return ` style="${escapeHtml(styles.join("; ")))}`;
921: }

```

richImageDownloadLink (script.js, lines 923-940)

richImageDownloadLink part of the public website runtime. In this file, it appears around lines 923-940 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, cleanRichImageTitle, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 928: return `

Important local values include label at line 924; url at line 925; target at line 926. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

923: function richImageDownloadLink(block = {}) {
924:   const label = escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file");
925:   const url = block.url || "#";
926:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) });
927:
928:   return `
929:     <p class="rich-download-only">
930:       <a
931:         class="resource-link"
932:         href="${escapeHtml(target)}"
933:         ${linkAttributes(target, block)}
934:         ${downloadAttribute(target, block)}
935:       >
936:         ${label}
937:       </a>
938:     </p>
939:   `;
940: }

```

cleanRichImageTitle (script.js, lines 942-945)

cleanRichImageTitle part of the public website runtime. In this file, it appears around lines 942-945 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 944: return /^pasted image\b/i.test(title) ? "" : title;.

Important local values include title at line 943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

942: function cleanRichImageTitle(block = {}) {
943:   const title = String(block.title || "").trim();
944:   return /^pasted image\b/i.test(title) ? "" : title;
945: }

```

renderRichContent (script.js, lines 947-982)

renderRichContent renders ui, markup, or display output. In this file, it appears around lines 947-982 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, richImageDownloadLink, cleanRichImageTitle, normalizeLinkTarget, normalizeCropAspect, richImageCropStyle, escapeHtml, unwrapFormula, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 949: return `; line 954: if (block.display === "download") return richImageDownloadLink(block);; line 957: return `; line 969: return `<p class="rich-paragraph rich-text-normal text-\${align}">\${renderInlineMath(formula)}</p>`;. There are 3 additional return statements in the function body.

Important local values include blocks at line 948; align at line 952; title at line 955; imageSrc at line 956; formula at line 967; size at line 974; content at line 975. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

947: function renderRichContent(rich, fallbackText = "") {
948:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
949:   return `
950:     <div class="rich-content">
951:       ${blocks.map((block) => {
952:         const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
953:         if (block.type === "image") {
954:           if (block.display === "download") return richImageDownloadLink(block);
955:           const title = cleanRichImageTitle(block);
956:           const imageSrc = normalizeLinkTarget(block.url, { assumeWeb: true });
957:           return `
958:             <figure class="rich-image justify-${align}">
959:               <span class="rich-image-viewport crop-${normalizeCropAspect(block.cropAspect) ===
"original" ? "original" : "active"}"${richImageCropStyle(block)}>
960:                 
961:               </span>
962:               ${title || block.caption} ? `<figcaption>${title} ?
`<strong>${escapeHtml(title)}</strong>` : ""}${block.caption ? `<span>${escapeHtml(block.caption)}</span>` :
""}</figcaption>` : ""}

```

```

963:         </figure>
964:     `;
965: }
966: if (block.type === "formula") {
967:     const formula = unwrapFormula(block.formula);
968:     if (looksLikeWebOrContactText(formula)) {
969:         return `
```

renderRichFieldContent (script.js, lines 984-994)

renderRichFieldContent renders ui, markup, or display output. In this file, it appears around lines 984-994 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, sanitizeRichInlineHtml, renderInlineMath, richTextStyle, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 986: return blocks.map((block) => {; line 987: if (block.type !== "paragraph") return "";; line 992: return `

Important local values include blocks at line 985; align at line 988; content at line 989. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

984: function renderRichFieldContent(rich, fallbackText = "") {
985:     const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
986:     return blocks.map((block) => {
987:         if (block.type !== "paragraph") return "";
988:         const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
989:         const content = block.html
990:             ? sanitizeRichInlineHtml(block.html)
991:             : renderInlineMath(block.text || "");
992:         return `

```

slugLabel (script.js, lines 996-999)

slugLabel part of the public website runtime. In this file, it appears around lines 996-999 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 998: return category ? category.label : value;.

Important local values include category at line 997. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

996: function slugLabel(value) {
997:     const category = categories.find((item) => item.id === value);
998:     return category ? category.label : value;
999: }

```

canonicalTemplateId (script.js, lines 1001-1003)

canonicalTemplateId part of the public website runtime. In this file, it appears around lines 1001-1003 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1002: return legacyTemplateSkins[id] || id || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1001: function canonicalTemplateId(id) {  
1002:   return legacyTemplateSkins[id] || id || "";  
1003: }
```

projectTemplateId (script.js, lines 1005-1007)

projectTemplateId part of the public website runtime. In this file, it appears around lines 1005-1007 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references canonicalTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1006: return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1005: function projectTemplateId(project) {  
1006:   return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");  
1007: }
```

projectTemplateClass (script.js, lines 1009-1012)

projectTemplateClass part of the public website runtime. In this file, it appears around lines 1009-1012 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1011: return templateId ? `project-template project-template-\${templateId}` : "project-template-white";.

Important local values include templateId at line 1010. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1009: function projectTemplateClass(project) {  
1010:   const templateId = projectTemplateId(project);  
1011:   return templateId ? `project-template project-template-${templateId}` : "project-template-white";  
1012: }
```

responsiveClassName (script.js, lines 1014-1019)

responsiveClassName part of the public website runtime. In this file, it appears around lines 1014-1019 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1015: return String(value || fallback || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1014: function responsiveClassName(value, fallback) {
1015:   return String(value || fallback || "")
1016:   .toLowerCase()
1017:   .replace(/^[^a-z0-9]+/g, "-")
1018:   .replace(/^-|-$/g, "") || fallback;
1019: }
```

projectResponsiveClass (script.js, lines 1021-1024)

projectResponsiveClass part of the public website runtime. In this file, it appears around lines 1021-1024 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile, and responsiveClassName. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1023: return `responsive-project responsive-density-\${responsiveClassName(profile.density, "balanced")} responsive-card-\${responsiveClassName(profile.cardLayout, "single")}`;.

Important local values include profile at line 1022. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1021: function projectResponsiveClass(project) {
1022:   const profile = projectResponsiveProfile(project);
1023:   return `responsive-project responsive-density-${responsiveClassName(profile.density, "balanced")}
responsive-card-${responsiveClassName(profile.cardLayout, "single")}`;
1024: }
```

responsiveStyleValues (script.js, lines 1026-1034)

responsiveStyleValues part of the public website runtime. In this file, it appears around lines 1026-1034 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1028: return {.

Important local values include profile at line 1027. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1026: function responsiveStyleValues(project) {
1027:   const profile = projectResponsiveProfile(project);
1028:   return {
1029:     "--responsive-section-card-min": profile.sectionCardMin,
1030:     "--responsive-content-max": profile.contentMax,
1031:     "--responsive-media-max": profile.mediaMax,
1032:     "--responsive-touch-target": profile.touchTarget
1033:   };
1034: }
```

hasPublicTemplate (script.js, lines 1036-1038)

hasPublicTemplate part of the public website runtime. In this file, it appears around lines 1036-1038 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1037: return Boolean(projectTemplateId(project));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1036: function hasPublicTemplate(project) {  
1037:   return Boolean(projectTemplateId(project));  
1038: }
```

projectTemplateVisual (script.js, lines 1040-1042)

projectTemplateVisual part of the public website runtime. In this file, it appears around lines 1040-1042 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1041: return project?.portfolioView?.template?.visual || project?.templateVisual || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1040: function projectTemplateVisual(project) {  
1041:   return project?.portfolioView?.template?.visual || project?.templateVisual || null;  
1042: }
```

projectTemplateStyle (script.js, lines 1044-1062)

projectTemplateStyle part of the public website runtime. In this file, it appears around lines 1044-1062 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateVisual, responsiveStyleValues, entries, filter, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1058: return Object.entries(values).

Important local values include visual at line 1045; values at line 1046. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1044: function projectTemplateStyle(project, accent) {  
1045:   const visual = projectTemplateVisual(project) || {};  
1046:   const values = {  
1047:     "--accent": accent,  
1048:     "--template-bg": visual.background,  
1049:     "--template-panel": visual.panel,  
1050:     "--template-accent": visual.accent,  
1051:     "--template-hover": visual.hover,  
1052:     "--template-click": visual.click,  
1053:     "--template-click-text": visual.clickText,  
1054:     "--template-line": visual.line,  
1055:     "--template-text": visual.text,  
1056:     ...responsiveStyleValues(project)  
1057:   };  
1058:   return Object.entries(values)  
1059:     .filter(([, value]) => value)  
1060:     .map(([key, value]) => `${key}: ${value}`)  
1061:     .join("; ");  
1062: }
```

applyProjectTemplateToElement (script.js, lines 1064-1098)

applyProjectTemplateToElement part of the public website runtime. In this file, it appears around lines 1064-1098 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, startsWith, forEach, remove, projectTemplateClass, split, add, projectResponsiveClass, projectTemplateVisual, responsiveStyleValues, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1065: if (!element) return;.

Important local values include visual at line 1078; values at line 1079. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1064: function applyProjectTemplateToElement(element, project, accent) {
1065:   if (!element) return;
1066:   [...element.classList]
1067:     .filter((className) =>
1068:       className === "project-template" ||
1069:       className === "project-template-white" ||
1070:       className === "responsive-project" ||
1071:       className.startsWith("project-template-") ||
1072:       className.startsWith("responsive-density-") ||
1073:       className.startsWith("responsive-card-")
1074:     )
1075:     .forEach((className) => element.classList.remove(className));
1076:   projectTemplateClass(project).split(" ").forEach((className) => element.classList.add(className));
1077:   projectResponsiveClass(project).split(" ").forEach((className) => element.classList.add(className));
1078:   const visual = projectTemplateVisual(project) || {};
1079:   const values = {
1080:     "--accent": accent,
1081:     "--template-bg": visual.background,
1082:     "--template-panel": visual.panel,
1083:     "--template-accent": visual.accent,
1084:     "--template-hover": visual.hover,
1085:     "--template-click": visual.click,
1086:     "--template-click-text": visual.clickText,
1087:     "--template-line": visual.line,
1088:     "--template-text": visual.text,
1089:     ...responsiveStyleValues(project)
1090:   };
1091:   Object.entries(values).forEach(([key, value]) => {
1092:     if (value) {
1093:       element.style.setProperty(key, value);
1094:     } else {
1095:       element.style.removeProperty(key);
1096:     }
1097:   });
1098: }
```

parsedItemTerms (script.js, lines 1100-1110)

parsedItemTerms part of the public website runtime. In this file, it appears around lines 1100-1110 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1101: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1100: function parsedItemTerms(item) {
1101:   return [
1102:     item?.title,
1103:     item?.description,
1104:     item?.meta,
1105:     item?.url,
1106:     item?.kind,
1107:     ...(item?.rich?.blocks || []).flatMap((block) => [block.text, block.formula, block.title,
1108:     block.caption]),
1109:     ...(item?.children || []).flatMap(parsedItemTerms)
1110:   ];
1111: }
```

flattenProject (script.js, lines 1112-1165)

flattenProject part of the public website runtime. In this file, it appears around lines 1112-1165 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugLabel, electronicsSearchKeywords, flatMap, map, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1138: return [.

Important local values include categoryLabel at line 1113; electronicsKeywords at line 1114; richSummaryTerms at line 1117; parsedChildTerms at line 1123; design at line 1126; designTerms at line 1127. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1112: function flattenProject(project) {
1113:   const categoryLabel = slugLabel(project.category);
1114:   const electronicsKeywords = window.electronicsSearchKeywords
1115:     ? window.electronicsSearchKeywords(project, categoryLabel)
1116:     : [];
1117:   const richSummaryTerms = (project.summaryRich?.blocks || []).flatMap((block) => [
1118:     block.text,
1119:     block.formula,
1120:     block.title,
1121:     block.caption
1122:   ]);
1123:   const parsedChildTerms = (project.portfolioView?.sections || []).flatMap((section) =>
1124:     (section.items || []).flatMap(parsedItemTerms)
1125:   );
1126:   const design = project.design || {};
1127:   const designTerms = [
1128:     design.brief?.summary,
1129:     ...(design.brief?.files || []).flatMap((item) => [item.title, item.description, item.url]),
1130:     design.documentation?.summary,
1131:     ...(design.documentation?.files || []).flatMap((item) => [item.title, item.description, item.url]),
1132:     ...(design.documentation?.references || []).flatMap((item) => [item.title, item.description,
item.url]),
1133:     ...(design.documentation?.mathAnalysis || []).flatMap((item) => [item.title, item.description]),
1134:     design.simulation?.summary,
1135:     ...(design.simulation?.files || []).flatMap((item) => [item.title, item.description, item.url]),
1136:     ...(design.simulation?.results || []).flatMap((item) => [item.title, item.description, item.url])
1137:   ];
1138:   return [
1139:     project.id,
1140:     project.title,
1141:     categoryLabel,
1142:     project.status,
1143:     project.summary,
1144:     ...richSummaryTerms,
1145:     ...parsedChildTerms,
1146:     ...designTerms,
1147:     ...(project.focus || []),
1148:     ...(project.highlights || []),
1149:     ...(project.tools || []).map((item) => typeof item === "string" ? item : [item.name, item.title,
item.label, item.description].filter(Boolean).join(" ")),
1150:     ...(project.languages || []),
1151:     ...(project.documents || []).flatMap((item) => [item.title, item.type, item.status, item.url]),
1152:     ...(project.tests || []).flatMap((item) => [item.name, item.method, item.status, item.result,
item.artifact]),
1153:     ...(project.pcbS || []).flatMap((item) => [item.name, item.revision, item.status, item.artifact]),
1154:     ...(project.media || []).flatMap((item) => [item.title, item.caption, item.url]),
1155:     ...(project.sections || []).flatMap((section) => [
1156:       section.title,
1157:       section.description,
1158:       ...(section.items || []).flatMap((item) => [item.title, item.description, item.type, item.status,
item.url])
1159:     ]),
1160:     ...(project.links || []).flatMap((item) => [item.label, item.url]),
1161:     ...electronicsKeywords
1162:   ]
1163:   .map(normalize)
1164:   .join(" ");
1165: }
```

richSummaryTerms (script.js, lines 1117-1126)

richSummaryTerms part of the public website runtime. In this file, it appears around lines 1117-1126 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include `richSummaryTerms` at line 1117; `parsedChildTerms` at line 1123; `design` at line 1126. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1117: const richSummaryTerms = (project.summaryRich?.blocks || []).flatMap((block) => [  
1118:   block.text,  
1119:   block.formula,  
1120:   block.title,  
1121:   block.caption  
1122:   ]);  
1123: const parsedChildTerms = (project.portfolioView?.sections || []).flatMap((section) =>  
1124:   (section.items || []).flatMap(parsedItemTerms)  
1125: );  
1126: const design = project.design || {};
```

parsedChildTerms (script.js, lines 1123-1126)

`parsedChildTerms` part of the public website runtime. In this file, it appears around lines 1123-1126 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `flatMap`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include `parsedChildTerms` at line 1123; `design` at line 1126. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1123: const parsedChildTerms = (project.portfolioView?.sections || []).flatMap((section) =>  
1124:   (section.items || []).flatMap(parsedItemTerms)  
1125: );  
1126: const design = project.design || {};
```

projectMatches (script.js, lines 1167-1169)

`projectMatches` part of the public website runtime. In this file, it appears around lines 1167-1169 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `flattenProject`, and `includes`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1168: `return !query || flattenProject(project).includes(query);`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1167: function projectMatches(project, query) {  
1168:   return !query || flattenProject(project).includes(query);  
1169: }
```

projectVisible (script.js, lines 1171-1174)

`projectVisible` part of the public website runtime. In this file, it appears around lines 1171-1174 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `projectMatches`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1173: `return categoryMatch && projectMatches(project, query);`.

Important local values include `categoryMatch` at line 1172. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1171: function projectVisible(project, query) {
1172:   const categoryMatch = activeFilter === "all" || project.category === activeFilter;
1173:   return categoryMatch && projectMatches(project, query);
1174: }

```

normalizeCategory (script.js, lines 1176-1184)

normalizeCategory validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1176-1184 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, normalizeTextColor, toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1178: return {.

Important local values include label at line 1177. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1176: function normalizeCategory(category = {}) {
1177:   const label = String(category.label || category.title || category.id || "Project category").trim() ||
"Project category";
1178:   return {
1179:     accent: normalizeTextColor(category.accent || "") || "#1677a8",
1180:     description: String(category.description || "").trim(),
1181:     id: String(category.id || label.toLowerCase().replace(/^[a-z0-9]+/g, "-").replace(/^-|-$/g, "")) ||
"category"),
1182:     label
1183:   };
1184: }

```

hydrateProjectCategories (script.js, lines 1186-1201)

hydrateProjectCategories part of the public website runtime. In this file, it appears around lines 1186-1201 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, Set, forEach, trim, has, split, filter, charAt, toUpperCase, slice, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1191: if (!id || knownIds.has(id)) return;; line 1200: return normalized;.

Important local values include normalized at line 1187; knownIds at line 1188; id at line 1190; label at line 1192. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1186: function hydrateProjectCategories(rawCategories = [], rawProjects = []) {
1187:   const normalized = rawCategories.map(normalizeCategory);
1188:   const knownIds = new Set(normalized.map((category) => category.id));
1189:   rawProjects.forEach((project) => {
1190:     const id = String(project?.category || "").trim();
1191:     if (!id || knownIds.has(id)) return;
1192:     const label = id
1193:       .split(/[_\s]+/)
1194:       .filter(Boolean)
1195:       .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
1196:       .join(" ") || "Project category";
1197:     normalized.push(normalizeCategory({ id, label, description: "Project category" }));
1198:     knownIds.add(id);
1199:   });
1200:   return normalized;
1201: }

```

setActiveFilter (script.js, lines 1203-1209)

setActiveFilter part of the public website runtime. In this file, it appears around lines 1203-1209 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some, forEach, and toggle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1203: function setActiveFilter(filter = "all") {
1204:   activeFilter = filter;
1205:   if (activeFilter !== "all" && !categories.some((category) => category.id === activeFilter)) {
1206:     activeFilter = "all";
1207:   }
1208:   filterButtons.forEach((item) => item.classList.toggle("active", item.dataset.filter === activeFilter));
1209: }
```

bindFilterButtons (script.js, lines 1211-1220)

bindFilterButtons part of the public website runtime. In this file, it appears around lines 1211-1220 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, addEventListener, setActiveFilter, renderProjects, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1211: function bindFilterButtons() {
1212:   filterButtons = [...document.querySelectorAll(".filter-button")];
1213:   filterButtons.forEach((button) => {
1214:     button.addEventListener("click", () => {
1215:       setActiveFilter(button.dataset.filter || "all");
1216:       renderProjects();
1217:       updateSearchDropdown();
1218:     });
1219:   });
1220: }
```

renderCategoryFilters (script.js, lines 1222-1241)

renderCategoryFilters renders ui, markup, or display output. In this file, it appears around lines 1222-1241 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, join, escapeHtml, bindFilterButtons, and setActiveFilter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1222: function renderCategoryFilters() {
1223:   if (projectTrackCount) {
1224:     projectTrackCount.textContent = `${categories.length} Track${categories.length === 1 ? "" : "s"}`;
1225:   }
1226:   if (projectTrackLabels) {
1227:     projectTrackLabels.textContent = categories.length
1228:     ? categories.map((category) => category.label).join(", ")
1229:     : "project categories";
1230:   }
1231:   if (projectFilters) {
1232:     projectFilters.innerHTML = `
1233:       <button class="filter-button active" type="button" data-filter="all">All</button>
1234:       ${categories.map((category) => `
1235:         <button class="filter-button" type="button" data-
1236:         filter="${escapeHtml(category.id)}">${escapeHtml(category.label)}</button>
1237:       `).join("")}
1238:     `;
1239:     bindFilterButtons();
1240:   }
1241: }
```

```
1240:   setActiveFilter(activeFilter);
1241: }
```

flattenSearchText (script.js, lines 1243-1251)

flattenSearchText part of the public website runtime. In this file, it appears around lines 1243-1251 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flat, filter, map, stringify, join, replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1244: return values.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1243: function flattenSearchText(values = []) {
1244:   return values
1245:     .flat(Infinity)
1246:     .filter((value) => value !== null && value !== undefined)
1247:     .map((value) => typeof value === "object" ? JSON.stringify(value) : String(value))
1248:     .join(" ")
1249:     .replace(/\s+/g, " ")
1250:     .trim();
1251: }
```

richTextTerms (script.js, lines 1253-1264)

richTextTerms part of the public website runtime. In this file, it appears around lines 1253-1264 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1254: return (rich?.blocks || []).flatMap((block) => [

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1253: function richTextTerms(rich) {
1254:   return (rich?.blocks || []).flatMap((block) => [
1255:     block.text,
1256:     block.html,
1257:     block.formula,
1258:     block.code,
1259:     block.language,
1260:     block.title,
1261:     block.caption,
1262:     block.url
1263:   ]);
1264: }
```

addSearchEntry (script.js, lines 1266-1276)

addSearchEntry part of the public website runtime. In this file, it appears around lines 1266-1276 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flattenSearchText, normalize, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1268: if (!entry.title && !text) return null;; line 1275: return storedEntry;.

Important local values include text at line 1267; storedEntry at line 1269. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1266: function addSearchEntry(entries, entry) {
1267:   const text = flattenSearchText([entry.title, entry.type, entry.context, entry.text, entry.url]);
1268:   if (!entry.title && !text) return null;
1269:   const storedEntry = {
1270:     ...entry,
1271:     normalizedText: normalize(text),
1272:     text
1273:   };
1274:   entries.push(storedEntry);
1275:   return storedEntry;
1276: }

```

searchSnippet (script.js, lines 1278-1286)

searchSnippet part of the public website runtime. In this file, it appears around lines 1278-1286 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, trim, normalize, indexOf, slice, max, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1280: if (!clean) return "";; line 1282: if (index < 0) return clean.slice(0, 150);; line 1285: return `\${start ? "...": ""}\${clean.slice(start, end)}\${end < clean.length ? "...": ""}`;

Important local values include clean at line 1279; index at line 1281; start at line 1283; end at line 1284. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1278: function searchSnippet(text = "", query = "") {
1279:   const clean = String(text || "").replace(/\s+/g, " ").trim();
1280:   if (!clean) return "";
1281:   const index = normalize(clean).indexOf(normalize(query));
1282:   if (index < 0) return clean.slice(0, 150);
1283:   const start = Math.max(0, index - 54);
1284:   const end = Math.min(clean.length, index + query.length + 80);
1285:   return `${start ? "...": ""}${clean.slice(start, end)}${end < clean.length ? "...": ""}`;
1286: }

```

entryScore (script.js, lines 1288-1314)

entryScore part of the public website runtime. In this file, it appears around lines 1288-1314 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, trim, split, filter, startsWith, includes, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1290: if (!normalizedQuery) return 0;; line 1313: return score;;

Important local values include normalizedQuery at line 1289; title at line 1291; type at line 1292; context at line 1293; text at line 1294; words at line 1295; score at line 1296. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1288: function entryScore(entry, query) {
1289:   const normalizedQuery = normalize(query).trim();
1290:   if (!normalizedQuery) return 0;
1291:   const title = normalize(entry.title);
1292:   const type = normalize(entry.type);
1293:   const context = normalize(entry.context);
1294:   const text = entry.normalizedText || "";
1295:   const words = normalizedQuery.split(/\s+/).filter(Boolean);
1296:   let score = 0;
1297:
1298:   if (title === normalizedQuery) score += 160;
1299:   if (title.startsWith(normalizedQuery)) score += 120;
1300:   if (title.includes(normalizedQuery)) score += 90;
1301:   if (context.includes(normalizedQuery)) score += 55;
1302:   if (type.includes(normalizedQuery)) score += 35;
1303:   if (text.includes(normalizedQuery)) score += 45;
1304:   words.forEach((word) => {
1305:     if (title.includes(word)) score += 16;
1306:     if (context.includes(word)) score += 10;
1307:     if (text.includes(word)) score += 6;
1308:   });
1309:
1310:   if (score > 0 && entry.kind === "file") score += 18;
1311:   if (score > 0 && entry.kind === "section") score += 14;

```

```

1312:   if (score > 0 && entry.kind === "project") score += 12;
1313:   return score;
1314: }

```

entryMatches (script.js, lines 1316-1318)

entryMatches part of the public website runtime. In this file, it appears around lines 1316-1318 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references entryScore. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1317: return entryScore(entry, query) > 0;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1316: function entryMatches(entry, query) {
1317:   return entryScore(entry, query) > 0;
1318: }

```

fileIsBrowserSearchable (script.js, lines 1320-1323)

fileIsBrowserSearchable part of the public website runtime. In this file, it appears around lines 1320-1323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1322: return /\.txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb\$/i.test(clean);.

Important local values include clean at line 1321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1320: function fileIsBrowserSearchable(url = "") {
1321:   const clean = String(url || "").split(/[?#]/)[0].toLowerCase();
1322:   return /\.txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb$/i.test(clean);
1323: }

```

indexSearchableFileText (script.js, lines 1325-1338)

indexSearchableFileText part of the public website runtime. In this file, it appears around lines 1325-1338 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileIsBrowserSearchable, startsWith, fetch, text, flattenSearchText, slice, normalize, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1326: if (!entry.url || !fileIsBrowserSearchable(entry.url)) return;; line 1327: if (!/^https?:?V\\/.i.test(entry.url) && !entry.url.startsWith(window.location.origin)) return;; line 1330: if (!response.ok) return;.

Important local values include response at line 1329; text at line 1331. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1325: async function indexSearchableFileText(entry) {
1326:   if (!entry.url || !fileIsBrowserSearchable(entry.url)) return;
1327:   if (!/^https?:?V\\/.i.test(entry.url) && !entry.url.startsWith(window.location.origin)) return;
1328:   try {
1329:     const response = await fetch(entry.url);
1330:     if (!response.ok) return;
1331:     const text = await response.text();
1332:     entry.text = flattenSearchText([entry.text, text.slice(0, 60000)]);

```

```

1333:     entry.normalizedText = normalize(entry.text);
1334:     updateSearchDropdown();
1335:   } catch {
1336:     // Files remain searchable by title, path, caption, and catalog text if direct text fetch is
unavailable.
1337:   }
1338: }

```

addProjectSearchEntries (script.js, lines 1340-1414)

addProjectSearchEntries part of the public website runtime. In this file, it appears around lines 1340-1414 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugLabel, richTextTerms, electronicsSearchKeywords, addSearchEntry, filter, sectionHasRenderableContent, forEach, displayTitle, parsedItemTerms, fileNameFromUrl, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include categoryLabel at line 1341; baseContext at line 1342; projectTerms at line 1343; sections at line 1364; walkItems at line 1376; nextPath at line 1378; title at line 1379; url at line 1380; plus 5 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1340: function addProjectSearchEntries(entries, project) {
1341:   const categoryLabel = slugLabel(project.category);
1342:   const baseContext = `${categoryLabel} / ${project.title}`;
1343:   const projectTerms = [
1344:     project.id,
1345:     project.summary,
1346:     richTextTerms(project.summaryRich),
1347:     project.focus,
1348:     project.highlights,
1349:     project.tools,
1350:     project.languages,
1351:     project.links,
1352:     window.electronicsSearchKeywords ? window.electronicsSearchKeywords(project, categoryLabel) : []
1353:   ];
1354:
1355:   addSearchEntry(entries, {
1356:     context: categoryLabel,
1357:     kind: "project",
1358:     projectId: project.id,
1359:     title: project.title,
1360:     type: "Project",
1361:     text: projectTerms
1362:   });
1363:
1364:   const sections = (project.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
1365:   sections.forEach((section, sectionIndex) => {
1366:     addSearchEntry(entries, {
1367:       context: baseContext,
1368:       kind: "section",
1369:       projectId: project.id,
1370:       sectionIndex: String(sectionIndex),
1371:       title: displayTitle(section.title, "Section"),
1372:       type: "Project section",
1373:       text: [section.description, richTextTerms(section.rich), parsedItemTerms(section)]
1374:     });
1375:
1376:     const walkItems = (items = [], path = [], parentTitles = []) => {
1377:       items.forEach((item, index) => {
1378:         const nextPath = [...path, index];
1379:         const title = displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url ||
""), "Project item");
1380:         const url = itemUrl(item);
1381:         const context = [baseContext, displayTitle(section.title, "Section"),
...parentTitles].filter(Boolean).join(" / ");
1382:         const entry = {
1383:           context,
1384:           kind: url ? "file" : "subsection",
1385:           projectId: project.id,
1386:           resourcePath: pathToString(nextPath),
1387:           sectionIndex: String(sectionIndex),
1388:           title,
1389:           type: url ? "File or uploaded asset" : "Subsection",
1390:           url,
1391:           text: [item.description, item.meta, item.kind, item.type, url, richTextTerms(item.rich),
parsedItemTerms(item)]
1392:         };
1393:         const storedEntry = addSearchEntry(entries, entry);
1394:         if (url && storedEntry) indexSearchableFileText(storedEntry);

```

```

1395:         walkItems(nodeChildren(item), nextPath, [...parentTitles, title]);
1396:     });
1397: };
1398: walkItems(section.items || []);
1399: });
1400:
1401: uniqueDownloads(collectDownloadsFromValue(project, "Project file", [])).forEach((download) => {
1402:     const entry = {
1403:         context: baseContext,
1404:         kind: "file",
1405:         projectId: project.id,
1406:         title: download.title || fileNameFromUrl(download.url),
1407:         type: download.type || "Uploaded file",
1408:         url: download.url,
1409:         text: [download.description, download.status, download.url]
1410:     };
1411:     const storedEntry = addSearchEntry(entries, entry);
1412:     if (storedEntry) indexSearchableFileText(storedEntry);
1413: });
1414: }

```

sections (script.js, lines 1364-1399)

sections part of the public website runtime. In this file, it appears around lines 1364-1399 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, sectionHasRenderableContent, forEach, addSearchEntry, displayTitle, richTextTerms, parsedItemTerms, fileNameFromUrl, itemUrl, join, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include sections at line 1364; walkItems at line 1376; nextPath at line 1378; title at line 1379; url at line 1380; context at line 1381; entry at line 1382; storedEntry at line 1393. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1364:     const sections = (project.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
1365:     sections.forEach((section, sectionIndex) => {
1366:         addSearchEntry(entries, {
1367:             context: baseContext,
1368:             kind: "section",
1369:             projectId: project.id,
1370:             sectionIndex: String(sectionIndex),
1371:             title: displayTitle(section.title, "Section"),
1372:             type: "Project section",
1373:             text: [section.description, richTextTerms(section.rich), parsedItemTerms(section)]
1374:         });
1375:
1376:         const walkItems = (items = [], path = [], parentTitles = []) => {
1377:             items.forEach((item, index) => {
1378:                 const nextPath = [...path, index];
1379:                 const title = displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url ||
""), "Project item");
1380:                 const url = itemUrl(item);
1381:                 const context = [baseContext, displayTitle(section.title, "Section"),
...parentTitles].filter(Boolean).join(" / ");
1382:                 const entry = {
1383:                     context,
1384:                     kind: url ? "file" : "subsection",
1385:                     projectId: project.id,
1386:                     resourcePath: pathToString(nextPath),
1387:                     sectionIndex: String(sectionIndex),
1388:                     title,
1389:                     type: url ? "File or uploaded asset" : "Subsection",
1390:                     url,
1391:                     text: [item.description, item.meta, item.kind, item.type, url, richTextTerms(item.rich),
parsedItemTerms(item)]
1392:                 };
1393:                 const storedEntry = addSearchEntry(entries, entry);
1394:                 if (url && storedEntry) indexSearchableFileText(storedEntry);
1395:                 walkItems(nodeChildren(item), nextPath, [...parentTitles, title]);
1396:             });
1397:         };
1398:         walkItems(section.items || []);
1399:     });

```

addSiteSectionSearchEntries (script.js, lines 1416-1463)

addSiteSectionSearchEntries part of the public website runtime. In this file, it appears around lines 1416-1463 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeProfile, splice, forEach, addSearchEntry, indexSearchableFileText, filter, and richTextTerms. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include current at line 1417; pageSections at line 1418; resumeTextEntry at line 1436. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1416: function addSiteSectionSearchEntries(entries) {
1417:   const current = normalizeProfile(profile || {});
1418:   const pageSections = [
1419:     { id: "top", title: heroTitle?.textContent || "Front page", type: "Page section", text:
[heroEyebrow?.textContent, heroCopy?.textContent] },
1420:     { id: "projects", title: "Engineering Projects", type: "Directory", text: "project directory
categories files sections" },
1421:     { id: "process", title: "Process", type: "Page section", text: "engineering process design testing
documentation" },
1422:     { id: "contact", title: "Contact", type: "Page section", text: [current.displayName, current.email,
current.phone, current.githubUrl, current.linkedinUrl, current.websiteUrl, current.contactIntro] }
1423:   ];
1424:   if (current.resumeUrl) {
1425:     pageSections.splice(2, 0, { id: "resume", title: "Resume", type: "Page section", text: "resume
professional profile document" });
1426:   }
1427:   pageSections.forEach((section) => addSearchEntry(entries, {
1428:     domTarget: section.id,
1429:     kind: "page",
1430:     title: section.title,
1431:     type: section.type,
1432:     text: section.text
1433:   }));
1434:
1435:   if (current.resumeUrl) {
1436:     const resumeTextEntry = addSearchEntry(entries, {
1437:       context: "Professional Profile / Resume",
1438:       kind: "file",
1439:       title: "Resume",
1440:       type: "Resume",
1441:       url: current.resumeUrl,
1442:       text: [current.displayName, current.portfolioLabel, current.contactIntro, current.resumeUrl]
1443:     });
1444:     if (resumeTextEntry) indexSearchableFileText(resumeTextEntry);
1445:   }
1446:
1447:   (siteSections || []).filter(siteSectionRenderable).forEach((section) => {
1448:     addSearchEntry(entries, {
1449:       domTarget: section.id,
1450:       kind: "page",
1451:       title: section.title || "Portfolio section",
1452:       type: "Portfolio section",
1453:       text: [section.description, richTextTerms(section.richDescription), section.links, section.assets]
1454:     });
1455:     (section.subsections || []).forEach((item) => addSearchEntry(entries, {
1456:       domTarget: section.id,
1457:       kind: "page",
1458:       title: item.title || "Portfolio subsection",
1459:       type: "Portfolio subsection",
1460:       text: [item.description, richTextTerms(item.richDescription), item.links]
1461:     }));
1462:   });
1463: }
```

rebuildSearchIndex (script.js, lines 1465-1470)

This function builds the searchable index used by the public website. It collects project titles, sections, rich text, files, profile content, public sections, and other searchable terms into structured entries.

A real search box needs an index rather than scanning random DOM text every time. This function makes search fast and able to jump to specific content.

It calls or references addSiteSectionSearchEntries, forEach, and addProjectSearchEntries. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include entries at line 1466. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1465: function rebuildSearchIndex() {
1466:   const entries = [];
1467:   addSiteSectionSearchEntries(entries);
1468:   projects.forEach((project) => addProjectSearchEntries(entries, project));
1469:   searchableEntries = entries;
1470: }
```

searchResultsFor (script.js, lines 1472-1486)

This function ranks search entries for the visitor's query. It compares normalized terms, phrase matches, entry scores, and searchable text to return the closest results.

It makes search behave more like a small search engine instead of a simple browser find box.

It calls or references normalize, trim, Set, map, entryScore, filter, sort, localeCompare, join, has, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 1474: if (!normalizedQuery) return []; line 1476: return searchableEntries; line 1482: if (seen.has(key)) return false;; line 1484: return true;.

Important local values include normalizedQuery at line 1473; seen at line 1475; key at line 1481. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1472: function searchResultsFor(query) {
1473:   const normalizedQuery = normalize(query).trim();
1474:   if (!normalizedQuery) return [];
1475:   const seen = new Set();
1476:   return searchableEntries
1477:     .map((entry) => ({ ...entry, score: entryScore(entry, normalizedQuery) }))
1478:     .filter((entry) => entry.score > 0)
1479:     .sort((a, b) => b.score - a.score || a.title.localeCompare(b.title))
1480:     .filter((entry) => {
1481:       const key = [entry.kind, entry.projectId, entry.sectionIndex, entry.resourcePath, entry.domTarget,
entry.url, entry.title].join("|");
1482:       if (seen.has(key)) return false;
1483:       seen.add(key);
1484:       return true;
1485:     });
1486: }
```

assistantEndpoint (script.js, lines 1488-1494)

assistantEndpoint part of the public website runtime. In this file, it appears around lines 1488-1494 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, querySelector, and includes. Endpoint references found inside it are /api/portfolio-ai. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 1489: if (typeof window.OMB_AI_ENDPOINT === "string") return window.OMB_AI_ENDPOINT.trim(); line 1491: if (metaEndpoint.trim()) return String(metaEndpoint).trim(); line 1492: if (!["localhost", "127.0.0.1"].includes(window.location.hostname)) return "/api/portfolio-ai"; line 1493: return "/api/portfolio-ai";.

Important local values include metaEndpoint at line 1490. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1488: function assistantEndpoint() {
1489:   if (typeof window.OMB_AI_ENDPOINT === "string") return window.OMB_AI_ENDPOINT.trim();
1490:   const metaEndpoint = document.querySelector('meta[name="portfolio-ai-endpoint"]')?.content || "";
1491:   if (metaEndpoint.trim()) return String(metaEndpoint).trim();
1492:   if (!["localhost", "127.0.0.1"].includes(window.location.hostname)) return "/api/portfolio-ai";
1493:   return "/api/portfolio-ai";
1494: }
```

assistantSourceLabel (script.js, lines 1496-1505)

assistantSourceLabel part of the public website runtime. In this file, it appears around lines 1496-1505 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, trim, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1498: if (result.kind === "project" || result.kind === "page") return title;; line 1504: return [title, specificContext].filter(Boolean).join(" - ");.

Important local values include title at line 1497; contextParts at line 1499; specificContext at line 1503. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1496: function assistantSourceLabel(result = {}) {
1497:   const title = result.title || "Portfolio item";
1498:   if (result.kind === "project" || result.kind === "page") return title;
1499:   const contextParts = String(result.context || "")
1500:     .split("/")
1501:     .map((item) => item.trim())
1502:     .filter(Boolean);
1503:   const specificContext = contextParts.slice(-2).join(" / ");
1504:   return [title, specificContext].filter(Boolean).join(" - ");
1505: }
```

assistantProjectSummary (script.js, lines 1507-1522)

assistantProjectSummary part of the public website runtime. In this file, it appears around lines 1507-1522 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugLabel, filter, sectionHasRenderableContent, map, and displayTitle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1512: return {.

Important local values include categoryLabel at line 1508; sections at line 1509. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1507: function assistantProjectSummary(project = {}) {
1508:   const categoryLabel = slugLabel(project.category);
1509:   const sections = (project.portfolioView?.sections || []);
1510:   .filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))
1511:   .map((section) => displayTitle(section.title, "Section"));
1512:   return {
1513:     category: categoryLabel,
1514:     focus: project.focus || [],
1515:     id: project.id,
1516:     links: project.links || [],
1517:     sections,
1518:     summary: project.summary || "",
1519:     title: project.portfolioView?.title || project.title,
1520:     tools: project.tools || []
1521:   };
1522: }
```

assistantProjectTitleAcronyms (script.js, lines 1524-1535)

assistantProjectTitleAcronyms part of the public website runtime. In this file, it appears around lines 1524-1535 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, split, filter, Set, min, add, slice, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1534: return [...acronyms].filter((item) => item.length >= 2);.

Important local values include words at line 1525; acronyms at line 1528. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1524: function assistantProjectTitleAcronyms(title = "") {
1525:   const words = normalize(title)
```

```

1526:     .split(/\s+/)
1527:     .filter((word) => word.length > 2);
1528: const acronyms = new Set();
1529: for (let start = 0; start < words.length; start += 1) {
1530:   for (let length = 2; length <= Math.min(5, words.length - start); length += 1) {
1531:     acronyms.add(words.slice(start, start + length).map((word) => word[0]).join(""));
1532:   }
1533: }
1534: return [...acronyms].filter((item) => item.length >= 2);
1535: }

```

assistantProjectIsNamedInQuestion (script.js, lines 1537-1553)

assistantProjectIsNamedInQuestion part of the public website runtime. In this file, it appears around lines 1537-1553 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, Set, includes, split, assistantProjectTitleAcronyms, some, has, filter, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 1539: if (!clean) return false;; line 1543: if (!title) return false;; line 1544: if (clean.includes(title)) return true;; line 1548: if (acronymMatch) return true;. There are 2 additional return statements in the function body.

Important local values include clean at line 1538; fillerWords at line 1540; rawTitle at line 1541; title at line 1542; cleanWords at line 1545; acronymMatch at line 1546; words at line 1549; matches at line 1551. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1537: function assistantProjectIsNamedInQuestion(question = "", project = {}) {
1538:   const clean = normalize(question);
1539:   if (!clean) return false;
1540:   const fillerWords = new Set(["with", "from", "into", "using", "project", "design", "system",
"systems"]);
1541:   const rawTitle = project.portfolioView?.title || project.title || project.id;
1542:   const title = normalize(rawTitle);
1543:   if (!title) return false;
1544:   if (clean.includes(title)) return true;
1545:   const cleanWords = new Set(clean.split(/\s+/));
1546:   const acronymMatch = assistantProjectTitleAcronyms(rawTitle)
1547:     .some((acronym) => cleanWords.has(acronym));
1548:   if (acronymMatch) return true;
1549:   const words = title.split(/\s+/).filter((word) => word.length > 3 && !fillerWords.has(word));
1550:   if (!words.length) return false;
1551:   const matches = words.filter((word) => clean.includes(word)).length;
1552:   return words.length <= 2 ? matches === words.length : matches >= Math.min(3, words.length);
1553: }

```

assistantNamedProjectIds (script.js, lines 1555-1559)

assistantNamedProjectIds part of the public website runtime. In this file, it appears around lines 1555-1559 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, assistantProjectIsNamedInQuestion, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1556: return projects.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1555: function assistantNamedProjectIds(question = "") {
1556:   return projects
1557:     .filter((project) => assistantProjectIsNamedInQuestion(question, project))
1558:     .map((project) => project.id);
1559: }

```

assistantProjectTitleMatches (script.js, lines 1561-1563)

assistantProjectTitleMatches part of the public website runtime. In this file, it appears around lines 1561-1563 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantNamedProjectIds. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1562: return assistantNamedProjectIds(question).length > 0;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1561: function assistantProjectTitleMatches(question = "") {
1562:   return assistantNamedProjectIds(question).length > 0;
1563: }
```

assistantUrlKind (script.js, lines 1565-1577)

assistantUrlKind part of the public website runtime. In this file, it appears around lines 1565-1577 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, toLowerCase, and fileIsBrowserSearchable. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 1567: if (!clean) return "unknown";; line 1568: if (/github\.com|raw\.githubusercontent\.com|gist\.githubusercontent\.com/i.test(clean)) return "github";; line 1569: if (/linkedin\.com/i.test(clean)) return "linkedin";; line 1570: if (/\.png|jpe?g|gif|webp|svg\$/i.test(clean)) return "image";. There are 6 additional return statements in the function body.

Important local values include clean at line 1566. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1565: function assistantUrlKind(url = "") {
1566:   const clean = String(url || "").split(/[?#]/)[0].toLowerCase();
1567:   if (!clean) return "unknown";
1568:   if (/github\.com|raw\.githubusercontent\.com|gist\.githubusercontent\.com/i.test(clean)) return
"github";
1569:   if (/linkedin\.com/i.test(clean)) return "linkedin";
1570:   if (/\.png|jpe?g|gif|webp|svg$/i.test(clean)) return "image";
1571:   if (/\.pdf$/i.test(clean)) return "pdf";
1572:   if (/\.zip|7z|rar$/i.test(clean)) return "archive";
1573:   if (/\.xlsx?|csv$/i.test(clean)) return "spreadsheet";
1574:   if (fileIsBrowserSearchable(clean)) return "text";
1575:   if (/^https?:\/\//i.test(clean)) return "webpage";
1576:   return "file";
1577: }
```

assistantAbsoluteUrl (script.js, lines 1579-1658)

assistantAbsoluteUrl part of the public website runtime. In this file, it appears around lines 1579-1658 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, looksLikeBareWebAddress, URL, assistantLinkRecordsFromValue, uniqueDownloads, collectDownloadsFromValue, forEach, push, assistantUrlKind, fileNameFromUrl, and 14 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 1581: if (!target) return "";; line 1583: return new URL(target, window.location.href).href;; line 1585: return target;; line 1594: if (!rawUrl) return;;. There are 5 additional return statements in the function body.

Important local values include target at line 1580; records at line 1590; downloads at line 1591; rawUrl at line 1593; text at line 1607; matches at line 1608; records at line 1622; add at line 1623; plus 6 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1579: function assistantAbsoluteUrl(url = "") {
1580:   const target = normalizeLinkTarget(url, { assumeWeb: looksLikeBareWebAddress(url) ||
/^https?:\/\//i.test(url) });
1581:   if (!target) return "";
```

```

1582:   try {
1583:     return new URL(target, window.location.href).href;
1584:   } catch {
1585:     return target;
1586:   }
1587: }
1588:
1589: function assistantLinkRecordsFromValue(value, context = "") {
1590:   const records = [];
1591:   const downloads = uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", []));
1592:   downloads.forEach((download) => {
1593:     const rawUrl = download.url || download.href || download.path || "";
1594:     if (!rawUrl) return;
1595:     records.push({
1596:       context,
1597:       description: download.description || "",
1598:       kind: download.kind || assistantUrlKind(rawUrl),
1599:       label: download.title || download.label || fileNameFromUrl(rawUrl),
1600:       url: assistantAbsoluteUrl(rawUrl)
1601:     });
1602:   });
1603:   return records;
1604: }
1605:
1606: function assistantUrlsFromTextValue(value, context = "") {
1607:   const text = flattenSearchText([value]);
1608:   const matches = text.match(/(b(?:https?:\\\/|www\\.)[^s<>"])]+/gi) || [];
1609:   return [...new Set(matches.map((url) => url.replace(/[,.;!]?+$/g, "")))]
1610:     .slice(0, 16)
1611:     .map((url) => ({
1612:       context,
1613:       description: "",
1614:       kind: assistantUrlKind(url),
1615:       label: /github\\.com/i.test(url) ? "GitHub repository or profile" : fileNameFromUrl(url),
1616:       type: "Public link",
1617:       url: assistantAbsoluteUrl(url)
1618:     }));
1619: }
1620:
1621: function assistantPublicProfileLinks() {
1622:   const records = [];
1623:   const add = (item = {}, context = "") => {
1624:     const rawUrl = item.url || item.href || item.path || "";
1625:     if (!rawUrl) return;
1626:     const label = item.label || item.title || fileNameFromUrl(rawUrl);
1627:     records.push({
1628:       context,
1629:       kind: item.kind || assistantUrlKind(rawUrl),
1630:       label,
1631:       text: item.description || "",
1632:       url: assistantAbsoluteUrl(rawUrl)
1633:     });
1634:   };
1635:
1636:   const current = normalizeProfile(profile || {});
1637:   if (current.resumeUrl) add({ label: "Resume", kind: "resume", url: current.resumeUrl }, "Professional Profile");
1638:   if (current.githubUrl) add({ label: "GitHub", kind: "github", url: current.githubUrl }, "Professional Profile");
1639:   if (current.linkedinUrl) add({ label: "LinkedIn", kind: "linkedin", url: current.linkedinUrl }, "Professional Profile");
1640:   if (current.websiteUrl) add({ label: "Website", kind: "webpage", url: current.websiteUrl }, "Professional Profile");
1641:
1642:   (siteSections || []).forEach((section) => {
1643:     (section.links || []).forEach((link) => add(link, section.title || "Portfolio section"));
1644:   });
1645:
1646:   return records.filter((record, index, array) => (
1647:     record.url && array.findIndex((item) => item.url === record.url && item.label === record.label) ===
index
1648:   ));
1649: }
1650:
1651: function assistantProjectEvidence(project = {}) {
1652:   const downloads = uniqueDownloads(collectDownloadsFromValue(project, "Project asset", []));
1653:   const downloadRecords = downloads.slice(0, 26).map((download) => {
1654:     const rawUrl = download.url || "";
1655:     return {
1656:       description: download.description || "",
1657:       kind: assistantUrlKind(rawUrl),
1658:       label: download.title || download.label || fileNameFromUrl(rawUrl),

```

assistantLinkRecordsFromValue (script.js, lines 1589-1604)

assistantLinkRecordsFromValue part of the public website runtime. In this file, it appears around lines 1589-1604 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, collectDownloadsFromValue, forEach, push, assistantUrlKind, fileNameFromUrl, and assistantAbsoluteUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1594: if (!rawUrl) return;; line 1603: return records;.

Important local values include records at line 1590; downloads at line 1591; rawUrl at line 1593. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1589: function assistantLinkRecordsFromValue(value, context = "") {
1590:   const records = [];
1591:   const downloads = uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", []));
1592:   downloads.forEach((download) => {
1593:     const rawUrl = download.url || download.href || download.path || "";
1594:     if (!rawUrl) return;
1595:     records.push({
1596:       context,
1597:       description: download.description || "",
1598:       kind: download.kind || assistantUrlKind(rawUrl),
1599:       label: download.title || download.label || fileNameFromUrl(rawUrl),
1600:       url: assistantAbsoluteUrl(rawUrl)
1601:     });
1602:   });
1603:   return records;
1604: }
```

assistantUrlsFromTextValue (script.js, lines 1606-1619)

assistantUrlsFromTextValue part of the public website runtime. In this file, it appears around lines 1606-1619 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flattenSearchText, match, b, Set, map, replace, slice, assistantUrlKind, fileNameFromUrl, and assistantAbsoluteUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1609: return [...new Set(matches.map((url) => url.replace(/[,;:!?]+\$/g, "")))].

Important local values include text at line 1607; matches at line 1608. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1606: function assistantUrlsFromTextValue(value, context = "") {
1607:   const text = flattenSearchText([value]);
1608:   const matches = text.match(/(?:https?:\/\/|www\.)[^<"]+/gi) || [];
1609:   return [...new Set(matches.map((url) => url.replace(/[,;:!?]+$/g, "")))]
1610:     .slice(0, 16)
1611:     .map((url) => ({
1612:       context,
1613:       description: "",
1614:       kind: assistantUrlKind(url),
1615:       label: /github\.com/i.test(url) ? "GitHub repository or profile" : fileNameFromUrl(url),
1616:       type: "Public link",
1617:       url: assistantAbsoluteUrl(url)
1618:     }));
1619: }
```

assistantPublicProfileLinks (script.js, lines 1621-1649)

assistantPublicProfileLinks part of the public website runtime. In this file, it appears around lines 1621-1649 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileNameFromUrl, push, assistantUrlKind, assistantAbsoluteUrl, normalizeProfile, add, forEach, filter, and findIndex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1625: if (!rawUrl) return;; line 1646: return records.filter((record, index, array) => (.

Important local values include records at line 1622; add at line 1623; rawUrl at line 1624; label at line 1626; current at line 1636. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1621: function assistantPublicProfileLinks() {
1622:   const records = [];
1623:   const add = (item = {}, context = "") => {
1624:     const rawUrl = item.url || item.href || item.path || "";
1625:     if (!rawUrl) return;
1626:     const label = item.label || item.title || fileNameFromUrl(rawUrl);
1627:     records.push({
1628:       context,
1629:       kind: item.kind || assistantUrlKind(rawUrl),
1630:       label,
1631:       text: item.description || "",
1632:       url: assistantAbsoluteUrl(rawUrl)
1633:     });
1634:   };
1635:
1636:   const current = normalizeProfile(profile || {});
1637:   if (current.resumeUrl) add({ label: "Resume", kind: "resume", url: current.resumeUrl }, "Professional
Profile");
1638:   if (current.githubUrl) add({ label: "GitHub", kind: "github", url: current.githubUrl }, "Professional
Profile");
1639:   if (current.linkedinUrl) add({ label: "LinkedIn", kind: "linkedin", url: current.linkedinUrl },
"Professional Profile");
1640:   if (current.websiteUrl) add({ label: "Website", kind: "webpage", url: current.websiteUrl },
"Professional Profile");
1641:
1642:   (siteSections || []).forEach((section) => {
1643:     (section.links || []).forEach((link) => add(link, section.title || "Portfolio section"));
1644:   });
1645:
1646:   return records.filter((record, index, array) => (
1647:     record.url && array.findIndex((item) => item.url === record.url && item.label === record.label) ===
index
1648:   ));
1649: }
```

assistantProjectEvidence (script.js, lines 1651-1667)

assistantProjectEvidence part of the public website runtime. In this file, it appears around lines 1651-1667 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, collectDownloadsFromValue, slice, map, assistantUrlKind, fileNameFromUrl, assistantAbsoluteUrl, assistantUrlsFromTextValue, filter, and findIndex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1655: return {; line 1664: return [...downloadRecords, ...textLinkRecords].filter((record, index, array) => (.

Important local values include downloads at line 1652; downloadRecords at line 1653; rawUrl at line 1654; textLinkRecords at line 1663. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1651: function assistantProjectEvidence(project = {}) {
1652:   const downloads = uniqueDownloads(collectDownloadsFromValue(project, "Project asset", []));
1653:   const downloadRecords = downloads.slice(0, 26).map((download) => {
1654:     const rawUrl = download.url || "";
1655:     return {
1656:       description: download.description || "",
1657:       kind: assistantUrlKind(rawUrl),
1658:       label: download.title || download.label || fileNameFromUrl(rawUrl),
1659:       type: download.type || "",
1660:       url: assistantAbsoluteUrl(rawUrl)
1661:     };
1662:   });
1663:   const textLinkRecords = assistantUrlsFromTextValue([project.summary, project.links,
project.portfolioView, project.portfolioView?.title || project.title]);
1664:   return [...downloadRecords, ...textLinkRecords].filter((record, index, array) => (
1665:     record.url && array.findIndex((item) => item.url === record.url) === index
1666:   ));
1667: }
```

assistantQuestionIsCasual (script.js, lines 1669-1675)

assistantQuestionIsCasual part of the public website runtime. In this file, it appears around lines 1669-1675 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1671: return /^(hi|hello|hey|good\s+(morning|afternoon|evening))|yo|sup|thanks|thank\s+you|ok|okay)\b/.test(clean).

Important local values include clean at line 1670. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1669: function assistantQuestionIsCasual(question = "") {
1670:   const clean = normalize(question);
1671:   return
1672:   /^(hi|hello|hey|good\s+(morning|afternoon|evening))|yo|sup|thanks|thank\s+you|ok|okay)\b/.test(clean)
1673:   || /^(how\s+are\s+you|what\s+can\s+you\s+help.*|what\s+do\s+you\s+do|who\s+are\s+you|help|help\s+me|c
1674:   an\s+you\s+help|nice\s+to\s+meet\s+you)\?\?\$/ .test(clean)
1675:   || /^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name\?\?\$/ .test(clean)
1676:   || /^who\s+am\s+i\?\?\$/ .test(clean);
1677: }
```

assistantQuestionHasPortfolioIntent (script.js, lines 1677-1682)

assistantQuestionHasPortfolioIntent part of the public website runtime. In this file, it appears around lines 1677-1682 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, assistantProjectTitleMatches, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1679: return assistantProjectTitleMatches(question).

Important local values include clean at line 1678. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1677: function assistantQuestionHasPortfolioIntent(question = "") {
1678:   const clean = normalize(question);
1679:   return assistantProjectTitleMatches(question)
1680:   || /\b(maurice|otieno|portfolio|resume|github|linkedin|contact|email|phone|project|projects|repo|repo
1681:   sitory|repositories|file|files|document|documents|artifact|artifacts|link|links|download|open|show|where|built|b
1682:   uild|created|designed|implemented)\b/.test(clean)
1683:   || /\b(your|his|maurice's)\s+(project|projects|resume|github|linkedin|portfolio|work|email|phone|cont
1684:   act|repo|repository|files?|documents?|links?)\b/.test(clean);
1685: }
```

assistantConversationSuggestsPortfolioContext (script.js, lines 1684-1688)

assistantConversationSuggestsPortfolioContext part of the public website runtime. In this file, it appears around lines 1684-1688 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice, some, b, and normalize. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1685: return history.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1684: function assistantConversationSuggestsPortfolioContext(history = []) {
1685:   return history
1686:   .slice(-4)
1687:   .some((item) => /\b(maurice|otieno|portfolio|project|resume|github|linkedin|electronics|engineering)\
1688:   b/.test(normalize(item?.content)));
1689: }
```

assistantQuestionAllowsPublicLookup (script.js, lines 1690-1695)

assistantQuestionAllowsPublicLookup part of the public website runtime. In this file, it appears around lines 1690-1695 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantQuestionIntent, normalize, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1692: return intent === "general_engineering".

Important local values include clean at line 1691. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1690: function assistantQuestionAllowsPublicLookup(question = "", intent = assistantQuestionIntent(question)) {
1691:   const clean = normalize(question);
1692:   return intent === "general_engineering"
1693:     || intent === "general_knowledge"
1694:     || /\b(github|linkedin|repo|repository|repositories|public\s+link|website|web\s+page|profile|resume|u
loaded\s+file|document|source\s+code)\b/.test(clean);
1695: }
```

assistantQuestionLooksConceptual (script.js, lines 1697-1702)

assistantQuestionLooksConceptual part of the public website runtime. In this file, it appears around lines 1697-1702 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, assistantQuestionIsEngineeringRelated, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1699: if (!assistantQuestionIsEngineeringRelated(question)) return false;; line 1700: return /^(what|what\s+is|what\s+are|what's|define|explain|describe|how|why|compare|differentiate)\b/.test(clean).

Important local values include clean at line 1698. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1697: function assistantQuestionLooksConceptual(question = "") {
1698:   const clean = normalize(question);
1699:   if (!assistantQuestionIsEngineeringRelated(question)) return false;
1700:   return
/^(what|what\s+is|what\s+are|what's|define|explain|describe|how|why|compare|differentiate)\b/.test(clean)
1701:   || /\b(definition|meaning|basics|overview|introduction|difference\s+between|how\s+does|how\s+do|why\s
+does|why\s+do)\b/.test(clean);
1702: }
```

assistantQuestionIntent (script.js, lines 1704-1717)

assistantQuestionIntent part of the public website runtime. In this file, it appears around lines 1704-1717 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantQuestionIsCasual, normalize, assistantQuestionHasPortfolioIntent, assistantConversationSuggestsPortfolioContext, b, assistantQuestionIsEngineeringRelated, and assistantQuestionLooksConceptual. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 1705: if (assistantQuestionIsCasual(question)) return "general_conversation";; line 1707: return "general_conversation";; line 1711: return "general_conversation";; line 1713: if (!hasPortfolioIntent && assistantQuestionIsEngineeringRelated(question)) return "general_engineering";. There are 3 additional return statements in the function body.

Important local values include hasPortfolioIntent at line 1709. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1704: function assistantQuestionIntent(question = "", history = assistantChatHistory) {
1705:   if (assistantQuestionIsCasual(question)) return "general_conversation";
1706:   if (/^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name\??$/i.test(normalize(question))) ||
/^\who\s+am\s+i\??$/i.test(normalize(question))) {
1707:     return "general_conversation";
1708:   }
1709:   const hasPortfolioIntent = assistantQuestionHasPortfolioIntent(question);
1710:   if (!hasPortfolioIntent && assistantConversationSuggestsPortfolioContext(history) &&
/b(name|you|your|he|his|h|him|that|this|it)\b/i.test(normalize(question))) {
1711:     return "general_conversation";
1712:   }
1713:   if (!hasPortfolioIntent && assistantQuestionIsEngineeringRelated(question)) return
"general_engineering";
1714:   if (!hasPortfolioIntent && assistantQuestionLooksConceptual(question)) return "general_engineering";
1715:   if (hasPortfolioIntent) return "portfolio_specific";
1716:   return "general_knowledge";
1717: }

```

assistantQuestionTokens (script.js, lines 1719-1730)

assistantQuestionTokens part of the public website runtime. In this file, it appears around lines 1719-1730 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, normalize, split, map, trim, filter, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1726: return normalize(question).

Important local values include stopWords at line 1720. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1719: function assistantQuestionTokens(question = "") {
1720:   const stopWords = new Set([
1721:     "about", "again", "also", "and", "are", "can", "could", "does", "for", "from", "give", "have", "his",
1722:     "into", "link", "links", "maurice", "me", "my", "open", "otieno", "please", "portfolio", "project",
1723:     "projects", "show", "tell", "that", "the", "their", "this", "what", "where", "which", "who", "with",
1724:     "work", "your"
1725:   ]);
1726:   return normalize(question)
1727:     .split(/[a-z0-9+#.]/i)
1728:     .map((token) => token.trim())
1729:     .filter((token) => token.length > 2 && !stopWords.has(token));
1730: }

```

assistantSpecificSourceTokens (script.js, lines 1732-1739)

assistantSpecificSourceTokens part of the public website runtime. In this file, it appears around lines 1732-1739 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, assistantQuestionTokens, filter, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1738: return assistantQuestionTokens(question).filter((token) => !genericTokens.has(token));.

Important local values include genericTokens at line 1733. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1732: function assistantSpecificSourceTokens(question = "") {
1733:   const genericTokens = new Set([
1734:     "built", "build", "created", "designed", "did", "done", "engineering", "file", "files", "hardware",
1735:     "item", "items", "section", "sections", "system", "systems", "thing", "things", "tool", "tools",
1736:     "use", "used", "using", "website"
1737:   ]);
1738:   return assistantQuestionTokens(question).filter((token) => !genericTokens.has(token));
1739: }

```

assistantPromptTargetsProjectLanding (script.js, lines 1741-1746)

assistantPromptTargetsProjectLanding part of the public website runtime. In this file, it appears around lines 1741-1746 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1743: return `/b(open|show|view|go|s+to|take|s+me|s+to)\b/.test(clean)`.

Important local values include clean at line 1742. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1741: function assistantPromptTargetsProjectLanding(question = "") {
1742:   const clean = normalize(question);
1743:   return /b(open|show|view|go|s+to|take|s+me|s+to)\b/.test(clean)
1744:     && /\b(project|overview|page)\b/.test(clean)
1745:     && !/\b(code|source|repo|repository|github|file|files|document|documents|test|tests|result|results|pc
b|schematic|simulation|tool|tools|resume|linkedin)\b/.test(clean);
1746: }
```

assistantEntryRelationScore (script.js, lines 1748-1762)

assistantEntryRelationScore part of the public website runtime. In this file, it appears around lines 1748-1762 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantSpecificSourceTokens, normalize, filter, join, includes, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1761: return relation;.

Important local values include tokens at line 1749; haystack at line 1750; tokenHits at line 1751; relation at line 1752. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1748: function assistantEntryRelationScore(result = {}, question = "") {
1749:   const tokens = assistantSpecificSourceTokens(question);
1750:   const haystack = normalize([result.title, result.context, result.type, result.text,
result.url].filter(Boolean).join(" "));
1751:   const tokenHits = tokens.filter((token) => haystack.includes(token)).length;
1752:   let relation = tokenHits * 30;
1753:
1754:   if (tokens.length && tokenHits === tokens.length) relation += 25;
1755:   if (Number(result.score || 0) >= 120) relation += 15;
1756:   if (Number(result.score || 0) >= 80) relation += 8;
1757:   if (result.kind === "project") relation += 12;
1758:   if (result.kind === "file" && /\b(file|files|download|document|documents|code|source|github|repo|reposit
ory|test|tests|result|results|pcb|schematic|simulation)\b/.test(normalize(question))) {
1759:     relation += 16;
1760:   }
1761:   return relation;
1762: }
```

assistantNamedProjectTopicTokens (script.js, lines 1764-1776)

assistantNamedProjectTopicTokens part of the public website runtime. In this file, it appears around lines 1764-1776 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantNamedProjectIds, Set, map, find, filter, forEach, normalize, split, add, assistantProjectTitleAcronyms, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1775: return assistantSpecificSourceTokens(question).filter((token) => !projectVocabulary.has(token));.

Important local values include projectVocabulary at line 1765; title at line 1770. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1764: function assistantNamedProjectTopicTokens(question = "", namedIds = assistantNamedProjectIds(question)) {
```

```

1765:   const projectVocabulary = new Set();
1766:   namedIds
1767:     .map((id) => projects.find((project) => project.id === id))
1768:     .filter(Boolean)
1769:     .forEach((project) => {
1770:       const title = project.portfolioView?.title || project.title || project.id;
1771:       normalize(title).split(/\s+/).filter((word) => word.length > 2).forEach((word) =>
projectVocabulary.add(word));
1772:       assistantProjectTitleAcronyms(title).forEach((acronym) => projectVocabulary.add(acronym));
1773:       projectVocabulary.add(normalize(project.id));
1774:     });
1775:   return assistantSpecificSourceTokens(question).filter((token) => !projectVocabulary.has(token));
1776: }

```

assistantSourcesForDisplay (script.js, lines 1778-1813)

assistantSourcesForDisplay part of the public website runtime. In this file, it appears around lines 1778-1813 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantQuestionIntent, assistantSpecificSourceTokens, assistantNamedProjectIds, Set, assistantNamedProjectTopicTokens, b, normalize, assistantPromptTargetsProjectLanding, map, assistantEntryRelationScore, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 1779: if (intent !== "portfolio_specific") return []; line 1789: if (!namedProjectIds.size && !pageLinkIntent && tokens.length === 0) return []; line 1794: if (!result) return false;; line 1795: if (namedProjectIds.size && !namedProjectIds.has(result.projectId)) return false;. There are 8 additional return statements in the function body.

Important local values include tokens at line 1780; namedProjectIdList at line 1781; namedProjectIds at line 1782; namedProjectTopicTokens at line 1783; asksForLinks at line 1784; asksForProjectLanding at line 1785; pageLinkIntent at line 1786; asksForSpecificArtifact at line 1787; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1778: function assistantSourcesForDisplay(question = "", results = [], intent =
assistantQuestionIntent(question)) {
1779:   if (intent !== "portfolio_specific") return [];
1780:   const tokens = assistantSpecificSourceTokens(question);
1781:   const namedProjectIdList = assistantNamedProjectIds(question);
1782:   const namedProjectIds = new Set(namedProjectIdList);
1783:   const namedProjectTopicTokens = assistantNamedProjectTopicTokens(question, namedProjectIdList);
1784:   const asksForLinks = /\b(open|show|where|link|links|github|linkedin|resume|download|file|files|repo|rep
ository|source\s+code)\b/.test(normalize(question));
1785:   const asksForProjectLanding = assistantPromptTargetsProjectLanding(question);
1786:   const pageLinkIntent = /\b(resume|github|linkedin|contact|email|phone)\b/.test(normalize(question));
1787:   const asksForSpecificArtifact = /\b(code|source|repo|repository|github|file|files|document|documents|te
st|tests|result|results|pcb|schematic|simulation|tool|tools)\b/.test(normalize(question));
1788:
1789:   if (!namedProjectIds.size && !pageLinkIntent && tokens.length === 0) return [];
1790:
1791:   const filtered = results
1792:     .map((result) => ({ ...result, relation: assistantEntryRelationScore(result, question) }));
1793:   .filter((result) => {
1794:     if (!result) return false;
1795:     if (namedProjectIds.size && !namedProjectIds.has(result.projectId)) return false;
1796:     if (asksForProjectLanding) return result.kind === "project";
1797:     const haystack = normalize([result.title, result.context, result.type, result.text,
result.url].filter(Boolean).join(" "));
1798:     const tokenHits = tokens.filter((token) => haystack.includes(token)).length;
1799:     const topicHits = namedProjectTopicTokens.filter((token) => haystack.includes(token)).length;
1800:     if (namedProjectIds.size && namedProjectTopicTokens.length && result.kind !== "project" &&
topicHits === 0) return false;
1801:     if (namedProjectIds.size && namedProjectTopicTokens.length && result.kind === "project" &&
asksForSpecificArtifact) return false;
1802:     if (namedProjectIds.size && result.kind === "project") return true;
1803:     if (pageLinkIntent && tokenHits >= 1) return true;
1804:     if (!tokens.length) return false;
1805:     if (tokenHits >= Math.min(2, tokens.length)) return true;
1806:     if (asksForLinks && tokenHits >= 1 && result.relation >= 35) return true;
1807:     return result.relation >= 55;
1808:   });
1809:
1810:   return filtered
1811:     .sort((a, b) => b.relation - a.relation || Number(b.score || 0) - Number(a.score || 0))
1812:     .slice(0, asksForLinks ? 4 : 2);
1813: }

```

assistantContextForQuestion (script.js, lines 1815-1890)

This function chooses which portfolio context should accompany an AI question. It identifies project names, profile links, evidence files, public URLs, and intent signals related to the visitor's prompt.

The assistant should not attach generic portfolio links to every answer. This function decides what context is actually related.

It calls or references `assistantQuestionIntent`, `includes`, `searchResultsFor`, `slice`, `Set`, `map`, `filter`, `assistantNamedProjectIds`, `find`, `b`, and 10 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1833: `if (!wantsPublicCode) return 0;`; line 1836: `return bGithub - aGithub;`; line 1860: `return {`.

Important local values include `directResults` at line 1816; `projectIds` at line 1819; `matchedProjects` at line 1823; `includePortfolioLinkContext` at line 1827; `wantsPublicCode` at line 1828; `publicProfileFetches` at line 1829; `aGithub` at line 1834; `bGithub` at line 1835; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1815: function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question),
knownResults = null) {
1816:   const directResults = ["general_engineering", "general_knowledge",
"general_conversation"].includes(intent)
1817:   ? []
1818:   : (knownResults || searchResultsFor(question)).slice(0, 8);
1819:   const projectIds = [...new Set([
1820:     ...directResults.map((item) => item.projectId).filter(Boolean),
1821:     ...assistantNamedProjectIds(question)
1822:   ]]);
1823:   const matchedProjects = projectIds
1824:     .map((id) => projects.find((project) => project.id === id))
1825:     .filter(Boolean)
1826:     .slice(0, 5);
1827:   const includePortfolioLinkContext = intent === "portfolio_specific";
1828:   const wantsPublicCode = /\b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|ver
ilog|systemverilog|python|javascript)\b/.test(normalize(question));
1829:   const publicProfileFetches = includePortfolioLinkContext
1830:     ? assistantPublicProfileLinks()
1831:     .map((item) => ({ ...item, question }))
1832:     .sort((a, b) => {
1833:       if (!wantsPublicCode) return 0;
1834:       const aGithub = /github/i.test(`${a.kind} || ""`);
1835:       const bGithub = /github/i.test(`${b.kind} || ""`);
1836:       return bGithub - aGithub;
1837:     })
1838:     : [];
1839:   const resultFetches = directResults.map((result) => ({
1840:     context: result.context,
1841:     kind: assistantUrlKind(result.url),
1842:     title: result.title,
1843:     type: result.type,
1844:     url: assistantAbsoluteUrl(result.url || ""),
1845:     question
1846:   }));
1847:   const projectFetches = matchedProjects.flatMap((project) => assistantProjectEvidence(project)
1848:     .filter((item) => ["text", "webpage", "github", "linkedin", "resume_text"].includes(item.kind))
1849:     .slice(0, 6)
1850:     .map((item) => ({ ...item, question })));
1851:   const sourceFetches = [
1852:     ...(wantsPublicCode ? publicProfileFetches : []),
1853:     ...resultFetches,
1854:     ...(wantsPublicCode ? [] : publicProfileFetches),
1855:     ...projectFetches
1856:   ].filter((item) => item.url);
1857:   const uniqueSourceFetches = sourceFetches.filter((item, index, array) => (
1858:     array.findIndex((candidate) => candidate.url === item.url) === index
1859:   )).slice(0, 14);
1860:   return {
1861:     categories: categories.map((category) => ({ id: category.id, label: category.label })),
1862:     intent,
1863:     profile: normalizeProfile(profile || {}),
1864:     knowledgeManifest: {
1865:       publicProfiles: includePortfolioLinkContext ? assistantPublicProfileLinks() : [],
1866:       publicSourcePolicy: "Only public profile and project links shown in the portfolio are used. GitHub
repository links can be expanded into repository metadata, README text, and selected public source files when
the visitor asks for code.",
1867:       matchedProjectEvidence: matchedProjects.map((project) => ({
1868:         evidence: assistantProjectEvidence(project),
1869:         projectId: project.id,
1870:         title: project.portfolioView?.title || project.title
1871:       })),
1872:       note: "Image records include filenames, captions, descriptions, and neighboring portfolio text. Use
a vision-capable backend before claiming visual details not present in captions or text."
1873:     },
1874:     portfolioContextPolicy: intent !== "portfolio_specific"
1875:       ? "Answer from general knowledge first. Do not lead with the portfolio owner's project context
unless the visitor explicitly asks to connect the concept to the portfolio."
1876:       : "Answer from the configured portfolio context first. Do not invent project details outside the
supplied context.",
```

```

1877:     projects: matchedProjects.map(assistantProjectSummary),
1878:     question,
1879:     results: directResults.map((result) => ({
1880:       context: result.context,
1881:       kind: result.kind,
1882:       text: searchSnippet(result.text, question),
1883:       title: result.title,
1884:       type: result.type,
1885:       url: result.url
1886:     })),
1887:     sourceFetches: uniqueSourceFetches,
1888:     siteSections: siteSections.filter(siteSectionRenderable).map((section) => ({ id: section.id, title:
section.title })))
1889:   };
1890: }

```

assistantRemember (script.js, lines 1892-1901)

assistantRemember part of the public website runtime. In this file, it appears around lines 1892-1901 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, push, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1895: if (!cleanContent) return;.

Important local values include cleanRole at line 1893; cleanContent at line 1894. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1892: function assistantRemember(role, content = "") {
1893:   const cleanRole = role === "assistant" ? "assistant" : "user";
1894:   const cleanContent = String(content || "").trim();
1895:   if (!cleanContent) return;
1896:   assistantChatHistory.push({
1897:     role: cleanRole,
1898:     content: cleanContent.slice(0, 1400)
1899:   });
1900:   assistantChatHistory = assistantChatHistory.slice(-10);
1901: }

```

assistantConversationForRequest (script.js, lines 1903-1912)

assistantConversationForRequest part of the public website runtime. In this file, it appears around lines 1903-1912 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice, map, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1911: return history;.

Important local values include history at line 1904. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1903: function assistantConversationForRequest(currentQuestion = "", priorConversation = assistantChatHistory)
{
1904:   const history = priorConversation.slice(-8).map((item) => ({
1905:     role: item.role === "assistant" ? "assistant" : "user",
1906:     content: String(item.content || "").slice(0, 1200)
1907:   }));
1908:   if (currentQuestion) {
1909:     history.push({ role: "user", content: String(currentQuestion).slice(0, 1200) });
1910:   }
1911:   return history;
1912: }

```

assistantFallbackResults (script.js, lines 1914-1972)

assistantFallbackResults part of the public website runtime. In this file, it appears around lines 1914-1972 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `assistantQuestionIntent`, `normalize`, `trim`, `assistantNamedProjectIds`, `searchResultsFor`, `filter`, `includes`, `slice`, `map`, `find`, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 1915: `if (intent !== "portfolio_specific") return [];` line 1917: `if (!clean) return [];` line 1922: `if (focusedResults.length) return focusedResults.slice(0, 6);` line 1924: `if (results.length) return results.slice(0, 6);`. There are 5 additional return statements in the function body.

Important local values include `clean` at line 1916; `namedProjectIds` at line 1918; `results` at line 1919; `focusedResults` at line 1921; `category` at line 1941. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1914: function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question)) {
1915:   if (intent !== "portfolio_specific") return [];
1916:   const clean = normalize(question).trim();
1917:   if (!clean) return [];
1918:   const namedProjectIds = assistantNamedProjectIds(question);
1919:   let results = searchResultsFor(clean);
1920:   if (namedProjectIds.length && results.length) {
1921:     const focusedResults = results.filter((result) => namedProjectIds.includes(result.projectId));
1922:     if (focusedResults.length) return focusedResults.slice(0, 6);
1923:   }
1924:   if (results.length) return results.slice(0, 6);
1925:
1926:   if (namedProjectIds.length) {
1927:     return namedProjectIds
1928:       .map((id) => projects.find((project) => project.id === id))
1929:       .filter(Boolean)
1930:       .map((project) => ({
1931:         context: slugLabel(project.category),
1932:         kind: "project",
1933:         projectId: project.id,
1934:         text: flattenSearchText([project.summary, project.focus, project.tools, project.languages]),
1935:         title: project.portfolioView?.title || project.title,
1936:         type: "Project"
1937:       })))
1938:     .slice(0, 6);
1939:   }
1940:
1941:   const category = categories.find((item) => clean.includes(normalize(item.label)) ||
clean.includes(normalize(item.id)));
1942:   if (category) {
1943:     return projects
1944:       .filter((project) => project.category === category.id)
1945:       .map((project) => ({
1946:         context: category.label,
1947:         kind: "project",
1948:         projectId: project.id,
1949:         text: flattenSearchText([project.summary, project.focus, project.tools, project.languages]),
1950:         title: project.portfolioView?.title || project.title,
1951:         type: "Project"
1952:       })))
1953:     .slice(0, 6);
1954:   }
1955:
1956:   if (/resume|professional|contact|email|phone|github|linkedin|profile/.test(clean)) {
1957:     return searchResultsFor("resume contact github professional profile").slice(0, 6);
1958:   }
1959:
1960:   if (/project|work|portfolio|hardware|design/.test(clean)) {
1961:     return projects.slice(0, 6).map((project) => ({
1962:       context: slugLabel(project.category),
1963:       kind: "project",
1964:       projectId: project.id,
1965:       text: project.summary || "",
1966:       title: project.portfolioView?.title || project.title,
1967:       type: "Project"
1968:     }));
1969:   }
1970:
1971:   return [];
1972: }
```

assistantQuestionIsEngineeringRelated (script.js, lines 1974-1977)

`assistantQuestionIsEngineeringRelated` part of the public website runtime. In this file, it appears around lines 1974-1977 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1976: return /\b(op\s*amp|amplifier|analog|mixed\s*signal|adc|dac|filter|vco|oscillator|pwm|charger|rectifier|regulator|buck|boost|ldo|mosfet|bjt|transistor|diode|pcb|schematic|layout|ground|noise|frequency|gain|phase|lts|pice|kicad|vivado|verilog|vhdl|fpga|asic|stm32|mcu|microcontroller|embedded|firmware|rtos|i2c|spi|uart|sensor|control|signal|circuit|electronics|hardware|power)\b/.test(clean);

Important local values include clean at line 1975. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1974: function assistantQuestionIsEngineeringRelated(question = "") {
1975:   const clean = normalize(question);
1976:   return /\b(op\s*amp|amplifier|analog|mixed\s*signal|adc|dac|filter|vco|oscillator|pwm|charger|rectifier|regulator|buck|boost|ldo|mosfet|bjt|transistor|diode|pcb|schematic|layout|ground|noise|frequency|gain|phase|lts|pice|kicad|vivado|verilog|vhdl|fpga|asic|stm32|mcu|microcontroller|embedded|firmware|rtos|i2c|spi|uart|sensor|control|signal|circuit|electronics|hardware|power)\b/.test(clean);
1977: }
```

assistantGeneralEngineeringAnswer (script.js, lines 1979-2091)

assistantGeneralEngineeringAnswer part of the public website runtime. In this file, it appears around lines 1979-2091 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, assistantQuestionIsEngineeringRelated, join, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 1981: if (!assistantQuestionIsEngineeringRelated(question)) return "";; line 1984: return []; line 2000: return []; line 2013: return []. There are 5 additional return statements in the function body.

Important local values include clean at line 1980. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1979: function assistantGeneralEngineeringAnswer(question = "") {
1980:   const clean = normalize(question);
1981:   if (!assistantQuestionIsEngineeringRelated(question)) return "";
1982:
1983:   if (/\\bembedded\\s+systems?\\b/.test(clean)) {
1984:     return [
1985:       "Embedded systems is a field of engineering focused on computers that are built into larger
products, machines, instruments, or control systems.",
1986:       "",
1987:       "Unlike a laptop or desktop computer, an embedded system is usually designed for a specific job. It
combines hardware, firmware, sensors, actuators, communication interfaces, and timing constraints so the
product can make measurements, control outputs, or respond to events reliably.",
1988:       "",
1989:       "Common examples include:",
1990:       "- A microcontroller controlling a motor drive.",
1991:       "- A sensor node collecting temperature, pressure, or motion data.",
1992:       "- A medical device running real-time measurement firmware.",
1993:       "- An automotive controller reading sensors and driving actuators.",
1994:       "",
1995:       "The important engineering ideas are timing, interrupts, power use, peripheral interfaces,
reliability, and how the firmware proves the hardware is behaving correctly."
1996:     ].join("\\n");
1997:   }
1998:
1999:   if (/\\b(op\\s*amp|amplifier|fully differential|differential)\\b/.test(clean)) {
2000:     return [
2001:       "A useful way to think about an op amp is as a high-gain error-correcting block. With negative
feedback, the circuit around it sets the useful behavior: gain, filtering, buffering, summing, integration, or
differential conversion.",
2002:       "",
2003:       "For a fully differential op amp:",
2004:       "- The signal is carried on two opposite-phase outputs instead of one output referenced to
ground.",
2005:       "- The differential output improves noise immunity and dynamic range.",
2006:       "- A common-mode feedback loop is normally needed to hold the average output voltage at the desired
common-mode level.",
2007:       "",
2008:       "In portfolio terms, this connects most naturally to analog and mixed-signal design, especially
filters, VCO control paths, ADC front ends, and precision measurement circuits."
2009:     ].join("\\n");
2010:   }
2011:
2012:   if (/\\b(pwm|vco|oscillator|frequency)\\b/.test(clean)) {
2013:     return [
2014:       "PWM-driven VCO work is about converting a digital timing signal into an analog control quantity
```

```

and then into a frequency.",
2015:     "",
2016:     "The usual signal chain is:",
2017:     "- PWM duty cycle encodes the control value.",
2018:     "- A low-pass filter turns the PWM waveform into an average voltage.",
2019:     "- The VCO maps that voltage to oscillation frequency.",
2020:     "- Tests compare duty cycle, control voltage, frequency range, ripple, and stability.",
2021:     "",
2022:     "That is a mixed-signal problem because digital timing, analog filtering, oscillator behavior, and
measurement all interact."
2023: ].join("\n");
2024: }
2025:
2026: if (/\\b(fpga|asic|verilog|vhdl|vivado|digital)\\b/.test(clean)) {
2027:     return [
2028:         "FPGA and ASIC work both implement digital hardware, but the engineering tradeoffs are different.",
2029:         "",
2030:         "- FPGA design is reconfigurable, fast to test, and useful for prototyping or deployment when
flexibility matters.",
2031:         "- ASIC design targets a fixed silicon implementation, so verification, timing closure, power,
area, and design-for-test become much more permanent decisions.",
2032:         "- Verilog or VHDL describes hardware structure and behavior; it is not software running line by
line.",
2033:         "",
2034:         "For a recruiter, strong evidence includes block diagrams, RTL, simulation waveforms, timing
reports, verification plans, and notes explaining design tradeoffs."
2035:     ].join("\n");
2036: }
2037:
2038: if (/\\b(stm32|mcu|microcontroller|embedded|firmware|rtos|i2c|spi|uart)\\b/.test(clean)) {
2039:     return [
2040:         "Embedded work is strongest when the firmware is tied clearly to hardware behavior.",
2041:         "",
2042:         "A good explanation usually covers:",
2043:         "- The MCU role in the system.",
2044:         "- The peripherals used, such as ADC, PWM, timers, I2C, SPI, UART, or GPIO.",
2045:         "- How timing, interrupts, sampling, power, and fault handling were managed.",
2046:         "- What was measured to prove the system worked.",
2047:         "",
2048:         "STM32CubeIDE is useful because it helps configure peripherals, generate startup code, debug
firmware, and manage the build flow around STM32 devices."
2049:     ].join("\n");
2050: }
2051:
2052: if (/\\b(pcb|layout|kicad|schematic|ground|noise)\\b/.test(clean)) {
2053:     return [
2054:         "PCB design is where the schematic becomes a physical electrical system.",
2055:         "",
2056:         "The important ideas are:",
2057:         "- Current returns matter as much as current paths.",
2058:         "- Grounding, decoupling, trace width, component placement, and loop area affect noise and
stability.",
2059:         "- Analog, power, and switching sections should be arranged so noisy currents do not corrupt
sensitive measurements.",
2060:         "- A good portfolio PCB section should show schematic intent, layout decisions, board files, bring-
up notes, and test results.",
2061:         "",
2062:         "KiCad evidence is strongest when the design files are paired with clear measurements or bring-up
photos."
2063:     ].join("\n");
2064: }
2065:
2066: if (/\\b(charger|rectifier|ac\\s*to\\s*dc|regulator|buck|boost|ldo|power)\\b/.test(clean)) {
2067:     return [
2068:         "An AC-to-DC charger or power supply usually has a few major stages.",
2069:         "",
2070:         "- Input protection and filtering handle transients and noise.",
2071:         "- Rectification converts AC into pulsating DC.",
2072:         "- Bulk capacitance smooths the rectified waveform.",
2073:         "- Regulation sets the desired output voltage or current.",
2074:         "- Load testing checks ripple, thermal behavior, regulation, and fault response.",
2075:         "",
2076:         "For portfolio presentation, the best proof is a clear schematic, expected waveforms, measured
output behavior, and a short explanation of component choices."
2077:     ].join("\n");
2078: }
2079:
2080: return [
2081:     "I can answer this as a general electronics or hardware-engineering question, even if it is not
directly tied to one saved project yet.",
2082:     "",
2083:     "The best way to approach it is:",
2084:     "- Define the signal or energy flow.",
2085:     "- Identify the active devices, passive networks, and control points.",
2086:     "- State what should be measured to prove the design works.",
2087:     "- Connect the explanation back to a project artifact: schematic, code, test data, PCB, or
simulation.",
2088:     "",
2089:     "If you ask about a specific circuit, tool, waveform, or failure mode, I can make the explanation
more concrete."
2090: ].join("\n");
2091: }

```

assistantLocalAnswer (script.js, lines 2093-2188)

This function builds a local fallback answer when the remote AI endpoint is unavailable or unnecessary.

The chat should still respond to greetings, basic portfolio questions, and common electronics prompts instead of failing silently when the backend is down.

It calls or references assistantQuestionIntent, normalizeProfile, normalize, b, join, assistantGeneralEngineeringAnswer, replace, Set, map, filter, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 11 return statement(s). The most important visible returns are: line 2098: return "Hi, what can I do for you?"; line 2101: return `I am an AI agent for \${ownerName}'s portfolio. I know the portfolio owner is \${ownerName}, but I do not know a visitor's personal name unless they tell me.`; line 2104: return `I am an AI agent for \${ownerName}'s portfolio. I can help with projects, resume links, public links, files, and related engineering questions.`; line 2106: return [. There are 7 additional return statements in the function body.

Important local values include ownerName at line 2094; clean at line 2096; generalAnswer at line 2118; generalAnswer at line 2128; projectIds at line 2152; matchedProjects at line 2153; projectNames at line 2156; fileCount at line 2157; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2093: function assistantLocalAnswer(question = "", results = [], intent = assistantQuestionIntent(question)) {
2094:   const ownerName = normalizeProfile(profile || {}).displayName || "the portfolio owner";
2095:   if (intent === "general_conversation") {
2096:     const clean = normalize(question);
2097:     if (/^(hi|hello|hey|yo|sup)\b/.test(clean)) {
2098:       return "Hi, what can I do for you?";
2099:     }
2100:     if (/^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name??$/i.test(clean) ||
/^who\s+am\s+i\??$/i.test(clean)) {
2101:       return `I am an AI agent for ${ownerName}'s portfolio. I know the portfolio owner is ${ownerName},
but I do not know a visitor's personal name unless they tell me.`;
2102:     }
2103:     if (/^(b|who are you|what are you)\b/.test(clean)) {
2104:       return `I am an AI agent for ${ownerName}'s portfolio. I can help with projects, resume links,
public links, files, and related engineering questions.`;
2105:     }
2106:     return [
2107:       `I am here with you. I can help explore ${ownerName}'s portfolio, explain projects, summarize
files, open relevant sections, or answer related engineering questions.`,
2108:       "",
2109:       "You can ask things like:",
2110:       "- What is embedded systems?",
2111:       "- Show me a project.",
2112:       "- What tools were used?",
2113:       "- Explain the difference between FPGA and ASIC design."
2114:     ].join("\n");
2115:   }
2116:
2117:   if (intent === "general_engineering") {
2118:     const generalAnswer = assistantGeneralEngineeringAnswer(question);
2119:     if (generalAnswer) return generalAnswer;
2120:     return [
2121:       "I can answer that as a general engineering question, but I need a little more detail to make the
answer useful.",
2122:       "",
2123:       "Try asking about a specific circuit, device, tool, protocol, waveform, or failure mode."
2124:     ].join("\n");
2125:   }
2126:
2127:   if (intent === "general_knowledge") {
2128:     const generalAnswer = assistantGeneralEngineeringAnswer(question);
2129:     if (generalAnswer) return generalAnswer;
2130:     return [
2131:       "I can answer that as a general question from the local fallback, but the live model will usually
give a deeper answer when it is reachable.",
2132:       "",
2133:       "Short answer:",
2134:       `~ ${question.replace(/\/\?+$/, "")} is a topic I can discuss by breaking it into definition,
purpose, main parts, and practical examples.`,
2135:       "- If the question connects to Maurice's portfolio, I will use the saved project context and links
as evidence.",
2136:       "- If it is broader than the portfolio, I will answer it as a general engineering or technology
concept first."
2137:     ].join("\n");
2138:   }
2139:
2140:   if (!projects.length) return "The project catalog is still loading. Try again in a moment.";
2141:   if (!results.length) {
2142:     return [
2143:       `I could not find a strong portfolio match for "${question}".`,
2144:       "",
2145:       "Try asking about:",
2146:       "- A project name or category.",
2147:       "- A tool such as LTspice, KiCad, Vivado, STM32CubeIDE, or GitHub.",

```

```

2148:         "- A hardware topic such as op amps, PCB testing, VCOs, embedded firmware, FPGA, ASIC, or power
circuits."
2149:     ].join("\n");
2150: }
2151:
2152: const projectIds = [...new Set(results.map((item) => item.projectId).filter(Boolean))];
2153: const matchedProjects = projectIds
2154:     .map((id) => projects.find((project) => project.id === id))
2155:     .filter(Boolean);
2156: const projectNames = matchedProjects.map((project) => project.portfolioView?.title ||
project.title).slice(0, 3);
2157: const fileCount = results.filter((item) => item.kind === "file").length;
2158: const sectionCount = results.filter((item) => item.kind === "section" || item.kind ===
"subsection").length;
2159: const pageCount = results.filter((item) => item.kind === "page").length;
2160: const parts = [];
2161:
2162: if (projectNames.length) {
2163:     parts.push(`The strongest portfolio match is ${projectNames.join(projectNames.length > 2 ? ", " : "
and ")}.`);
2164: } else if (pageCount) {
2165:     parts.push("The strongest matches are in the profile, resume, contact, or portfolio information
sections.");
2166: } else {
2167:     parts.push(`I found ${results.length} relevant portfolio item${results.length === 1 ? "" : "s"}.`);
2168: }
2169:
2170: parts.push("");
2171: parts.push("What I found:");
2172: if (sectionCount || fileCount) {
2173:     parts.push(`- ${sectionCount} section match${sectionCount === 1 ? "" : "es"}.`);
2174:     if (fileCount) parts.push(`- ${fileCount} file or link match${fileCount === 1 ? "" : "es"}.`);
2175: }
2176: if (pageCount) parts.push(`- ${pageCount} page or profile match${pageCount === 1 ? "" : "es"}.`);
2177: if (projectNames.length) {
2178:     parts.push(`- Best project candidate${projectNames.length === 1 ? "" : "s"}: ${projectNames.join(",
")}.`);
2179: }
2180:
2181: const firstSnippet = searchSnippet(results[0].text, question);
2182: if (firstSnippet) {
2183:     parts.push("");
2184:     parts.push("Why it matched:");
2185:     parts.push(firstSnippet);
2186: }
2187: return parts.join("\n");
2188: }

```

assistantSourcesMarkup (script.js, lines 2190-2208)

assistantSourcesMarkup part of the public website runtime. In this file, it appears around lines 2190-2208 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice, map, set, escapeHtml, assistantSourceLabel, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2192: if (!sources.length) return "";; line 2196: return `; line 2200: return `.

Important local values include sources at line 2191; sourceButtons at line 2193; id at line 2194. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2190: function assistantSourcesMarkup(results = []) {
2191:     const sources = results.slice(0, 5);
2192:     if (!sources.length) return "";
2193:     const sourceButtons = sources.map((source) => {
2194:         const id = String(assistantSourceCounter++);
2195:         assistantSourceMap.set(id, source);
2196:         return `
2197:             <button type="button" data-ai-source-
id=${id}>${escapeHtml(assistantSourceLabel(source))}</button>
2198:         `;
2199:     }).join("");
2200:     return
2201:         <div class="ai-source-panel">
2202:             <p class="ai-source-heading">Relevant portfolio links</p>
2203:             <div class="ai-source-list" aria-label="Assistant sources">
2204:                 ${sourceButtons}
2205:             </div>
2206:         </div>
2207:     `;
2208: }

```

setAssistantStatus (script.js, lines 2210-2214)

setAssistantStatus part of the public website runtime. In this file, it appears around lines 2210-2214 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2211: if (!aiAssistantStatus) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2210: function setAssistantStatus(message, state = "ready") {
2211:   if (!aiAssistantStatus) return;
2212:   aiAssistantStatus.textContent = message;
2213:   aiAssistantStatus.dataset.state = state;
2214: }
```

assistantDesktopDockedHeight (script.js, lines 2216-2221)

assistantDesktopDockedHeight part of the public website runtime. In this file, it appears around lines 2216-2221 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getComputedStyle, getPropertyValue, trim, endsWith, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2217: if (!aiAssistantPanel) return 360;; line 2219: if (value.endsWith("px")) return Number.parseFloat(value) || 360;; line 2220: return Math.min(Math.max(window.innerHeight * 0.38, 340), 390);.

Important local values include value at line 2218. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2216: function assistantDesktopDockedHeight() {
2217:   if (!aiAssistantPanel) return 360;
2218:   const value = getComputedStyle(aiAssistantPanel).getPropertyValue("--ai-console-rest-height").trim();
2219:   if (value.endsWith("px")) return Number.parseFloat(value) || 360;
2220:   return Math.min(Math.max(window.innerHeight * 0.38, 340), 390);
2221: }
```

updateAssistantPanelGrowth (script.js, lines 2223-2246)

updateAssistantPanelGrowth updates ui, state, cache, or stored data. In this file, it appears around lines 2223-2246 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references matchMedia, removeProperty, remove, assistantDesktopDockedHeight, getBoundingClientRect, querySelector, querySelectorAll, reduce, min, round, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2224: if (!aiAssistantPanel || !aiAssistantLog || !aiAssistantForm) return;; line 2229: return;.

Important local values include isDesktop at line 2225; dockedHeight at line 2232; formHeight at line 2233; clearHeight at line 2234; titleHeight at line 2235; chromeHeight at line 2236; messages at line 2237; messageGap at line 2238; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2223: function updateAssistantPanelGrowth() {
2224:   if (!aiAssistantPanel || !aiAssistantLog || !aiAssistantForm) return;
2225:   const isDesktop = window.matchMedia("(min-width: 901px)").matches;
2226:   if (!isDesktop) {
2227:     aiAssistantPanel.style.removeProperty("--ai-console-live-height");
2228:     aiAssistantPanel.classList.remove("ai-assistant-panel-expanded");
2229:     return;
2230:   }
2231:   // ... (rest of the function body)
2246: }
```

```

2230:   }
2231:
2232:   const dockedHeight = assistantDesktopDockedHeight();
2233:   const formHeight = aiAssistantForm.getBoundingClientRect().height || 46;
2234:   const clearChatButton?.getBoundingClientRect().height || 38;
2235:   const titleHeight = document.querySelector("#ai-assistant-title")?.getBoundingClientRect().height ||
34;
2236:   const chromeHeight = titleHeight + formHeight + clearHeight + 92;
2237:   const messages = [...aiAssistantLog.querySelectorAll(".ai-message")];
2238:   const messageGap = messages.length > 1 ? (messages.length - 1) * 10 : 0;
2239:   const messageHeight = messages.reduce((sum, message) => sum + message.getBoundingClientRect().height,
0) + messageGap;
2240:   const contentHeight = messageHeight + chromeHeight;
2241:   const maxHeight = Math.min(window.innerHeight * 0.76, 720);
2242:   const nextHeight = Math.round(Math.max(dockedHeight, Math.min(contentHeight, maxHeight)));
2243:
2244:   aiAssistantPanel.style.setProperty("--ai-console-live-height", `${nextHeight}px`);
2245:   aiAssistantPanel.classList.toggle("ai-assistant-panel-expanded", nextHeight > dockedHeight + 12);
2246: }

```

renderAssistantInline (script.js, lines 2248-2250)

renderAssistantInline renders ui, markup, or display output. In this file, it appears around lines 2248-2250 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderInlineMath, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2249: return renderInlineMath(value).replace(/\^*([^\^]+)\^*/g, "\$1");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2248: function renderAssistantInline(value = "") {
2249:   return renderInlineMath(value).replace(/\^*([^\^]+)\^*/g, "<strong>$1</strong>");
2250: }

```

renderAssistantAnswerContent (script.js, lines 2252-2304)

renderAssistantAnswerContent renders ui, markup, or display output. In this file, it appears around lines 2252-2304 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, split, push, map, renderAssistantInline, join, escapeHtml, forEach, trim, match, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 2258: if (!bulletItems.length) return;; line 2263: if (!codeBlock) return;; line 2279: return;; line 2283: return;. There are 3 additional return statements in the function body.

Important local values include lines at line 2253; html at line 2254; bulletItems at line 2255; codeBlock at line 2256; flushBullets at line 2257; flushCode at line 2262; language at line 2264; trimmed at line 2270; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2252: function renderAssistantAnswerContent(content = "") {
2253:   const lines = String(content || "").replace(/\r\n?/g, "\n").split("\n");
2254:   const html = [];
2255:   let bulletItems = [];
2256:   let codeBlock = null;
2257:   const flushBullets = () => {
2258:     if (!bulletItems.length) return;
2259:     html.push(`<ul>${bulletItems.map((item) =>
`<li>${renderAssistantInline(item)}</li>`).join("")}</ul>`);
2260:     bulletItems = [];
2261:   };
2262:   const flushCode = () => {
2263:     if (!codeBlock) return;
2264:     const language = codeBlock.language ? ` data-language="${escapeHtml(codeBlock.language)}"` : "";
2265:     html.push(`<pre class="ai-code-
block"${language}><code>${escapeHtml(codeBlock.lines.join("\n"))}</code></pre>`);
2266:     codeBlock = null;
2267:   };
2268: }

```

```

2269:   lines.forEach((line) => {
2270:     const trimmed = line.trim();
2271:     const fence = trimmed.match(/^```([a-z0-9_+.#-]*)\s*$/i);
2272:     if (fence) {
2273:       if (codeBlock) {
2274:         flushCode();
2275:       } else {
2276:         flushBullets();
2277:         codeBlock = { language: fence[1] || "", lines: [] };
2278:       }
2279:       return;
2280:     }
2281:     if (codeBlock) {
2282:       codeBlock.lines.push(line);
2283:       return;
2284:     }
2285:     if (!trimmed) {
2286:       flushBullets();
2287:       return;
2288:     }
2289:     const bullet = trimmed.match(/^[-*\u2022]\s+(.+)/u);
2290:     if (bullet) {
2291:       bulletItems.push(bullet[1]);
2292:       return;
2293:     }
2294:     flushBullets();
2295:     if (/^[^!?!?]{2,64}:$.test(trimmed)) {
2296:       html.push(`<p><strong>${renderAssistantInline(trimmed.slice(0, -1))}</strong></p>`);
2297:     } else {
2298:       html.push(`<p>${renderAssistantInline(trimmed)}</p>`);
2299:     }
2300:   });
2301:   flushCode();
2302:   flushBullets();
2303:   return `<div class="ai-answer-content">${html.join("")} || "<p>I am ready.</p>"</div>`;
2304: }

```

flushBullets (script.js, lines 2257-2261)

flushBullets part of the public website runtime. In this file, it appears around lines 2257-2261 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, map, renderAssistantInline, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2258: if (!bulletItems.length) return;.

Important local values include flushBullets at line 2257. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2257:   const flushBullets = () => {
2258:     if (!bulletItems.length) return;
2259:     html.push(`<ul>${bulletItems.map((item) =>
`<li>${renderAssistantInline(item)}</li>`).join("")}</ul>`);
2260:     bulletItems = [];
2261:   };

```

flushCode (script.js, lines 2262-2267)

flushCode part of the public website runtime. In this file, it appears around lines 2262-2267 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2263: if (!codeBlock) return;.

Important local values include flushCode at line 2262; language at line 2264. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2262:   const flushCode = () => {
2263:     if (!codeBlock) return;
2264:     const language = codeBlock.language ? ` data-language="${escapeHtml(codeBlock.language)}" ` : "";

```

```

2265:     html.push(`<pre class="ai-code-
block"${language}><code>${escapeHtml(codeBlock.lines.join("\n"))}</code></pre>`);
2266:     codeBlock = null;
2267:   };

```

appendAssistantMessage (script.js, lines 2306-2317)

appendAssistantMessage part of the public website runtime. In this file, it appears around lines 2306-2317 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, renderAssistantAnswerContent, assistantSourcesMarkup, renderMultilineInlineText, append, and updateAssistantPanelGrowth. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2307: if (!aiAssistantLog) return;; line 2316: return article;.

Important local values include article at line 2308. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2306: function appendAssistantMessage(role, content, sources = []) {
2307:   if (!aiAssistantLog) return;
2308:   const article = document.createElement("article");
2309:   article.className = `ai-message ai-message-${role}`;
2310:   article.innerHTML = role === "assistant"
2311:     ? `${renderAssistantAnswerContent(content)}${assistantSourcesMarkup(sources)}`
2312:     : `<p>${renderMultilineInlineText(content)}</p>`;
2313:   aiAssistantLog.append(article);
2314:   updateAssistantPanelGrowth();
2315:   aiAssistantLog.scrollTop = aiAssistantLog.scrollHeight;
2316:   return article;
2317: }

```

scrollAssistantAnswerToStart (script.js, lines 2319-2329)

scrollAssistantAnswerToStart part of the public website runtime. In this file, it appears around lines 2319-2329 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, and requestAnimationFrame. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2320: if (!aiAssistantLog || !article) return;.

Important local values include scrollToAnswer at line 2321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2319: function scrollAssistantAnswerToStart(article) {
2320:   if (!aiAssistantLog || !article) return;
2321:   const scrollToAnswer = () => {
2322:     aiAssistantLog.scrollTop = Math.max(0, article.offsetTop - aiAssistantLog.offsetTop - 4);
2323:   };
2324:   if (typeof window.requestAnimationFrame === "function") {
2325:     window.requestAnimationFrame(scrollToAnswer);
2326:   } else {
2327:     window.setTimeout(scrollToAnswer, 0);
2328:   }
2329: }

```

scrollToAnswer (script.js, lines 2321-2323)

scrollToAnswer part of the public website runtime. In this file, it appears around lines 2321-2323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include `scrollToAnswer` at line 2321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2321: const scrollToAnswer = () => {
2322:   aiAssistantLog.scrollTop = Math.max(0, article.offsetTop - aiAssistantLog.offsetTop - 4);
2323: };
```

appendAssistantPendingMessage (script.js, lines 2331-2335)

`appendAssistantPendingMessage` part of the public website runtime. In this file, it appears around lines 2331-2335 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `appendAssistantMessage`, and `add`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2334: `return article;`.

Important local values include `article` at line 2332. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2331: function appendAssistantPendingMessage() {
2332:   const article = appendAssistantMessage("assistant", "Thinking", []);
2333:   article?.classList.add("ai-message-pending");
2334:   return article;
2335: }
```

replaceAssistantMessage (script.js, lines 2337-2346)

`replaceAssistantMessage` part of the public website runtime. In this file, it appears around lines 2337-2346 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `appendAssistantMessage`, `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `updateAssistantPanelGrowth`, and `scrollAssistantAnswerToStart`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2340: `return;`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2337: function replaceAssistantMessage(article, content, sources = []) {
2338:   if (!article) {
2339:     appendAssistantMessage("assistant", content, sources);
2340:     return;
2341:   }
2342:   article.className = "ai-message ai-message-assistant";
2343:   article.innerHTML = `${renderAssistantAnswerContent(content)}${assistantSourcesMarkup(sources)}`;
2344:   updateAssistantPanelGrowth();
2345:   scrollAssistantAnswerToStart(article);
2346: }
```

setAssistantBusy (script.js, lines 2348-2352)

`setAssistantBusy` part of the public website runtime. In this file, it appears around lines 2348-2352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `querySelector`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include `submitButton` at line 2349. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2348: function setAssistantBusy(isBusy) {
2349:   const submitButton = aiAssistantForm?.querySelector('button[type="submit"]');
2350:   if (aiAssistantInput) aiAssistantInput.disabled = isBusy;
2351:   if (submitButton) submitButton.disabled = isBusy;
2352: }
```

clearAssistantChat (script.js, lines 2354-2363)

clearAssistantChat part of the public website runtime. In this file, it appears around lines 2354-2363 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Map, setAssistantStatus, updateAssistantPanelGrowth, and focus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2355: if (!aiAssistantLog) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2354: function clearAssistantChat() {
2355:   if (!aiAssistantLog) return;
2356:   assistantSourceCounter = 0;
2357:   assistantSourceMap = new Map();
2358:   assistantChatHistory = [];
2359:   aiAssistantLog.innerHTML = "";
2360:   setAssistantStatus("Chat cleared");
2361:   updateAssistantPanelGrowth();
2362:   aiAssistantInput?.focus();
2363: }
```

assistantRemoteTimeoutMs (script.js, lines 2365-2374)

assistantRemoteTimeoutMs part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2365-2374 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, isArray, some, toLowerCase, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2370: return 45000;; line 2372: if (options.allowWebSearch) return 30000;; line 2373: return 18000;.

Important local values include clean at line 2366; sourceFetches at line 2367; usesGitHub at line 2368. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2365: function assistantRemoteTimeoutMs(question = "", context = {}, options = {}) {
2366:   const clean = normalize(question);
2367:   const sourceFetches = Array.isArray(context.sourceFetches) ? context.sourceFetches : [];
2368:   const usesGitHub = sourceFetches.some((source) => String(source.kind || "").toLowerCase() === "github"
|| /github\.com\/i.test(source.url || ""));
2369:   if (usesGitHub || /\b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|verilog|s
ystemverilog|python|javascript|circuit\s+file)\b/.test(clean)) {
2370:     return 45000;
2371:   }
2372:   if (options.allowWebSearch) return 30000;
2373:   return 18000;
2374: }
```

askRemoteAssistant (script.js, lines 2376-2411)

This function sends the user's question and selected context to the configured AI endpoint, waits with a timeout, and returns the remote answer if available.

It is the browser side of the website AI backend call. It lets the public site use Cloudflare/OpenAI/Workers AI without exposing implementation details in the UI.

It calls or references assistantEndpoint, AbortController, abort, assistantRemoteTimeoutMs, fetch, stringify, Error, json, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage

keys detected.

It has 2 return statement(s). The most important visible returns are: line 2378: if (!endpoint) return null;; line 2410: return answer ? { answer, model: data.model || "", usedWebSearch: Boolean(data.usedWebSearch) } : null;.

Important local values include endpoint at line 2377; controller at line 2379; didTimeout at line 2380; timer at line 2381; data at line 2408; answer at line 2409. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2376: async function askRemoteAssistant(question, context, options = {}) {
2377:   const endpoint = assistantEndpoint();
2378:   if (!endpoint) return null;
2379:   const controller = new AbortController();
2380:   let didTimeout = false;
2381:   const timer = setTimeout(() => {
2382:     didTimeout = true;
2383:     controller.abort();
2384:   }, assistantRemoteTimeoutMs(question, context, options));
2385:   let response;
2386:   try {
2387:     response = await fetch(endpoint, {
2388:       method: "POST",
2389:       headers: { "Content-Type": "application/json" },
2390:       signal: controller.signal,
2391:       body: JSON.stringify({
2392:         context,
2393:         conversation: options.conversation || [],
2394:         intent: options.intent || context.intent || "portfolio_specific",
2395:         allowWebSearch: Boolean(options.allowWebSearch),
2396:         question,
2397:         mode: "portfolio-plus-general-knowledge",
2398:         responseType: "chatgpt-like-answer-first-links-second"
2399:       })
2400:     });
2401:   } catch (error) {
2402:     if (didTimeout) throw new Error("AI request timed out while reading public sources.");
2403:     throw error;
2404:   } finally {
2405:     clearTimeout(timer);
2406:   }
2407:   if (!response.ok) throw new Error("AI service unavailable");
2408:   const data = await response.json();
2409:   const answer = String(data.answer || data.message || "").trim();
2410:   return answer ? { answer, model: data.model || "", usedWebSearch: Boolean(data.usedWebSearch) } : null;
2411: }
```

answerAssistantQuestion (script.js, lines 2413-2458)

This function coordinates the full chat response flow: classify the question, build context, try the remote assistant, fall back locally if needed, and render the response.

It is the public website's AI conversation orchestrator. It keeps the interface conversational while still grounding answers in portfolio context when appropriate.

It calls or references trim, slice, appendAssistantMessage, assistantRemember, appendAssistantPendingMessage, setAssistantBusy, setAssistantStatus, assistantEndpoint, assistantQuestionIntent, assistantFallbackResults, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2415: if (!cleanQuestion) return;; line 2439: return;;

Important local values include cleanQuestion at line 2414; priorConversation at line 2416; pendingMessage at line 2419; intent at line 2424; rawSources at line 2425; sources at line 2426; context at line 2427; conversation at line 2428; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2413: async function answerAssistantQuestion(question = "") {
2414:   const cleanQuestion = String(question || "").trim();
2415:   if (!cleanQuestion) return;
2416:   const priorConversation = assistantChatHistory.slice();
2417:   appendAssistantMessage("user", cleanQuestion);
2418:   assistantRemember("user", cleanQuestion);
2419:   const pendingMessage = appendAssistantPendingMessage();
2420:   setAssistantBusy(true);
2421:   try {
2422:     setAssistantStatus(assistantEndpoint() ? "Composing AI answer" : "Composing local answer",
"working");
2423:
2424:     const intent = assistantQuestionIntent(cleanQuestion, priorConversation);
2425:     const rawSources = assistantFallbackResults(cleanQuestion, intent);
2426:     const sources = assistantSourcesForDisplay(cleanQuestion, rawSources, intent);
2427:     const context = assistantContextForQuestion(cleanQuestion, intent, rawSources);
2428:     const conversation = assistantConversationForRequest(cleanQuestion, priorConversation);
2429:     try {
```

```

2430:     const remoteAnswer = await askRemoteAssistant(cleanQuestion, context, {
2431:       allowWebSearch: assistantQuestionAllowsPublicLookup(cleanQuestion, intent),
2432:       conversation,
2433:       intent
2434:     });
2435:     if (remoteAnswer?.answer) {
2436:       replaceAssistantMessage(pendingMessage, remoteAnswer.answer, sources);
2437:       assistantRemember("assistant", remoteAnswer.answer);
2438:       setAssistantStatus(remoteAnswer.model ? `AI answer ready: ${remoteAnswer.model}` : "AI answer
ready");
2439:       return;
2440:     }
2441:   } catch (error) {
2442:     setAssistantStatus(error?.message || "Local answer used", "error");
2443:   }
2444:
2445:   const fallbackAnswer = assistantLocalAnswer(cleanQuestion, sources, intent);
2446:   replaceAssistantMessage(pendingMessage, fallbackAnswer, sources);
2447:   assistantRemember("assistant", fallbackAnswer);
2448:   if (!assistantEndpoint()) setAssistantStatus("Local portfolio intelligence");
2449: } catch {
2450:   const errorAnswer = "I had trouble processing that question. Try rephrasing it with a project name,
tool name, or electronics topic.";
2451:   replaceAssistantMessage(pendingMessage, errorAnswer, []);
2452:   assistantRemember("assistant", errorAnswer);
2453:   setAssistantStatus("Assistant error", "error");
2454: } finally {
2455:   setAssistantBusy(false);
2456:   aiAssistantInput?.focus();
2457: }
2458: }

```

linkAttributes (script.js, lines 2459-2462)

linkAttributes part of the public website runtime. In this file, it appears around lines 2459-2462 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, and isWebsiteLinkItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2461: return /^https?:\V/i.test(target) ? ' target="_blank" rel="noreferrer"' : "";

Important local values include target at line 2460. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2459: function linkAttributes(url, item = {}) {
2460:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) });
2461:   return /^https?:\V/i.test(target) ? ' target="_blank" rel="noreferrer"' : "";
2462: }

```

isLocalDownloadTarget (script.js, lines 2464-2467)

isLocalDownloadTarget part of the public website runtime. In this file, it appears around lines 2464-2467 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2466: return Boolean(value) && !/^(https?:)\V/i.test(value) && !/^(mailto:|tel:|#)/i.test(value);

Important local values include value at line 2465. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2464: function isLocalDownloadTarget(target = "") {
2465:   const value = normalizeLinkTarget(target, { assumeWeb: true });
2466:   return Boolean(value) && !/^(https?:)\V/i.test(value) && !/^(mailto:|tel:|#)/i.test(value);
2467: }

```

downloadAttribute (script.js, lines 2469-2472)

downloadAttribute part of the public website runtime. In this file, it appears around lines 2469-2472 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, isWebsiteLinkItem, and isLocalDownloadTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2471: return isLocalDownloadTarget(value) ? " download" : "";

Important local values include value at line 2470. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2469: function downloadAttribute(target = "", item = {}) {
2470:   const value = normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) });
2471:   return isLocalDownloadTarget(value) ? " download" : "";
2472: }
```

fileNameFromUrl (script.js, lines 2474-2481)

fileNameFromUrl part of the public website runtime. In this file, it appears around lines 2474-2481 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, and pop. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2477: return decodeURIComponent(clean); line 2479: return clean;

Important local values include clean at line 2475. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2474: function fileNameFromUrl(url = "") {
2475:   const clean = String(url || "").split(/[?#]/)[0].split("/").pop() || "Download file";
2476:   try {
2477:     return decodeURIComponent(clean);
2478:   } catch {
2479:     return clean;
2480:   }
2481: }
```

resourceLink (script.js, lines 2483-2501)

resourceLink part of the public website runtime. In this file, it appears around lines 2483-2501 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileNameFromUrl, normalizeLinkTarget, isWebsiteLinkItem, escapeHtml, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2488: return `

Important local values include rawTarget at line 2484; target at line 2485. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2483: function resourceLink(item = {}, label = item.label || item.title || item.name ||
fileNameFromUrl(item.url || item.artifact)) {
2484:   const rawTarget = item.url || item.artifact || item.href || item.file || item.path || item.src || "";
2485:   const target = normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) });
2486:
2487:   if (!target || item.status === "planned") {
2488:     return `
```

```

2492:     <a
2493:         class="resource-link"
2494:         href="${escapeHtml(target)}"
2495:         ${linkAttributes(target, item)}
2496:         ${downloadAttribute(target, item)}
2497:     >
2498:         ${escapeHtml(label || fileNameFromUrl(target))}
2499:     </a>
2500: `;
2501: }

```

pillList (script.js, lines 2503-2510)

pillList part of the public website runtime. In this file, it appears around lines 2503-2510 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2504: return (items || []); line 2507: return label ? `<span class="tag \${className}">\${label}</span>` : "".

Important local values include label at line 2506. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2503: function pillList(items, className = "") {
2504:     return (items || [])
2505:         .map((item) => {
2506:             const label = typeof item === "string" ? item : item.name || item.title || item.label || "";
2507:             return label ? `<span class="tag ${className}">${label}</span>` : "";
2508:         })
2509:         .join("");
2510: }

```

evidenceList (script.js, lines 2512-2518)

evidenceList part of the public website runtime. In this file, it appears around lines 2512-2518 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2514: return `<p class="evidence-empty">\${emptyMessage}</p>`; line 2517: return `<ul>\${items.map(renderItem).join("")}</ul>`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2512: function evidenceList(items, renderItem, emptyMessage) {
2513:     if (!items || !items.length) {
2514:         return `<p class="evidence-empty">${emptyMessage}</p>`;
2515:     }
2516:
2517:     return `<ul>${items.map(renderItem).join("")}</ul>`;
2518: }

```

detailBlock (script.js, lines 2520-2527)

detailBlock part of the public website runtime. In this file, it appears around lines 2520-2527 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2521: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2520: function detailBlock(title, className, content) {
2521:   return `
2522:     <details class="${className}">
2523:       <summary>${title}</summary>
2524:       ${content}
2525:     </details>
2526:   `;
2527: }
```

mediaGrid (script.js, lines 2529-2549)

mediaGrid part of the public website runtime. In this file, it appears around lines 2529-2549 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2531: return `

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2529: function mediaGrid(items) {
2530:   if (!items || !items.length) {
2531:     return `
```

siteSectionHasContent (script.js, lines 2551-2560)

siteSectionHasContent part of the public website runtime. In this file, it appears around lines 2551-2560 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2552: return Boolean(

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2551: function siteSectionHasContent(section) {
2552:   return Boolean(
2553:     section?.description ||
2554:     section?.richDescription?.blocks?.length ||
2555:     section?.backgroundImage ||
2556:     (section?.links || []).some((link) => link.label || link.url) ||
2557:     (section?.assets || []).some((asset) => asset.title || asset.url) ||
2558:     (section?.subsections || []).some((item) => item.title || item.description ||
item.richDescription?.blocks?.length || (item.links || []).length)
2559:   );
2560: }
```

siteSectionRenderable (script.js, lines 2562-2564)

siteSectionRenderable part of the public website runtime. In this file, it appears around lines 2562-2564 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references siteSectionHasContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2563: return section?.visible !== false && siteSectionHasContent(section);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2562: function siteSectionRenderable(section) {
2563:   return section?.visible !== false && siteSectionHasContent(section);
2564: }
```

renderDynamicLinks (script.js, lines 2566-2570)

renderDynamicLinks renders ui, markup, or display output. In this file, it appears around lines 2566-2570 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, map, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2568: if (!links.length) return "";; line 2569: return `<div class="resource-list dynamic-link-list">\${links.map((item) => resourceLink(item, item.label || item.title || "Open")).join("")}</div>`.

Important local values include links at line 2567. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2566: function renderDynamicLinks(items = []) {
2567:   const links = items.filter((item) => item?.url && (item.label || item.title));
2568:   if (!links.length) return "";
2569:   return `<div class="resource-list dynamic-link-list">${links.map((item) => resourceLink(item,
item.label || item.title || "Open")).join("")}</div>`;
2570: }
```

renderSiteSections (script.js, lines 2572-2609)

renderSiteSections renders ui, markup, or display output. In this file, it appears around lines 2572-2609 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, filter, map, url, escapeHtml, renderRichContent, renderDynamicLinks, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2574: if (!mount) return;; line 2580: return `

Important local values include mount at line 2573; visibleSections at line 2575; style at line 2577; links at line 2578; subsections at line 2579. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2572: function renderSiteSections() {
2573:   const mount = document.querySelector("#dynamic-sections");
2574:   if (!mount) return;
2575:   const visibleSections = (siteSections || []).filter(siteSectionRenderable);
2576:   mount.innerHTML = visibleSections.map((section) => {
2577:     const style = section.backgroundImage ? ` style="--section-bg:
url('${escapeHtml(section.backgroundImage)}')` : "";
2578:     const links = [...(section.links || []), ...(section.assets || [])];
2579:     const subsections = (section.subsections || []).filter((item) => item.title || item.description ||
item.richDescription?.blocks?.length || (item.links || []).length);
2580:     return `
2581:       <section class="section dynamic-section" id="${escapeHtml(section.id || "")}"${style}>
2582:         <div class="dynamic-section-surface">
```



```

2583:         <div class="section-heading">
2584:             <div>
2585:                 <h2>${escapeHtml(section.title || "Untitled section")}</h2>
2586:             </div>
2587:         </div>
2588:         ${section.description || section.richDescription?.blocks?.length
2589:         ? `<div class="dynamic-section-copy">${renderRichContent(section.richDescription,
section.description || "")}</div>`
2590:         : ""}
2591:         ${subsections.length ? `
2592:         <div class="dynamic-section-grid">
2593:             ${subsections.map((item) => `
2594:                 <article class="dynamic-section-card">
2595:                     <h3>${escapeHtml(item.title || "Untitled")}</h3>
2596:                     ${item.description || item.richDescription?.blocks?.length
2597:                     ? renderRichContent(item.richDescription, item.description || "")
2598:                     : ""}
2599:                     ${renderDynamicLinks(item.links || [])}
2600:                 </article>
2601:             `).join("")}
2602:         </div>
2603:         ` : ""}
2604:         ${renderDynamicLinks(links)}
2605:     </div>
2606: </section>
2607: `;
2608: }).join("");
2609: }

```

subsections (script.js, lines 2579-2581)

subsections part of the public website runtime. In this file, it appears around lines 2579-2581 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2580: return `.

Important local values include subsections at line 2579. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2579:     const subsections = (section.subsections || []).filter((item) => item.title || item.description ||
item.richDescription?.blocks?.length || (item.links || []).length);
2580:     return `
2581:     <section class="section dynamic-section" id="${escapeHtml(section.id || "")}"${style}>

```

customSectionBlocks (script.js, lines 2611-2622)

customSectionBlocks part of the public website runtime. In this file, it appears around lines 2611-2622 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, detailBlock, evidenceList, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2612: return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-wide", `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2611: function customSectionBlocks(project) {
2612:     return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
2613:     ${section.description ? `<p class="evidence-empty">${section.description}</p>` : ""}
2614:     ${evidenceList(section.items || [], (item) => `
2615:         <li>
2616:             ${item.url ? resourceLink(item, item.title) : `<strong>${item.title}</strong>`}
2617:             <span>${item.type || "Section item"} &middot; ${item.status || "tracked">
2618:             ${item.description ? `<p>${item.description}</p>` : ""}
2619:         </li>
2620:         ` , "No content has been added yet.")}
2621:     `)).join("");
2622: }

```

itemUrl (script.js, lines 2623-2625)

itemUrl part of the public website runtime. In this file, it appears around lines 2623-2625 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2624: return item.url || item.artifact || item.href || item.file || item.path || item.src || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2623: function itemUrl(item = {}) {
2624:   return item.url || item.artifact || item.href || item.file || item.path || item.src || "";
2625: }
```

itemLabel (script.js, lines 2627-2630)

itemLabel part of the public website runtime. In this file, it appears around lines 2627-2630 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references itemUrl, and fileNameFromUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2629: return item.title || item.label || item.name || item.caption || fileNameFromUrl(url) || fallback;

Important local values include url at line 2628. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2627: function itemLabel(item = {}, fallback = "Download file") {
2628:   const url = itemUrl(item);
2629:   return item.title || item.label || item.name || item.caption || fileNameFromUrl(url) || fallback;
2630: }
```

normalizeDownloadAsset (script.js, lines 2632-2643)

normalizeDownloadAsset validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2632-2643 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references itemUrl, and itemLabel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2634: if (!url || item.status === "planned") return null;; line 2636: return {.

Important local values include url at line 2633. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2632: function normalizeDownloadAsset(item = {}, fallbackType = "File") {
2633:   const url = itemUrl(item);
2634:   if (!url || item.status === "planned") return null;
2635:
2636:   return {
2637:     title: itemLabel(item),
2638:     url,
2639:     status: item.status || "uploaded",
2640:     type: item.type || item.kind || item.meta || fallbackType,
2641:     description: item.description || item.caption || item.summary || ""
2642:   };
2643: }
```

collectRichDownloads (script.js, lines 2645-2659)

collectRichDownloads part of the public website runtime. In this file, it appears around lines 2645-2659 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references forEach, push, cleanRichImageTitle, and fileNameFromUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2647: if (!block?.url) return;; line 2658: return output;

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2645: function collectRichDownloads(rich, fallbackType = "File", output = []) {
2646:   (rich?.blocks || []).forEach((block) => {
2647:     if (!block?.url) return;
2648:
2649:     output.push({
2650:       title: cleanRichImageTitle(block) || block.title || block.caption ||
fileNameFromUrl(block.url),
2651:       url: block.url,
2652:       status: "uploaded",
2653:       type: block.type === "image" ? "Image" : fallbackType,
2654:       description: block.caption || ""
2655:     });
2656:   });
2657:   return output;
2658: }
2659: }
```

collectDownloadsFromValue (script.js, lines 2661-2689)

collectDownloadsFromValue part of the public website runtime. In this file, it appears around lines 2661-2689 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, forEach, normalizeDownloadAsset, push, and collectRichDownloads. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2662: if (!value) return output;; line 2666: return output;; line 2669: if (typeof value !== "object") return output;; line 2688: return output;

Important local values include asset at line 2671. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2661: function collectDownloadsFromValue(value, fallbackType = "File", output = []) {
2662:   if (!value) return output;
2663:
2664:   if (Array.isArray(value)) {
2665:     value.forEach((item) => collectDownloadsFromValue(item, fallbackType, output));
2666:     return output;
2667:   }
2668:
2669:   if (typeof value !== "object") return output;
2670:
2671:   const asset = normalizeDownloadAsset(value, fallbackType);
2672:   if (asset) output.push(asset);
2673:
2674:   collectRichDownloads(value.rich, fallbackType, output);
2675:   collectRichDownloads(value.summaryRich, fallbackType, output);
2676:
2677:   [
2678:     "files",
2679:     "assets",
2680:     "items",
2681:     "children",
2682:     "links",
2683:     "results",
2684:     "references",
2685:     "mathAnalysis"
2686:   ].forEach((key) => collectDownloadsFromValue(value[key], fallbackType, output));
2687:
2688:   return output;
2689: }
```

uniqueDownloads (script.js, lines 2691-2700)

uniqueDownloads part of the public website runtime. In this file, it appears around lines 2691-2700 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, filter, has, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2694: return items.filter((item) => {; line 2696: if (!item.url || seen.has(key)) return false;; line 2698: return true;;

Important local values include seen at line 2692; key at line 2695. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2691: function uniqueDownloads(items = []) {
2692:   const seen = new Set();
2693:
2694:   return items.filter((item) => {
2695:     const key = `${item.url}:${item.title}`;
2696:     if (!item.url || seen.has(key)) return false;
2697:     seen.add(key);
2698:     return true;
2699:   });
2700: }
```

renderDownloadBlock (script.js, lines 2702-2717)

renderDownloadBlock renders ui, markup, or display output. In this file, it appears around lines 2702-2717 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, detailBlock, escapeHtml, map, resourceLink, renderMultilineInlineText, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2704: if (!downloads.length) return "";; line 2706: return detailBlock(escapeHtml(title), "evidence-block evidence-wide download-evidence-block", `

Important local values include downloads at line 2703. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2702: function renderDownloadBlock(title, items = []) {
2703:   const downloads = uniqueDownloads(items);
2704:   if (!downloads.length) return "";
2705:
2706:   return detailBlock(escapeHtml(title), "evidence-block evidence-wide download-evidence-block", `
2707:     <ul class="download-list">
2708:       ${downloads.map((item) => `
2709:         <li>
2710:           ${resourceLink(item, item.title)}
2711:           <span>${escapeHtml(item.type || "File")} &middot; ${escapeHtml(item.status ||
"uploaded")}</span>
2712:           ${item.description ? `<p>${renderMultilineInlineText(item.description)}</p>` : ""}
2713:         </li>
2714:       `).join("")}
2715:     </ul>
2716:   `);
2717: }
```

renderCollectedDownloadSections (script.js, lines 2719-2760)

renderCollectedDownloadSections renders ui, markup, or display output. In this file, it appears around lines 2719-2760 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, collectDownloadsFromValue, push, renderDownloadBlock, addBlock, forEach, displayTitle, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no

local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2759: return blocks.join("");.

Important local values include blocks at line 2720; addBlock at line 2722; downloads at line 2723. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2719: function renderCollectedDownloadSections(project) {
2720:   const blocks = [];
2721:
2722:   const addBlock = (title, source, fallbackType = "File") => {
2723:     const downloads = uniqueDownloads(collectDownloadsFromValue(source, fallbackType, []));
2724:     if (downloads.length) blocks.push(renderDownloadBlock(title, downloads));
2725:   };
2726:
2727:   addBlock("Overview downloads", [
2728:     { summaryRich: project.summaryRich },
2729:     project.design?.brief?.files,
2730:     project.design?.brief?.summaryRich
2731:   ], "Overview file");
2732:
2733:   addBlock("Design downloads", [
2734:     project.design?.documentation?.files,
2735:     project.design?.documentation?.references,
2736:     project.design?.documentation?.mathAnalysis
2737:   ], "Design file");
2738:
2739:   addBlock("Simulation downloads", [
2740:     project.design?.simulation?.files,
2741:     project.design?.simulation?.results
2742:   ], "Simulation file");
2743:
2744:   addBlock("Project documents", project.documents, "Document");
2745:   addBlock("Test artifacts", project.tests, "Test artifact");
2746:   addBlock("PCB artifacts", project.pcb, "PCB artifact");
2747:   addBlock("Images and media", project.media, "Image");
2748:
2749:   (project.sections || []).forEach((section) => {
2750:     addBlock(`${displayTitle(section.title, "Custom section")} downloads`, section.items ||
section.files || [], section.title || "Section file");
2751:   });
2752:
2753:   (project.portfolioView?.sections || []).
2754:     .filter((section) => section.id !== "brief")
2755:     .forEach((section) => {
2756:       addBlock(`${displayTitle(section.title, "Section")} downloads`, section.items || [],
section.title || "Section file");
2757:     });
2758:
2759:   return blocks.join("");
2760: }
```

renderParsedBriefBlock (script.js, lines 2761-2771)

renderParsedBriefBlock renders ui, markup, or display output. In this file, it appears around lines 2761-2771 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2764: if (!briefItem.rich?.blocks?.length && !briefText) return "";; line 2765: return `.

Important local values include briefItem at line 2762; briefText at line 2763. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2761: function renderParsedBriefBlock(section, fallbackSummary = "") {
2762:   const briefItem = section?.items?.[0] || {};
2763:   const briefText = briefItem.description || fallbackSummary || "";
2764:   if (!briefItem.rich?.blocks?.length && !briefText) return "";
2765:   return `
2766:     <details class="project-brief-default parsed-summary" open>
2767:       <summary>Overview</summary>
2768:       ${renderRichContent(briefItem.rich, briefText)}
2769:     </details>
2770: `;
2771: }
```

pathToString (script.js, lines 2773-2775)

pathToString part of the public website runtime. In this file, it appears around lines 2773-2775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2774: return path.join(".");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2773: function pathToString(path = []) {  
2774:   return path.join(".");  
2775: }
```

pathFromString (script.js, lines 2777-2780)

pathFromString part of the public website runtime. In this file, it appears around lines 2777-2780 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, filter, and isInteger. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2778: if (!value) return []; line 2779: return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2777: function pathFromString(value = "") {  
2778:   if (!value) return [];  
2779:   return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));  
2780: }
```

sectionRouteState (script.js, lines 2782-2789)

sectionRouteState part of the public website runtime. In this file, it appears around lines 2782-2789 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2783: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2782: function sectionRouteState(projectId, sectionIndex, resourcePath = "") {  
2783:   return {  
2784:     appWindow: sectionRouteKey,  
2785:     projectId: String(projectId || ""),  
2786:     resourcePath: String(resourcePath || ""),  
2787:     sectionIndex: String(sectionIndex ?? "")  
2788:   };  
2789: }
```

canUseSectionHistory (script.js, lines 2791-2793)

canUseSectionHistory part of the public website runtime. In this file, it appears around lines 2791-2793 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2792: return Boolean(window.history && typeof window.history.pushState === "function" && typeof window.history.replaceState === "function");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2791: function canUseSectionHistory() {
2792:   return Boolean(window.history && typeof window.history.pushState === "function" && typeof
window.history.replaceState === "function");
2793: }
```

sectionBaseUrl (script.js, lines 2795-2799)

sectionBaseUrl part of the public website runtime. In this file, it appears around lines 2795-2799 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, values, forEach, and delete. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2798: return url;.

Important local values include url at line 2796. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2795: function sectionBaseUrl() {
2796:   const url = new URL(window.location.href);
2797:   Object.values(sectionRouteParams).forEach((param) => url.searchParams.delete(param));
2798:   return url;
2799: }
```

sectionRouteUrl (script.js, lines 2801-2809)

sectionRouteUrl part of the public website runtime. In this file, it appears around lines 2801-2809 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionBaseUrl, set, and delete. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2808: return url;.

Important local values include url at line 2802. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2801: function sectionRouteUrl(projectId, sectionIndex, resourcePath = "") {
2802:   const url = sectionBaseUrl();
2803:   url.searchParams.set(sectionRouteParams.project, String(projectId || ""));
2804:   url.searchParams.set(sectionRouteParams.section, String(sectionIndex ?? ""));
2805:   if (resourcePath) url.searchParams.set(sectionRouteParams.path, String(resourcePath));
2806:   else url.searchParams.delete(sectionRouteParams.path);
2807:   url.hash = "projects";
2808:   return url;
2809: }
```

sectionRouteFromState (script.js, lines 2811-2819)

sectionRouteFromState part of the public website runtime. In this file, it appears around lines 2811-2819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2812: if (!state || state.appWindow !== sectionRouteKey) return null;; line 2813: if (!state.projectId || state.sectionIndex === "") return null;; line 2814: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2811: function sectionRouteFromState(state) {
2812:   if (!state || state.appWindow !== sectionRouteKey) return null;
2813:   if (!state.projectId || state.sectionIndex === "") return null;
2814:   return {
2815:     projectId: String(state.projectId),
2816:     resourcePath: String(state.resourcePath || ""),
2817:     sectionIndex: String(state.sectionIndex)
2818:   };
2819: }
```

sectionRouteFromLocation (script.js, lines 2821-2831)

sectionRouteFromLocation part of the public website runtime. In this file, it appears around lines 2821-2831 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URLSearchParams, and get. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2825: if (!projectId || sectionIndex === null) return null;; line 2826: return {.

Important local values include params at line 2822; projectId at line 2823; sectionIndex at line 2824. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2821: function sectionRouteFromLocation() {
2822:   const params = new URLSearchParams(window.location.search);
2823:   const projectId = params.get(sectionRouteParams.project);
2824:   const sectionIndex = params.get(sectionRouteParams.section);
2825:   if (!projectId || sectionIndex === null) return null;
2826:   return {
2827:     projectId,
2828:     resourcePath: params.get(sectionRouteParams.path) || "",
2829:     sectionIndex
2830:   };
2831: }
```

sameSectionRoute (script.js, lines 2833-2838)

sameSectionRoute part of the public website runtime. In this file, it appears around lines 2833-2838 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2834: return Boolean(left && right &&.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2833: function sameSectionRoute(left, right) {
2834:   return Boolean(left && right &&
2835:     left.projectId === right.projectId &&
2836:     String(left.sectionIndex) === String(right.sectionIndex) &&
2837:     String(left.resourcePath || "") === String(right.resourcePath || ""));
2838: }
```

syncSectionRouteToHistory (script.js, lines 2840-2851)

syncSectionRouteToHistory keeps two places aligned, such as local data and target files. In this file, it appears around lines 2840-2851 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `canUseSectionHistory`, `sectionRouteState`, `sectionRouteUrl`, `sectionRouteFromState`, `sectionRouteFromLocation`, and `sameSectionRoute`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2841: `if (isApplyingSectionRoute || mode === "none" || !canUseSectionHistory()) return;`

Important local values include `route` at line 2842; `url` at line 2843; `currentRoute` at line 2844; `method` at line 2845. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2840: function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push") {
2841:   if (isApplyingSectionRoute || mode === "none" || !canUseSectionHistory()) return;
2842:   const route = sectionRouteState(projectId, sectionIndex, resourcePath);
2843:   const url = sectionRouteUrl(projectId, sectionIndex, resourcePath);
2844:   const currentRoute = sectionRouteFromState(history.state) || sectionRouteFromLocation();
2845:   const method = mode === "replace" || sameSectionRoute(currentRoute, route) ? "replaceState" :
"pushState";
2846:   try {
2847:     history[method](route, "", url);
2848:   } catch {
2849:     // Preview iframes and restricted browser contexts may reject history writes.
2850:   }
2851: }
```

clearSectionRouteInHistory (script.js, lines 2853-2860)

`clearSectionRouteInHistory` part of the public website runtime. In this file, it appears around lines 2853-2860 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `canUseSectionHistory`, `replaceState`, and `sectionBaseUrl`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2854: `if (isApplyingSectionRoute || !canUseSectionHistory()) return;`

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2853: function clearSectionRouteInHistory() {
2854:   if (isApplyingSectionRoute || !canUseSectionHistory()) return;
2855:   try {
2856:     history.replaceState({ appWindow: "portfolio-base" }, "", sectionBaseUrl());
2857:   } catch {
2858:     // The visible dialog is already closed; URL cleanup is best effort.
2859:   }
2860: }
```

initializeSectionHistoryState (script.js, lines 2862-2871)

`initializeSectionHistoryState` part of the public website runtime. In this file, it appears around lines 2862-2871 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `canUseSectionHistory`, `sectionRouteFromLocation`, `sectionRouteFromState`, `replaceState`, `sectionRouteState`, and `sectionRouteUrl`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2863: `if (!canUseSectionHistory()) return;`

Important local values include `route` at line 2864. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2862: function initializeSectionHistoryState() {
2863:   if (!canUseSectionHistory()) return;
2864:   const route = sectionRouteFromLocation() || sectionRouteFromState(history.state);
2865:   try {
2866:     if (route) history.replaceState(sectionRouteState(route.projectId, route.sectionIndex,
route.resourcePath), "", sectionRouteUrl(route.projectId, route.sectionIndex, route.resourcePath));
2867:     else if (!history.state?.appWindow) history.replaceState({ appWindow: "portfolio-base" }, "",
window.location.href);

```

```

2868:   } catch {
2869:     // Browser history state is progressive enhancement for the portfolio windows.
2870:   }
2871: }

```

nodeAtPath (script.js, lines 2873-2882)

nodeAtPath part of the public website runtime. In this file, it appears around lines 2873-2882 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2878: if (!node) return null;; line 2881: return node;.

Important local values include node at line 2874; children at line 2875. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2873: function nodeAtPath(section, path = []) {
2874:   let node = section;
2875:   let children = section?.items || [];
2876:   for (const index of path) {
2877:     node = children[index];
2878:     if (!node) return null;
2879:     children = node.children || [];
2880:   }
2881:   return node;
2882: }

```

nodeChildren (script.js, lines 2884-2886)

nodeChildren part of the public website runtime. In this file, it appears around lines 2884-2886 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2885: return node?.items || node?.children || [];

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2884: function nodeChildren(node) {
2885:   return node?.items || node?.children || [];
2886: }

```

richHasRenderableContent (script.js, lines 2888-2897)

richHasRenderableContent part of the public website runtime. In this file, it appears around lines 2888-2897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2889: return Boolean(rich?.blocks?.some((block) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2888: function richHasRenderableContent(rich) {
2889:   return Boolean(rich?.blocks?.some((block) =>
2890:     block.type === "image" && block.url ||
2891:     block.type === "formula" && block.formula ||
2892:     block.type === "code" && block.code ||
2893:     block.text ||
2894:     block.title ||
2895:     block.caption

```

```
2896:   });
2897: }
```

nodeHasRenderableContent (script.js, lines 2899-2910)

nodeHasRenderableContent part of the public website runtime. In this file, it appears around lines 2899-2910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasRenderableContent, nodeChildren, some, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 2900: if (!node) return false;; line 2901: if (node.kind === "summary") return Boolean(node.description || richHasRenderableContent(node.rich));; line 2904: return Boolean(node.description || node.url || richHasRenderableContent(node.rich) || children.some(nodeHasRenderableContent));; line 2907: return Boolean(node.title || node.description || node.url || richHasRenderableContent(node.rich) || children.some(nodeHasRenderableContent));. There are 1 additional return statements in the function body.

Important local values include children at line 2902. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2899: function nodeHasRenderableContent(node) {
2900:   if (!node) return false;
2901:   if (node.kind === "summary") return Boolean(node.description || richHasRenderableContent(node.rich));
2902:   const children = nodeChildren(node);
2903:   if (node.kind === "subsection") {
2904:     return Boolean(node.description || node.url || richHasRenderableContent(node.rich) ||
children.some(nodeHasRenderableContent));
2905:   }
2906:   if (["tool", "language"].includes(node.kind)) {
2907:     return Boolean(node.title || node.description || node.url || richHasRenderableContent(node.rich) ||
children.some(nodeHasRenderableContent));
2908:   }
2909:   return Boolean(node.description || node.url || richHasRenderableContent(node.rich) ||
children.some(nodeHasRenderableContent));
2910: }
```

sectionHasRenderableContent (script.js, lines 2912-2914)

sectionHasRenderableContent part of the public website runtime. In this file, it appears around lines 2912-2914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasRenderableContent, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2913: return Boolean(section?.description || richHasRenderableContent(section?.rich) || (section?.items || []).some(nodeHasRenderableContent));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2912: function sectionHasRenderableContent(section) {
2913:   return Boolean(section?.description || richHasRenderableContent(section?.rich) || (section?.items ||
[]).some(nodeHasRenderableContent));
2914: }
```

responsiveChildren (script.js, lines 2916-2918)

responsiveChildren part of the public website runtime. In this file, it appears around lines 2916-2918 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2917: return node?.items || node?.children || [];

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2916: function responsiveChildren(node) {  
2917:   return node?.items || node?.children || [];  
2918: }
```

richContainsMedia (script.js, lines 2920-2922)

richContainsMedia part of the public website runtime. In this file, it appears around lines 2920-2922 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2921: return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2920: function richContainsMedia(rich) {  
2921:   return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));  
2922: }
```

responsiveNodeHasMedia (script.js, lines 2924-2930)

responsiveNodeHasMedia part of the public website runtime. In this file, it appears around lines 2924-2930 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richContainsMedia, responsiveChildren, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2925: if (!node) return false;; line 2926: if (node.kind === "image" || richContainsMedia(node.rich)) return true;; line 2928: if (/^(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|\$)/i.test(url)) return true;; line 2929: return responsiveChildren(node).some(responsiveNodeHasMedia);.

Important local values include url at line 2927. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2924: function responsiveNodeHasMedia(node) {  
2925:   if (!node) return false;  
2926:   if (node.kind === "image" || richContainsMedia(node.rich)) return true;  
2927:   const url = String(node.url || "");  
2928:   if (/^(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|$)/i.test(url)) return true;  
2929:   return responsiveChildren(node).some(responsiveNodeHasMedia);  
2930: }
```

responsiveNodeCount (script.js, lines 2932-2935)

responsiveNodeCount part of the public website runtime. In this file, it appears around lines 2932-2935 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references nodeHasRenderableContent, responsiveChildren, and reduce. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2933: if (!node || !nodeHasRenderableContent(node)) return 0;; line 2934: return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2932: function responsiveNodeCount(node) {  
2933:   if (!node || !nodeHasRenderableContent(node)) return 0;
```

```
2934:   return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);
2935: }
```

responsiveNodeDepth (script.js, lines 2937-2942)

responsiveNodeDepth part of the public website runtime. In this file, it appears around lines 2937-2942 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references nodeHasRenderableContent, responsiveChildren, filter, max, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2938: if (!node || !nodeHasRenderableContent(node)) return 0;; line 2940: if (!children.length) return 1;; line 2941: return 1 + Math.max(...children.map(responsiveNodeDepth));.

Important local values include children at line 2939. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2937: function responsiveNodeDepth(node) {
2938:   if (!node || !nodeHasRenderableContent(node)) return 0;
2939:   const children = responsiveChildren(node).filter(nodeHasRenderableContent);
2940:   if (!children.length) return 1;
2941:   return 1 + Math.max(...children.map(responsiveNodeDepth));
2942: }
```

inferResponsiveProfile (script.js, lines 2944-2976)

inferResponsiveProfile part of the public website runtime. In this file, it appears around lines 2944-2976 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, sectionHasRenderableContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, and hasPublicTemplate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2961: return {.

Important local values include sections at line 2945; visibleSections at line 2946; itemCount at line 2947; maxDepth at line 2951; hasMedia at line 2955; density at line 2956. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2944: function inferResponsiveProfile(project) {
2945:   const sections = project?.portfolioView?.sections || [];
2946:   const visibleSections = sections.filter((section) => section?.id !== "brief" &&
sectionHasRenderableContent(section));
2947:   const itemCount = visibleSections.reduce(
2948:     (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
2949:     0
2950:   );
2951:   const maxDepth = visibleSections.reduce(
2952:     (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
2953:     1
2954:   );
2955:   const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
2956:   const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
2957:     ? "dense"
2958:     : itemCount <= 6 && visibleSections.length <= 2
2959:     ? "simple"
2960:     : "balanced";
2961:   return {
2962:     version: 1,
2963:     breakpoints: {
2964:       mobile: 640,
2965:       tablet: 900,
2966:       desktop: 1180
2967:     },
2968:     cardLayout: hasPublicTemplate(project) ? "single" : "split",
2969:     contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
2970:     density,
2971:     mediaMax: hasMedia ? "860px" : "720px",
2972:     sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
2973:     touchTarget: "44px",
2974:     windowMode: "full-screen"
2975:   };
2976: }
```

```
2976: }
```

projectResponsiveProfile (script.js, lines 2978-2982)

projectResponsiveProfile part of the public website runtime. In this file, it appears around lines 2978-2982 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references inferResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2980: if (savedProfile?.version) return { ...inferResponsiveProfile(project), ...savedProfile }; line 2981: return inferResponsiveProfile(project);.

Important local values include savedProfile at line 2979. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2978: function projectResponsiveProfile(project) {
2979:   const savedProfile = project?.portfolioView?.responsive;
2980:   if (savedProfile?.version) return { ...inferResponsiveProfile(project), ...savedProfile };
2981:   return inferResponsiveProfile(project);
2982: }
```

nodeSummary (script.js, lines 2984-2992)

nodeSummary part of the public website runtime. In this file, it appears around lines 2984-2992 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2985: if (!rich?.blocks?.length && !text) return ""; line 2986: return `

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2984: function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") {
2985:   if (!rich?.blocks?.length && !text) return "";
2986:   return `
2987:     <details class="parsed-summary" open>
2988:       <summary>${escapeHtml(title)}</summary>
2989:       ${renderRichContent(rich, text || "")}
2990:     </details>
2991:   `;
2992: }
```

nodeIsOverview (script.js, lines 2994-2997)

nodeIsOverview part of the public website runtime. In this file, it appears around lines 2994-2997 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, displayTitle, and endsWith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2996: return item.kind === "summary" || title === "overview" || title.endsWith(" overview");.

Important local values include title at line 2995. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2994: function nodeIsOverview(item = {}) {
2995:   const title = normalize(displayTitle(item.title || item.label || ""));
2996:   return item.kind === "summary" || title === "overview" || title.endsWith(" overview");
2997: }
```

nodeOverviewDetails (script.js, lines 2999-3017)

nodeOverviewDetails part of the public website runtime. In this file, it appears around lines 2999-3017 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, push, renderRichContent, forEach, renderInlineSectionFiles, nodeChildren, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3010: if (!blocks.length) return ""; line 3011: return `

Important local values include overviewItems at line 3000; blocks at line 3001; itemFiles at line 3007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2999: function nodeOverviewDetails(node, children = []) {
3000:   const overviewItems = children.filter(nodeIsOverview);
3001:   const blocks = [];
3002:   if (node?.rich?.blocks?.length || node?.description) {
3003:     blocks.push(renderRichContent(node.rich, node.description || ""));
3004:   }
3005:   overviewItems.forEach((item) => {
3006:     if (item.rich?.blocks?.length || item.description) blocks.push(renderRichContent(item.rich,
item.description || ""));
3007:     const itemFiles = renderInlineSectionFiles(nodeChildren(item));
3008:     if (itemFiles) blocks.push(itemFiles);
3009:   });
3010:   if (!blocks.length) return "";
3011:   return `
3012:     <details class="parsed-summary section-overview-default" open>
3013:       <summary>Overview</summary>
3014:       ${blocks.join("")}
3015:     </details>
3016: `;
3017: }
```

parsedNodeCard (script.js, lines 3019-3032)

parsedNodeCard part of the public website runtime. In this file, it appears around lines 3019-3032 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references displayTitle, pathToString, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3021: return `

Important local values include title at line 3020. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3019: function parsedNodeCard(node, projectId, sectionIndex, path) {
3020:   const title = displayTitle(node.title);
3021:   return `
3022:     <button
3023:       class="section-open-card subsection-open-card"
3024:       type="button"
3025:       data-section-project="${projectId}"
3026:       data-section-index="${sectionIndex}"
3027:       data-resource-path="${pathToString(path)}"
3028:     >
3029:       <span>${escapeHtml(title)}</span>
3030:     </button>
3031: `;
3032: }
```

parsedChildCards (script.js, lines 3034-3042)

parsedChildCards part of the public website runtime. In this file, it appears around lines 3034-3042 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, nodeHasRenderableContent, nodeIsOverview, map, parsedNodeCard, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3036: if (!visibleChildren.length) return "";; line 3037: return `.

Important local values include visibleChildren at line 3035. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3034: function parsedChildCards(children, projectId, sectionIndex, basePath = []) {
3035:   const visibleChildren = children.filter((child) => nodeHasRenderableContent(child) && !child.url &&
!nodeIsOverview(child));
3036:   if (!visibleChildren.length) return "";
3037:   return `
3038:     <div class="subsection-grid section-content-grid">
3039:       ${children.map((child, index) => nodeHasRenderableContent(child) && !child.url &&
!nodeIsOverview(child) ? parsedNodeCard(child, projectId, sectionIndex, [...basePath, index]) : "").join("")}
3040:     </div>
3041:   `;
3042: }
```

parsedNodeContent (script.js, lines 3044-3058)

parsedNodeContent part of the public website runtime. In this file, it appears around lines 3044-3058 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderInlineSectionFiles, nodeChildren, filter, nodeIsOverview, nodeOverviewDetails, and parsedChildCards. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3046: if (!isContainer && node.url) return renderInlineSectionFiles([node]);; line 3051: return `.

Important local values include isContainer at line 3045; children at line 3048; contentChildren at line 3049. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3044: function parsedNodeContent(node, projectId, sectionIndex, path = []) {
3045:   const isContainer = !node.kind || node.kind === "subsection";
3046:   if (!isContainer && node.url) return renderInlineSectionFiles([node]);
3047:
3048:   const children = nodeChildren(node);
3049:   const contentChildren = children.filter((child) => !nodeIsOverview(child));
3050:
3051:   return `
3052:   <article class="parsed-window-panel section-directory">
3053:     ${nodeOverviewDetails(node, children)}
3054:     ${parsedChildCards(children, projectId, sectionIndex, path)}
3055:     ${renderInlineSectionFiles(contentChildren)}
3056:   </article>
3057:   `;
3058: }
```

parsedSectionContent (script.js, lines 3060-3064)

parsedSectionContent part of the public website runtime. In this file, it appears around lines 3060-3064 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references nodeAtPath, and parsedNodeContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3062: if (!node) return `<p class="evidence-empty">This section could not be found.</p>;; line 3063: return parsedNodeContent(node, projectId, sectionIndex, path);.

Important local values include node at line 3061. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3060: function parsedSectionContent(section, projectId, sectionIndex, path = []) {
3061:   const node = path.length ? nodeAtPath(section, path) : section;
3062:   if (!node) return `<p class="evidence-empty">This section could not be found.</p>`;
```



```

3063:   return parsedNodeContent(node, projectId, sectionIndex, path);
3064: }

```

parsedSection (script.js, lines 3066-3080)

`parsedSection` part of the public website runtime. In this file, it appears around lines 3066-3080 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `filter`, `some`, `escapeHtml`, and `displayTitle`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3070: `return ``.

Important local values include `visibleItems` at line 3067; `hasImages` at line 3068. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3066: function parsedSection(section, index, project) {
3067:   const visibleItems = (section.items || []).filter(nodeHasRenderableContent);
3068:   const hasImages = visibleItems.some((item) => item.kind === "image");
3069:
3070:   return `
3071:     <button
3072:       class="section-open-card evidence-block ${hasImages ? "evidence-wide" : ""}"
3073:       type="button"
3074:       data-section-project="${project.id}"
3075:       data-section-index="${index}"
3076:     >
3077:       <span>${escapeHtml(displayTitle(section.title, "Section"))}</span>
3078:     </button>
3079:   `;
3080: }

```

projectCard (script.js, lines 3082-3164)

`projectCard` part of the public website runtime. In this file, it appears around lines 3082-3164 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `find`, `filter`, `sectionHasRenderableContent`, `projectTemplateClass`, `projectResponsiveClass`, `projectTemplateStyle`, `renderParsedBriefBlock`, `map`, `parsedSection`, `join`, and 7 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3090: `return ``; line 3106: `return ``.

Important local values include `category` at line 3084; `accent` at line 3085; `sections` at line 3086; `briefSection` at line 3087; `otherSections` at line 3088; `category` at line 3103; `accent` at line 3104. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3082: function projectCard(project) {
3083:   if (project.portfolioView) {
3084:     const category = categories.find((item) => item.id === project.category) || {};
3085:     const accent = category.accent || "#117c7a";
3086:     const sections = project.portfolioView.sections || [];
3087:     const briefSection = sections.find((section) => section.id === "brief");
3088:     const otherSections = sections.filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
3089:
3090:     return `
3091:       <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
3092:         <div class="project-body">
3093:           <h3>${project.portfolioView.title || project.title}</h3>
3094:           ${renderParsedBriefBlock(briefSection, project.summary)}
3095:           <div class="evidence-grid" aria-label="${project.title} parsed project content">
3096:             ${otherSections.map((section, index) => parsedSection(section, index, project)).join("")}
3097:           </div>
3098:         </div>
3099:       </article>
3100:     `;
3101:   }
3102:
3103:   const category = categories.find((item) => item.id === project.category) || {};
3104:   const accent = category.accent || "#117c7a";

```

```

3105:
3106:   return `
3107:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
3108:       <div class="project-body">
3109:         <h3>${project.title}</h3>
3110:         <p class="rich-paragraph">${renderMultilineInlineText(project.summary)}</p>
3111:
3112:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
drawer", `
3113:           <ul class="highlight-list">
3114:             ${project.highlights.map((item) => `<li>${item}</li>`).join("")}
3115:           </ul>
3116:         `) : ""}
3117:
3118:         <div class="evidence-grid" aria-label="${project.title} evidence blocks">
3119:           ${detailBlock("Documents", "evidence-block", evidenceList(project.documents, (item) => `
3120:             <li>
3121:               ${resourceLink(item, item.title)}
3122:               <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
3123:             </li>
3124:           `, "No document artifact has been added yet.))}
3125:
3126:           ${detailBlock("Tests and results", "evidence-block", evidenceList(project.tests, (item) => `
3127:             <li>
3128:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3129:               <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
3130:               ${item.result ? `<p>${item.result}</p>` : ""}
3131:             </li>
3132:           `, "No test artifact has been added yet.))}
3133:
3134:           ${detailBlock("PCBs built", "evidence-block", evidenceList(project.pcb, (item) => `
3135:             <li>
3136:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3137:               <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
3138:             </li>
3139:           `, "No PCB build has been added yet.))}
3140:
3141:           ${detailBlock("Images", "evidence-block evidence-wide", mediaGrid(project.media))}
3142:         ${customSectionBlocks(project)}
3143:       </div>
3144:
3145:       ${detailBlock("Tools and implementation files", "project-drawer", `
3146:         <div class="project-tooling">
3147:           <div>
3148:             <h4>Tools Used</h4>
3149:             <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
3150:           </div>
3151:           <div>
3152:             <h4>Languages</h4>
3153:             <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
3154:           </div>
3155:         </div>
3156:       `)}
3157:
3158:       <div class="resource-list">
3159:         ${project.links || []}.map((item) => resourceLink(item)).join("")
3160:       </div>
3161:     </div>
3162:   </article>
3163: `;
3164: }

```

categorySection (script.js, lines 3166-3181)

categorySection part of the public website runtime. In this file, it appears around lines 3166-3181 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3167: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3166: function categorySection(category, visibleProjects) {
3167:   return `
3168:     <section class="category-section ${visibleProjects.length ? "" : "empty-category-section"}" aria-
labelledby="${escapeHtml(category.id)}-title">
3169:       <div class="category-heading">
3170:         <div>
3171:           <h3 id="${escapeHtml(category.id)}-title">${escapeHtml(category.label)}</h3>
3172:         </div>

```

```

3173:         ${visibleProjects.length ? `<span>${visibleProjects.length} project${visibleProjects.length === 1
? "" : "s"}</span>` : ""}
3174:     </div>
3175:     ${visibleProjects.length && category.description ? `<p class="category-
description">${escapeHtml(category.description)}</p>` : ""}
3176:     <div class="category-projects">
3177:         ${visibleProjects.map(projectCard).join("")}
3178:     </div>
3179: </section>
3180: `;
3181: }

```

escapeRegExp (script.js, lines 3183-3185)

escapeRegExp part of the public website runtime. In this file, it appears around lines 3183-3185 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3184: return String(value).replace(/[/.*+?^\$(){}|\[\]\\"/g, "\\\$&");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3183: function escapeRegExp(value = "") {
3184:   return String(value).replace(/[/.*+?^$(){}|\[\]\\"/g, "\\$&");
3185: }

```

highlightQueryHtml (script.js, lines 3187-3193)

highlightQueryHtml part of the public website runtime. In this file, it appears around lines 3187-3193 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, trim, escapeRegExp, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3190: if (!cleanQuery) return escaped;; line 3192: return escaped.replace(pattern, '<mark class="search-result-mark">\$1</mark>');

Important local values include escaped at line 3188; cleanQuery at line 3189; pattern at line 3191. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3187: function highlightQueryHtml(value = "", query = "") {
3188:   const escaped = escapeHtml(value);
3189:   const cleanQuery = String(query || "").trim();
3190:   if (!cleanQuery) return escaped;
3191:   const pattern = new RegExp(`${escapeRegExp(escapeHtml(cleanQuery))}`, "ig");
3192:   return escaped.replace(pattern, '<mark class="search-result-mark">$1</mark>');
3193: }

```

clearSearchHighlights (script.js, lines 3195-3197)

clearSearchHighlights part of the public website runtime. In this file, it appears around lines 3195-3197 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3195: function clearSearchHighlights() {
3196:   window.CSS?.highlights?.delete?.(searchHighlightName);

```

```
3197: }
```

textNodeSearchTargets (script.js, lines 3199-3204)

textNodeSearchTargets part of the public website runtime. In this file, it appears around lines 3199-3204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3200: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3199: function textNodeSearchTargets() {
3200:   return [
3201:     document.querySelector("main"),
3202:     document.querySelector("#section-view-dialog[open] .section-view-content")
3203:   ].filter(Boolean);
3204: }
```

applySearchHighlights (script.js, lines 3206-3238)

applySearchHighlights part of the public website runtime. In this file, it appears around lines 3206-3238 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clearSearchHighlights, trim, normalize, textNodeSearchTargets, forEach, createTreeWalker, acceptNode, includes, closest, nextNode, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 3209: if (!cleanQuery || !window.CSS?.highlights || typeof window.Highlight !== "function") return 0;; line 3216: if (!node.textContent || !normalize(node.textContent).includes(needle)) return NodeFilter.FILTER_REJECT;; line 3217: if (node.parentElement?.closest("script, style, input, textarea, select, button, .search-engine, .site-header")) return NodeFilter.FILTER_REJECT;; line 3218: return NodeFilter.FILTER_ACCEPT;. There are 1 additional return statements in the function body.

Important local values include cleanQuery at line 3208; ranges at line 3210; needle at line 3211; walker at line 3214; node at line 3221; haystack at line 3223; index at line 3224; range at line 3226. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3206: function applySearchHighlights(query) {
3207:   clearSearchHighlights();
3208:   const cleanQuery = String(query || "").trim();
3209:   if (!cleanQuery || !window.CSS?.highlights || typeof window.Highlight !== "function") return 0;
3210:   const ranges = [];
3211:   const needle = normalize(cleanQuery);
3212:
3213:   textNodeSearchTargets().forEach((root) => {
3214:     const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
3215:       acceptNode(node) {
3216:         if (!node.textContent || !normalize(node.textContent).includes(needle)) return
NodeFilter.FILTER_REJECT;
3217:         if (node.parentElement?.closest("script, style, input, textarea, select, button, .search-engine,
.site-header")) return NodeFilter.FILTER_REJECT;
3218:         return NodeFilter.FILTER_ACCEPT;
3219:       }
3220:     });
3221:     let node = walker.nextNode();
3222:     while (node) {
3223:       const haystack = normalize(node.textContent);
3224:       let index = haystack.indexOf(needle);
3225:       while (index >= 0) {
3226:         const range = document.createRange();
3227:         range.setStart(node, index);
3228:         range.setEnd(node, index + cleanQuery.length);
3229:         ranges.push(range);
3230:         index = haystack.indexOf(needle, index + Math.max(1, cleanQuery.length));
3231:       }
3232:       node = walker.nextNode();
3233:     }
3234:   });
```

```

3235:
3236:   if (ranges.length) window.CSS.highlights.set(searchHighlightName, new window.Highlight(...ranges));
3237:   return ranges.length;
3238: }

```

queueSearchHighlights (script.js, lines 3240-3243)

queueSearchHighlights part of the public website runtime. In this file, it appears around lines 3240-3243 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applySearchHighlights. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3240: function queueSearchHighlights() {
3241:   clearTimeout(searchHighlightTimer);
3242:   searchHighlightTimer = setTimeout(() => applySearchHighlights(searchInput?.value || ""), 30);
3243: }

```

ensureSearchPanel (script.js, lines 3245-3283)

ensureSearchPanel part of the public website runtime. In this file, it appears around lines 3245-3283 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, replaceWith, append, querySelector, addEventListener, updateSearchDropdown, preventDefault, closest, and goToSearchResult. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3246: if (!searchInput || !searchWrap || searchPanel) return;; line 3279: if (!button) return;.

Important local values include shell at line 3247; button at line 3278; result at line 3280. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3245: function ensureSearchPanel() {
3246:   if (!searchInput || !searchWrap || searchPanel) return;
3247:   const shell = document.createElement("div");
3248:   shell.className = "search-engine";
3249:   searchWrap.replaceWith(shell);
3250:   shell.append(searchWrap);
3251:   searchPanel = document.createElement("div");
3252:   searchPanel.className = "search-results-panel";
3253:   searchPanel.hidden = true;
3254:   searchPanel.innerHTML = `
3255:     <div class="search-results-tools">
3256:       <span id="search-status">Type to search projects, files, sections, and phrases.</span>
3257:       <label>
3258:         <span>Show</span>
3259:         <select id="search-result-limit" aria-label="Number of search results">
3260:           <option value="5">5</option>
3261:           <option value="10" selected>10</option>
3262:           <option value="15">15</option>
3263:           <option value="20">20</option>
3264:         </select>
3265:       </label>
3266:     </div>
3267:     <div class="search-results-list" role="listbox" aria-label="Search results"></div>
3268:   `;
3269:   shell.append(searchPanel);
3270:   searchStatus = searchPanel.querySelector("#search-status");
3271:   searchLimitSelect = searchPanel.querySelector("#search-result-limit");
3272:   searchLimitSelect.addEventListener("change", () => {
3273:     searchResultLimit = Number(searchLimitSelect.value) || 10;
3274:     updateSearchDropdown();
3275:   });
3276:   searchPanel.addEventListener("mousedown", (event) => event.preventDefault());
3277:   searchPanel.addEventListener("click", (event) => {
3278:     const button = event.target.closest("[data-search-result-index]");
3279:     if (!button) return;
3280:     const result = currentSearchResults[Number(button.dataset.searchResultIndex)];

```

```

3281:     if (result) goToSearchResult(result);
3282:   });
3283: }

```

setSearchMessage (script.js, lines 3285-3289)

setSearchMessage part of the public website runtime. In this file, it appears around lines 3285-3289 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3286: if (!searchStatus) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3285: function setSearchMessage(message, state = "neutral") {
3286:   if (!searchStatus) return;
3287:   searchStatus.textContent = message;
3288:   searchStatus.dataset.state = state;
3289: }

```

renderSearchResults (script.js, lines 3291-3327)

renderSearchResults renders ui, markup, or display output. In this file, it appears around lines 3291-3327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSearchPanel, querySelector, trim, searchResultsFor, slice, setAttribute, setSearchMessage, removeAttribute, map, highlightQueryHtml, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3293: if (!searchPanel) return;; line 3305: return;; line 3312: return;;.

Important local values include list at line 3294; cleanQuery at line 3295; limitedResults at line 3297; visibleText at line 3316. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3291: function renderSearchResults(query, visibleCount) {
3292:   ensureSearchPanel();
3293:   if (!searchPanel) return;
3294:   const list = searchPanel.querySelector(".search-results-list");
3295:   const cleanQuery = String(query || "").trim();
3296:   currentSearchResults = searchResultsFor(cleanQuery);
3297:   const limitedResults = currentSearchResults.slice(0, searchResultLimit);
3298:   searchPanel.hidden = !cleanQuery;
3299:   searchInput.setAttribute("aria-expanded", String(Boolean(cleanQuery)));
3300:
3301:   if (!cleanQuery) {
3302:     list.innerHTML = "";
3303:     setSearchMessage("Type to search projects, files, sections, and phrases.");
3304:     searchInput.removeAttribute("aria-invalid");
3305:     return;
3306:   }
3307:
3308:   if (!currentSearchResults.length) {
3309:     list.innerHTML = "";
3310:     setSearchMessage(`"${cleanQuery}" was not found on this portfolio.`, "not-found");
3311:     searchInput.setAttribute("aria-invalid", "true");
3312:     return;
3313:   }
3314:
3315:   searchInput.removeAttribute("aria-invalid");
3316:   const visibleText = visibleCount
3317:     ? `${visibleCount} visible match${visibleCount === 1 ? "" : "es"} highlighted.`
3318:     : "Matches found elsewhere. Choose a result below.";
3319:   setSearchMessage(`${visibleText} Showing ${limitedResults.length} of ${currentSearchResults.length}.`,
"found");
3320:   list.innerHTML = limitedResults.map((result, index) => `
3321:     <button class="search-result-item" type="button" role="option" data-search-result-index="${index}">
3322:       <span class="search-result-title">${highlightQueryHtml(result.title || "Untitled result",
cleanQuery)}</span>
3323:       <span class="search-result-meta">${escapeHtml([result.type, result.context].filter(Boolean).join("
/ "))}</span>
3324:       ${searchSnippet(result.text, cleanQuery)} ? `<span class="search-result-

```

```

snippet">${highlightQueryHtml(searchSnippet(result.text, cleanQuery), cleanQuery)}</span>` : ""}
3325:   </button>
3326:   `).join("");
3327: }

```

updateSearchDropdown (script.js, lines 3329-3333)

updateSearchDropdown updates ui, state, cache, or stored data. In this file, it appears around lines 3329-3333 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applySearchHighlights, and renderSearchResults. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include query at line 3330; visibleCount at line 3331. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3329: function updateSearchDropdown() {
3330:   const query = searchInput?.value || "";
3331:   const visibleCount = applySearchHighlights(query);
3332:   renderSearchResults(query, visibleCount);
3333: }

```

showSearchPanellIfNeeded (script.js, lines 3335-3338)

showSearchPanellIfNeeded controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3335-3338 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3336: if (!searchInput?.value?.trim()) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3335: function showSearchPanelIfNeeded() {
3336:   if (!searchInput?.value?.trim()) return;
3337:   updateSearchDropdown();
3338: }

```

scrollToDomTarget (script.js, lines 3340-3352)

scrollToDomTarget part of the public website runtime. In this file, it appears around lines 3340-3352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getElementById, closeSectionDialogForRoute, scrollIntoView, replaceState, and queueSearchHighlights. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3342: if (!target) return false;; line 3351: return true;.

Important local values include target at line 3341. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3340: function scrollToDomTarget(id) {
3341:   const target = id ? document.getElementById(id) : null;
3342:   if (!target) return false;
3343:   closeSectionDialogForRoute();
3344:   target.scrollIntoView({ behavior: "smooth", block: "start" });
3345:   try {
3346:     history.replaceState(history.state || { appWindow: "portfolio-base" }, "", `#${id}`);
3347:   } catch {
3348:     // Scrolling is still the important behavior.

```

```

3349:   }
3350:   queueSearchHighlights();
3351:   return true;
3352: }

```

goToSearchResult (script.js, lines 3354-3377)

goToSearchResult part of the public website runtime. In this file, it appears around lines 3354-3377 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references scrollToDomTarget, openParsedSection, queueSearchHighlights, getElementById, setActiveFilter, renderProjects, scrollIntoView, and open. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3355: if (!result) return;; line 3357: if (result.domTarget && scrollToDomTarget(result.domTarget)) return;; line 3362: return;; line 3373: return;.

Important local values include target at line 3370. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3354: function goToSearchResult(result) {
3355:   if (!result) return;
3356:   searchPanel.hidden = true;
3357:   if (result.domTarget && scrollToDomTarget(result.domTarget)) return;
3358:
3359:   if (result.projectId && result.sectionIndex !== undefined) {
3360:     openParsedSection(result.projectId, result.sectionIndex, result.resourcePath || "");
3361:     queueSearchHighlights();
3362:     return;
3363:   }
3364:
3365:   if (result.projectId) {
3366:     if (!document.getElementById(result.projectId)) {
3367:       setActiveFilter("all");
3368:       renderProjects();
3369:     }
3370:     const target = document.getElementById(result.projectId);
3371:     target?.scrollIntoView({ behavior: "smooth", block: "center" });
3372:     queueSearchHighlights();
3373:     return;
3374:   }
3375:
3376:   if (result.url) window.open(result.url, "_blank", "noopener");
3377: }

```

handleSearchInput (script.js, lines 3379-3382)

handleSearchInput handles an event, request, command, or user action. In this file, it appears around lines 3379-3382 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderProjects, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3379: function handleSearchInput() {
3380:   renderProjects();
3381:   updateSearchDropdown();
3382: }

```

renderProjects (script.js, lines 3384-3409)

renderProjects renders ui, markup, or display output. In this file, it appears around lines 3384-3409 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, trim, filter, projectVisible, queueSearchHighlights, map, categorySection, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3398: return;; line 3405: return visibleProjects.length || shouldShowEmptyCategory ? categorySection(category, visibleProjects) : "";

Important local values include query at line 3385; visible at line 3386; visibleProjects at line 3403; shouldShowEmptyCategory at line 3404. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3384: function renderProjects() {
3385:   const query = normalize(searchInput.value).trim();
3386:   const visible = projects.filter((project) => projectVisible(project, query));
3387:
3388:   projectCount.textContent = projects.length;
3389:
3390:   if (!visible.length) {
3391:     grid.innerHTML = `
3392:       <div class="empty-state">
3393:         <h3>No projects match that view.</h3>
3394:         <p>Search covers categories, project names, documents, tests, PCB builds, tools, languages, and
links.</p>
3395:       </div>
3396:     `;
3397:     queueSearchHighlights();
3398:     return;
3399:   }
3400:
3401:   grid.innerHTML = categories
3402:     .map((category) => {
3403:       const visibleProjects = visible.filter((project) => project.category === category.id);
3404:       const shouldShowEmptyCategory = !query && (activeFilter === "all" || activeFilter === category.id);
3405:       return visibleProjects.length || shouldShowEmptyCategory ? categorySection(category,
visibleProjects) : "";
3406:     })
3407:     .join("");
3408:   queueSearchHighlights();
3409: }
```

ensureSectionDialog (script.js, lines 3411-3470)

ensureSectionDialog part of the public website runtime. In this file, it appears around lines 3411-3470 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, createElement, append, addEventListener, closeSectionDialog, clearSectionRouteInHistory, remove, updateSectionDialogMinimize, focus, pathFromString, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 3413: if (dialog) return dialog;; line 3449: return;; line 3460: if (!nextPath) return;; line 3466: if (!nodeButton) return;. There are 1 additional return statements in the function body.

Important local values include dialog at line 3412; returnTarget at line 3440; path at line 3446; stack at line 3451; stack at line 3458; nextPath at line 3459; nodeButton at line 3465. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3411: function ensureSectionDialog() {
3412:   let dialog = document.querySelector("#section-view-dialog");
3413:   if (dialog) return dialog;
3414:
3415:   dialog = document.createElement("dialog");
3416:   dialog.id = "section-view-dialog";
3417:   dialog.className = "section-view-dialog";
3418:   dialog.innerHTML = `
3419:     <div class="section-view-shell">
3420:       <div class="section-view-heading">
3421:         <div class="section-view-titlebar">
3422:           <button class="section-view-back" type="button" title="Previous view" aria-label="Previous
view" hidden>&larr;</button>
3423:           <button class="section-view-forward" type="button" title="Next view" aria-label="Next view"
hidden>&rarr;</button>
3424:           <h2 id="section-view-title">Section</h2>
3425:         </div>
3426:         <div class="section-view-actions">
3427:           <button class="section-view-close" type="button" aria-label="Close section">&times;</button>
3428:         </div>
3429:       </div>
3430:       <div class="section-view-content" id="section-view-content"></div>
3431:     </div>
```

```

3432: `;
3433: document.body.append(dialog);
3434: dialog.querySelector(".section-view-close").addEventListener("click", () =>
closeSectionDialog(dialog));
3435: dialog.addEventListener("cancel", () => clearSectionRouteInHistory());
3436: dialog.addEventListener("close", () => {
3437:     document.body.classList.remove("full-window-open");
3438:     dialog.classList.remove("is-minimized-dialog");
3439:     updateSectionDialogMinimize(dialog);
3440:     const returnTarget = document.querySelector(
3441:         `[data-section-project="${dialog.dataset.projectId}"][data-section-
index="${dialog.dataset.sectionIndex}"]`
3442:     );
3443:     returnTarget?.focus({ preventScroll: true });
3444: });
3445: dialog.querySelector(".section-view-back").addEventListener("click", () => {
3446:     const path = pathFromString(dialog.dataset.resourcePath || "");
3447:     if (!path.length) {
3448:         closeSectionDialog(dialog);
3449:         return;
3450:     }
3451:     const stack = sectionForwardStack(dialog);
3452:     stack.push(pathToString(path));
3453:     dialog.dataset.forwardStack = JSON.stringify(stack);
3454:     path.pop();
3455:     openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, pathToString(path), {
historyMode: "replace", preserveForward: true });
3456: });
3457: dialog.querySelector(".section-view-forward").addEventListener("click", () => {
3458:     const stack = sectionForwardStack(dialog);
3459:     const nextPath = stack.pop();
3460:     if (!nextPath) return;
3461:     dialog.dataset.forwardStack = JSON.stringify(stack);
3462:     openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, nextPath, { historyMode:
"replace", preserveForward: true });
3463: });
3464: dialog.querySelector("#section-view-content").addEventListener("click", (event) => {
3465:     const nodeButton = event.target.closest("[data-resource-path]");
3466:     if (!nodeButton) return;
3467:     openParsedSection(nodeButton.dataset.sectionProject, nodeButton.dataset.sectionIndex,
nodeButton.dataset.resourcePath);
3468: });
3469: return dialog;
3470: }

```

sectionForwardStack (script.js, lines 3472-3478)

sectionForwardStack part of the public website runtime. In this file, it appears around lines 3472-3478 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3474: return JSON.parse(dialog.dataset.forwardStack || "[]"); line 3476: return [];

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3472: function sectionForwardStack(dialog) {
3473:     try {
3474:         return JSON.parse(dialog.dataset.forwardStack || "[]");
3475:     } catch {
3476:         return [];
3477:     }
3478: }

```

updateSectionNavigation (script.js, lines 3480-3486)

updateSectionNavigation updates ui, state, cache, or stored data. In this file, it appears around lines 3480-3486 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, setAttribute, and sectionForwardStack. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include back at line 3481. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3480: function updateSectionNavigation(dialog, path) {
3481:   const back = dialog.querySelector(".section-view-back");
3482:   back.hidden = false;
3483:   back.title = path.length ? "Previous view" : "Back to projects";
3484:   back.setAttribute("aria-label", path.length ? "Previous view" : "Back to projects");
3485:   dialog.querySelector(".section-view-forward").hidden = !sectionForwardStack(dialog).length;
3486: }
```

closeSectionDialog (script.js, lines 3488-3491)

closeSectionDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3488-3491 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references close, and clearSectionRouteInHistory. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3488: function closeSectionDialog(dialog) {
3489:   dialog.close();
3490:   clearSectionRouteInHistory();
3491: }
```

closeOrStepBackSectionDialog (script.js, lines 3493-3504)

closeOrStepBackSectionDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3493-3504 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathFromString, sectionForwardStack, push, pathToString, stringify, pop, openParsedSection, and closeSectionDialog. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3501: return;.

Important local values include path at line 3494; stack at line 3496. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3493: function closeOrStepBackSectionDialog(dialog) {
3494:   const path = pathFromString(dialog.dataset.resourcePath || "");
3495:   if (path.length) {
3496:     const stack = sectionForwardStack(dialog);
3497:     stack.push(pathToString(path));
3498:     dialog.dataset.forwardStack = JSON.stringify(stack);
3499:     path.pop();
3500:     openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, pathToString(path), {
historyMode: "replace", preserveForward: true });
3501:     return;
3502:   }
3503:   closeSectionDialog(dialog);
3504: }
```

openParsedSection (script.js, lines 3506-3532)

This function opens a recruiter-facing section window from parsed project data. It handles route state, section history, content rendering, and navigation controls.

The public website uses clickable sections instead of one long flat project page. This function drives that section-window experience.

It calls or references find, filter, sectionHasRenderableContent, pathFromString, nodeAtPath, nodeHasRenderableContent, ensureSectionDialog, applyProjectTemplateToElement, pathToString, syncSectionRouteToHistory, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3510: if (!section) return false;; line 3513: if (!node) return false;; line 3514: if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return false;; line 3531: return true;.

Important local values include project at line 3507; sections at line 3508; section at line 3509; path at line 3511; node at line 3512; dialog at line 3516; category at line 3517. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3506: function openParsedSection(projectId, sectionIndex, resourcePath = "", options = {}) {
3507:   const project = projects.find((item) => item.id === projectId);
3508:   const sections = (project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
3509:   const section = sections[Number(sectionIndex)];
3510:   if (!section) return false;
3511:   const path = pathFromString(resourcePath);
3512:   const node = path.length ? nodeAtPath(section, path) : section;
3513:   if (!node) return false;
3514:   if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return
false;
3515:
3516:   const dialog = ensureSectionDialog();
3517:   const category = categories.find((item) => item.id === project.category) || {};
3518:   applyProjectTemplateToElement(dialog, project, category.accent || "#117c7a");
3519:   dialog.dataset.projectId = projectId;
3520:   dialog.dataset.sectionIndex = String(sectionIndex);
3521:   dialog.dataset.resourcePath = pathToString(path);
3522:   if (!options.preserveForward) dialog.dataset.forwardStack = "[]";
3523:   syncSectionRouteToHistory(projectId, sectionIndex, pathToString(path), options.historyMode || "push");
3524:   dialog.querySelector("#section-view-title").textContent = displayTitle(node.title || section.title,
"Section");
3525:   updateSectionNavigation(dialog, path);
3526:   dialog.querySelector("#section-view-content").innerHTML = parsedSectionContent(section, projectId,
Number(sectionIndex), path);
3527:   document.body.classList.add("full-window-open");
3528:   if (!dialog.open) dialog.showModal();
3529:   dialog.scrollTop = 0;
3530:   queueSearchHighlights();
3531:   return true;
3532: }
```

sections (script.js, lines 3508-3514)

sections part of the public website runtime. In this file, it appears around lines 3508-3514 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, sectionHasRenderableContent, pathFromString, nodeAtPath, and nodeHasRenderableContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3510: if (!section) return false;; line 3513: if (!node) return false;; line 3514: if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return false;.

Important local values include sections at line 3508; section at line 3509; path at line 3511; node at line 3512. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3508:   const sections = (project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
3509:   const section = sections[Number(sectionIndex)];
3510:   if (!section) return false;
3511:   const path = pathFromString(resourcePath);
3512:   const node = path.length ? nodeAtPath(section, path) : section;
3513:   if (!node) return false;
3514:   if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return
false;
```

closeSectionDialogForRoute (script.js, lines 3534-3537)

closeSectionDialogForRoute controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3534-3537 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and close. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include `dialog` at line 3535. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3534: function closeSectionDialogForRoute() {
3535:   const dialog = document.querySelector("#section-view-dialog");
3536:   if (dialog?.open) dialog.close();
3537: }
```

applySectionRoute (script.js, lines 3539-3555)

`applySectionRoute` part of the public website runtime. In this file, it appears around lines 3539-3555 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `closeSectionDialogForRoute`, and `openParsedSection`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3544: `return;`.

Important local values include `opened` at line 3547. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3539: function applySectionRoute(route) {
3540:   isApplyingSectionRoute = true;
3541:   try {
3542:     if (!route) {
3543:       closeSectionDialogForRoute();
3544:       return;
3545:     }
3546:
3547:     const opened = openParsedSection(route.projectId, route.sectionIndex, route.resourcePath, {
3548:       historyMode: "none",
3549:       preserveForward: true
3550:     });
3551:     if (!opened) closeSectionDialogForRoute();
3552:   } finally {
3553:     isApplyingSectionRoute = false;
3554:   }
3555: }
```

restoreSectionRouteAfterCatalogLoad (script.js, lines 3557-3560)

`restoreSectionRouteAfterCatalogLoad` part of the public website runtime. In this file, it appears around lines 3557-3560 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `initializeSectionHistoryState`, `applySectionRoute`, `sectionRouteFromLocation`, and `sectionRouteFromState`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3557: function restoreSectionRouteAfterCatalogLoad() {
3558:   initializeSectionHistoryState();
3559:   applySectionRoute(sectionRouteFromLocation() || sectionRouteFromState(history.state));
3560: }
```

renderInlineSectionFiles (script.js, lines 3621-3639)

`renderInlineSectionFiles` renders ui, markup, or display output. In this file, it appears around lines 3621-3639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, map, normalizeDownloadAsset, filter, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, downloadAttribute, fileNameFromUrl, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3623: if (!downloads.length) return ""; line 3625: return `.

Important local values include downloads at line 3622. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3621: function renderInlineSectionFiles(items = []) {
3622:   const downloads = uniqueDownloads(items.map((item) => normalizeDownloadAsset(item,
"File")).filter(Boolean));
3623:   if (!downloads.length) return "";
3624:
3625:   return `
3626:   <div class="section-file-list">
3627:     ${downloads.map((item) => `
3628:       <a
3629:         class="section-file-link"
3630:         href="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item, item.url)
3631:           ${linkAttributes(item.url, item)}
3632:           ${downloadAttribute(item.url, item)}
3633:         >
3634:           ${escapeHtml(item.title || fileNameFromUrl(item.url))}
3635:         </a>
3636:       `).join("")}
3637:   </div>
3638: `;
3639: }
```

normalizeTextColor (script.js, lines 3640-3647)

normalizeTextColor validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 3640-3647 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3642: return /^#([0-9a-f]{3}|[0-9a-f]{6})\$/i.test(color) ||.

Important local values include color at line 3641. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3640: function normalizeTextColor(value = "") {
3641:   const color = String(value || "").trim();
3642:   return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||
3643:     /^rgba?\(/i.test(color) ||
3644:     /^hsla?\(/i.test(color)
3645:     ? color
3646:     : "";
3647: }
```

loadProjectCatalog (script.js, lines 3648-3708)

loadProjectCatalog loads data from disk, network, browser storage, or a service. In this file, it appears around lines 3648-3708 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references hydrateProjectCategories, normalizeFieldStyles, normalizeSiteContent, normalizeProfile, normalizeFunFacts, renderProfile, renderSiteContent, renderFunFacts, renderSiteSections, renderCategoryFilters, and 11 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3669: return;; line 3677: return response.json();.

Important local values include catalogUrl at line 3672. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3648: function loadProjectCatalog() {
3649:   if (window.__PORTFOLIO_CATALOG__) {
3650:     projects = window.__PORTFOLIO_CATALOG__.projects || [];
3651:     categories = hydrateProjectCategories(window.__PORTFOLIO_CATALOG__.categories || [], projects);
3652:     siteSections = window.__PORTFOLIO_CATALOG__.siteSections || [];
3653:     fieldStyles = normalizeFieldStyles(window.__PORTFOLIO_CATALOG__.fieldStyles || {});
3654:     siteContent = normalizeSiteContent(window.__PORTFOLIO_CATALOG__.siteContent || {});
3655:     siteContentRich = window.__PORTFOLIO_CATALOG__.siteContentRich || {};
3656:     profile = normalizeProfile(window.__PORTFOLIO_CATALOG__.profile || {});
3657:     profileRich = window.__PORTFOLIO_CATALOG__.profileRich || {};
3658:     funFacts = normalizeFunFacts(window.__PORTFOLIO_CATALOG__.funFacts || []);
3659:     funFactsRich = window.__PORTFOLIO_CATALOG__.funFactsRich || null;
3660:     renderProfile();
3661:     renderSiteContent();
3662:     renderFunFacts();
3663:     renderSiteSections();
3664:     renderCategoryFilters();
3665:     renderProjects();
3666:     rebuildSearchIndex();
3667:     updateSearchDropdown();
3668:     restoreSectionRouteAfterCatalogLoad();
3669:     return;
3670:   }
3671:
3672:   const catalogUrl = new URL("projects.json", window.location.href);
3673:   catalogUrl.searchParams.set("v", String(Date.now()));
3674:   fetch(catalogUrl.href, { cache: "no-store" })
3675:     .then((response) => {
3676:       if (!response.ok) throw new Error("Could not load projects.json");
3677:       return response.json();
3678:     })
3679:     .then((data) => {
3680:       projects = data.projects || [];
3681:       categories = hydrateProjectCategories(data.categories || [], projects);
3682:       siteSections = data.siteSections || [];
3683:       fieldStyles = normalizeFieldStyles(data.fieldStyles || {});
3684:       siteContent = normalizeSiteContent(data.siteContent || {});
3685:       siteContentRich = data.siteContentRich || {};
3686:       profile = normalizeProfile(data.profile || {});
3687:       profileRich = data.profileRich || {};
3688:       funFacts = normalizeFunFacts(data.funFacts || []);
3689:       funFactsRich = data.funFactsRich || null;
3690:       renderProfile();
3691:       renderSiteContent();
3692:       renderFunFacts();
3693:       renderSiteSections();
3694:       renderCategoryFilters();
3695:       renderProjects();
3696:       rebuildSearchIndex();
3697:       updateSearchDropdown();
3698:       restoreSectionRouteAfterCatalogLoad();
3699:     })
3700:     .catch(() => {
3701:       grid.innerHTML = `
3702:         <div class="empty-state">
3703:           <h3>Project data did not load.</h3>
3704:           <p>The project catalog is temporarily unavailable.</p>
3705:         </div>
3706:       `;
3707:     });
3708: }

```

Representative opening snippet

```

1: const grid = document.querySelector("#project-grid");
2: const searchInput = document.querySelector("#project-search");
3: const projectFilters = document.querySelector("#project-filters");
4: let filterButtons = [...document.querySelectorAll(".filter-button")];
5: const projectCount = document.querySelector("#project-count");
6: const projectTrackCount = document.querySelector("#project-track-count");
7: const projectTrackLabels = document.querySelector("#project-track-labels");
8: const year = document.querySelector("#year");
9: const funFactsCallout = document.querySelector("#fun-facts-callout");
10: const heroEyebrow = document.querySelector(".hero-content .eyebrow");
11: const heroTitle = document.querySelector("#hero-title");
12: const heroCopy = document.querySelector(".hero-copy");
13: const brandName = document.querySelector(".brand strong");
14: const brandSubtitle = document.querySelector(".brand small");
15: const brandIcon = document.querySelector(".brand-icon");
16: const brandText = document.querySelector(".omb-engraving");
17: const headerAvatar = document.querySelector(".header-avatar");
18: const headerAvatarImage = document.querySelector(".header-avatar img");
19: const heroImage = document.querySelector(".hero-image");
20: const resumeSection = document.querySelector("#resume");
21: const contactBand = document.querySelector("#contact");
22: const profilePhoto = document.querySelector(".profile-photo");
23: const contactTitle = document.querySelector("#contact-title");
24: const contactIntro = document.querySelector(".contact-intro");
25: const contactDetails = document.querySelector(".contact-details");
26: const contactLinks = document.querySelector(".contact-links");

```

```

27: const footerOwner = document.querySelector(".site-footer span");
28: const searchWrap = searchInput?.closest(".search-wrap");
29: const aiAssistantForm = document.querySelector("#ai-assistant-form");
30: const aiAssistantPanel = document.querySelector("#ask-ai");
31: const aiAssistantInput = document.querySelector("#ai-assistant-input");
32: const aiAssistantLog = document.querySelector("#ai-assistant-log");
33: const aiAssistantStatus = document.querySelector("#ai-assistant-status");
34: const aiClearChatButton = document.querySelector("#ai-clear-chat");
35:
36: let categories = [];
37: let projects = [];
38: let siteSections = [];
39: let funFacts = [];
40: let funFactsRich = null;
41: let fieldStyles = {};
42: let siteContent = null;
43: let siteContentRich = {};
44: let profile = null;
45: let profileRich = {};
46: let activeFilter = "all";
47: let activeSectionDialogDrag = null;
48: let activeSectionDialogResize = null;
49: let sectionDialogDragEnabled = false;
50: let isApplyingSectionRoute = false;
51: let searchPanel = null;
52: let searchStatus = null;
53: let searchLimitSelect = null;
54: let currentSearchResults = [];
55: let assistantSourceCounter = 0;
56: let assistantSourceMap = new Map();
57: let assistantChatHistory = [];
58: let searchableEntries = [];
59: let searchResultLimit = 10;
60: let searchHighlightTimer = 0;
61: const searchHighlightName = "portfolio-search-highlight";
62:
63: const sectionRouteKey = "portfolio-section";
64: const sectionRouteParams = {
65:   path: "portfolioPath",
66:   project: "portfolioProject",
67:   section: "portfolioSection"
68: };
69:
70: const supportedCodeLanguages = [
71:   { id: "c", label: "C", aliases: ["c"] },
72:   { id: "cpp", label: "C++", aliases: ["cpp", "c++", "cplusplus"] },

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.